

CSE 4303

Data Structures

Topic: Binary Search Trees

Sabbir Ahmed

Assistant Prof | CSE | IUT

sabbirahmed@iut-dhaka.edu

*Let's assume a scenario where you need to
insert/update/search/remove values frequently*

	Search/ Update	Insert	Remove
Array	$O(N)$	$O(1)$ at end	$O(N)$

	Search/ Update	Insert	Remove
Array	$O(N)$	$O(1)$ at end	$O(N)$
Linked list	$O(N)$	$O(1)$ at end	$O(N)$

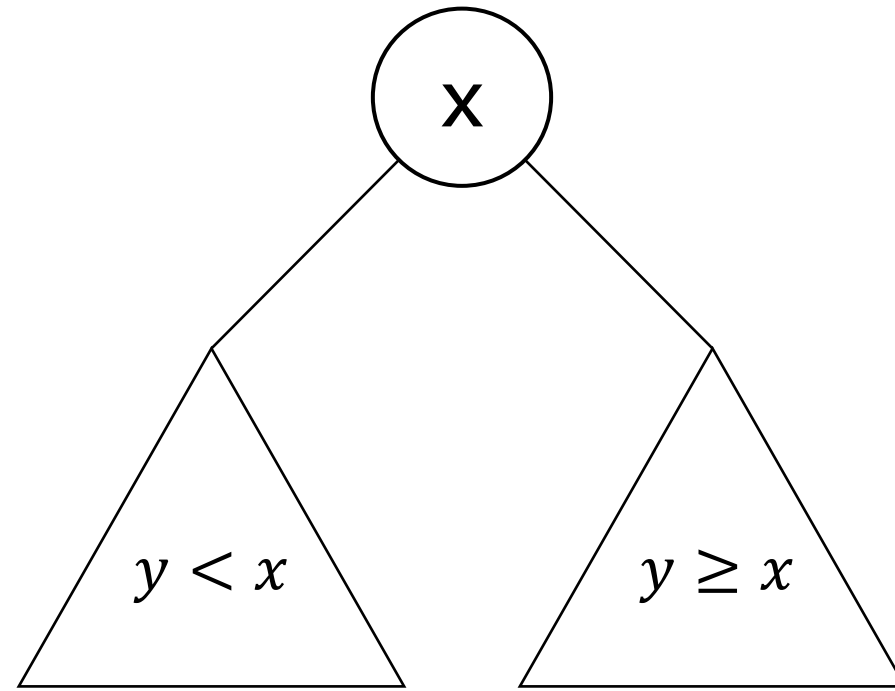
	Search/ Update	Insert	Remove
Array	$O(N)$	$O(1)$ at end	$O(N)$
Linked list	$O(N)$	$O(1)$ at end	$O(N)$
Sorted array	$O(?)$	$O(?)$	$O(?)$

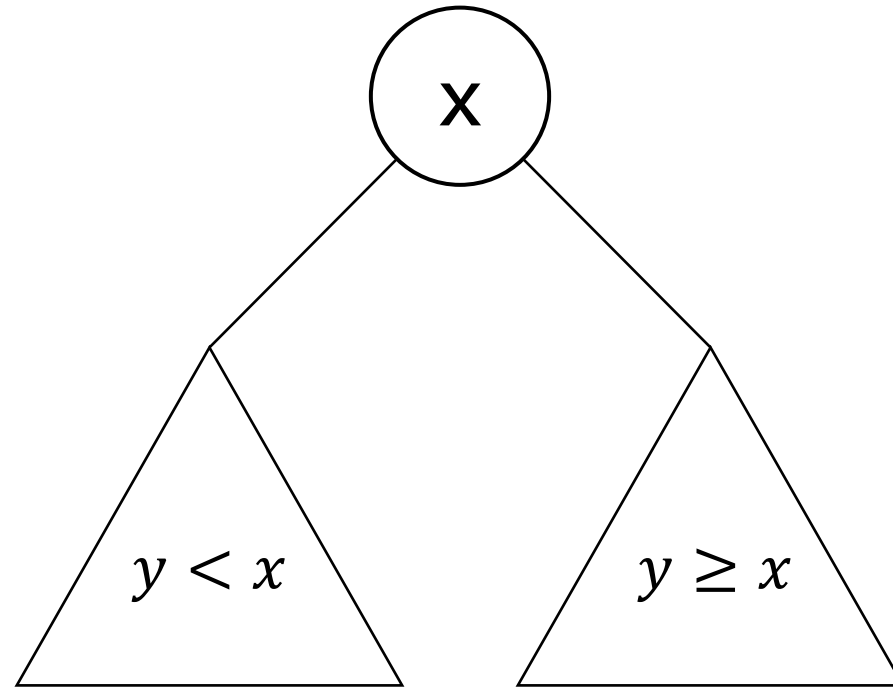
	Search/ Update	Insert	Remove
Array	$O(N)$	$O(1)$ at end	$O(N)$
Linked list	$O(N)$	$O(1)$ at end	$O(N)$
Sorted array	$O(\log N)$	$O(N)$	$O(N)$

	Search/ Update	Insert	Remove
Array	$O(N)$	$O(1)$ at end	$O(N)$
Linked list	$O(N)$	$O(1)$ at end	$O(N)$
Sorted array	$O(\log N)$	$O(N)$	$O(N)$
Binary search Tree (BST)	$O(\text{height})$	$O(\text{height})$	$O(\text{height})$

	Search/ Update	Insert	Remove
Array	$O(N)$	$O(1)$ at end	$O(N)$
Linked list	$O(N)$	$O(1)$ at end	$O(N)$
Sorted array	$O(\log N)$	$O(N)$	$O(N)$
Binary search Tree (BST)	$O(\text{height})$	$O(\text{height})$	$O(\text{height})$
Balanced BST	$O(\log N)$	$O(\log N)$	$O(\log N)$

*BST allows
insert/search/remove/
max/min/
successor/predecessor
in $O(\text{height})$*





For every node:

All nodes in the left subtree $<$ root of that tree

All nodes in the right subtree $\geq$ root of that tree

Insert (15)

Insert (20)

Insert (17)

Insert (10)

Insert (8)

Insert (2)

Insert (25)

Insert (23)

Insert (5)

Insert (12)

15

Insert (15)

Insert (20)

Insert (17)

Insert (10)

Insert (8)

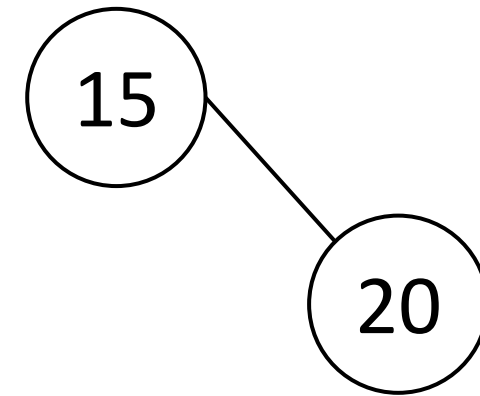
Insert (2)

Insert (25)

Insert (23)

Insert (5)

Insert (12)



Insert (15)

Insert (20)

Insert (17)

Insert (10)

Insert (8)

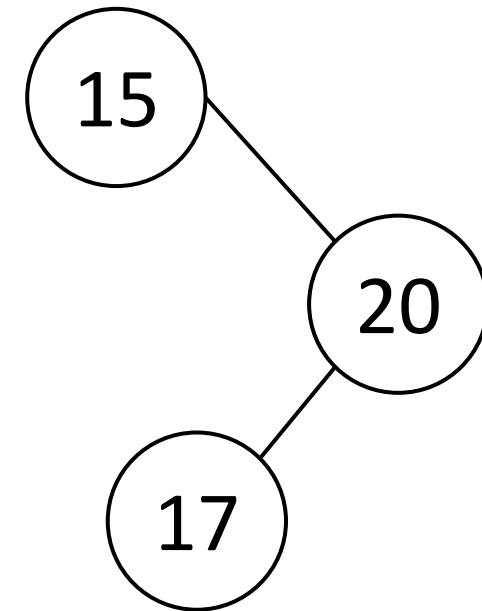
Insert (2)

Insert (25)

Insert (23)

Insert (5)

Insert (12)



Insert (15)

Insert (20)

Insert (17)

Insert (10)

Insert (8)

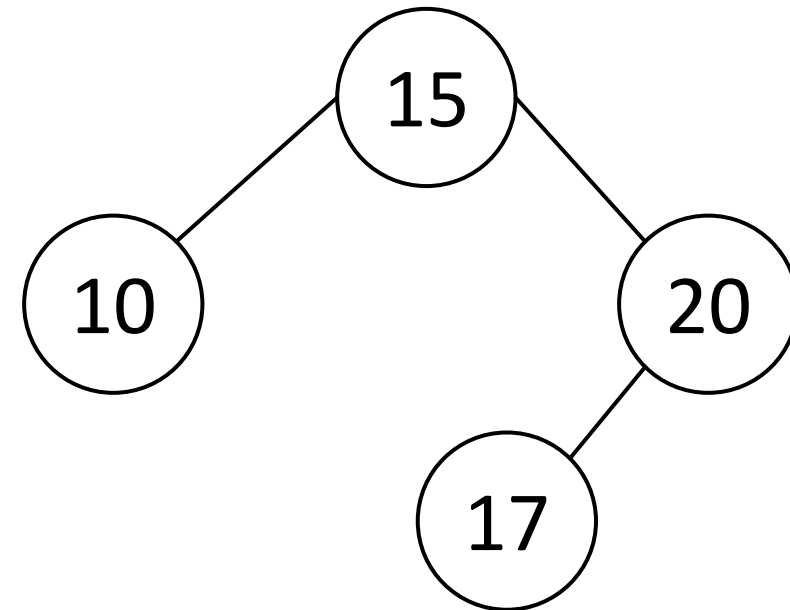
Insert (2)

Insert (25)

Insert (23)

Insert (5)

Insert (12)



Insert (15)

Insert (20)

Insert (17)

Insert (10)

Insert (8)

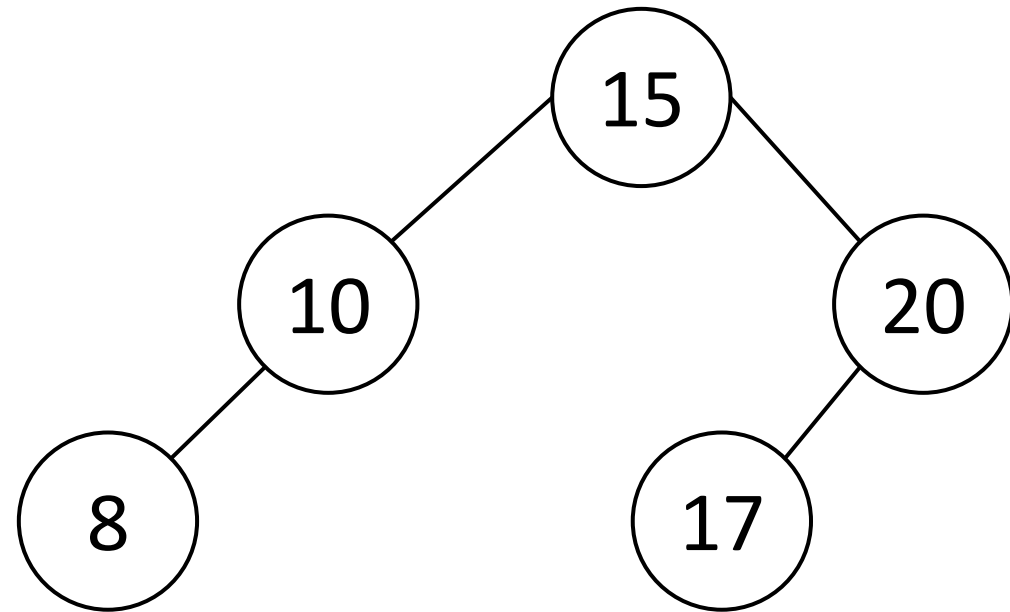
Insert (2)

Insert (25)

Insert (23)

Insert (5)

Insert (12)



Insert (15)

Insert (20)

Insert (17)

Insert (10)

Insert (8)

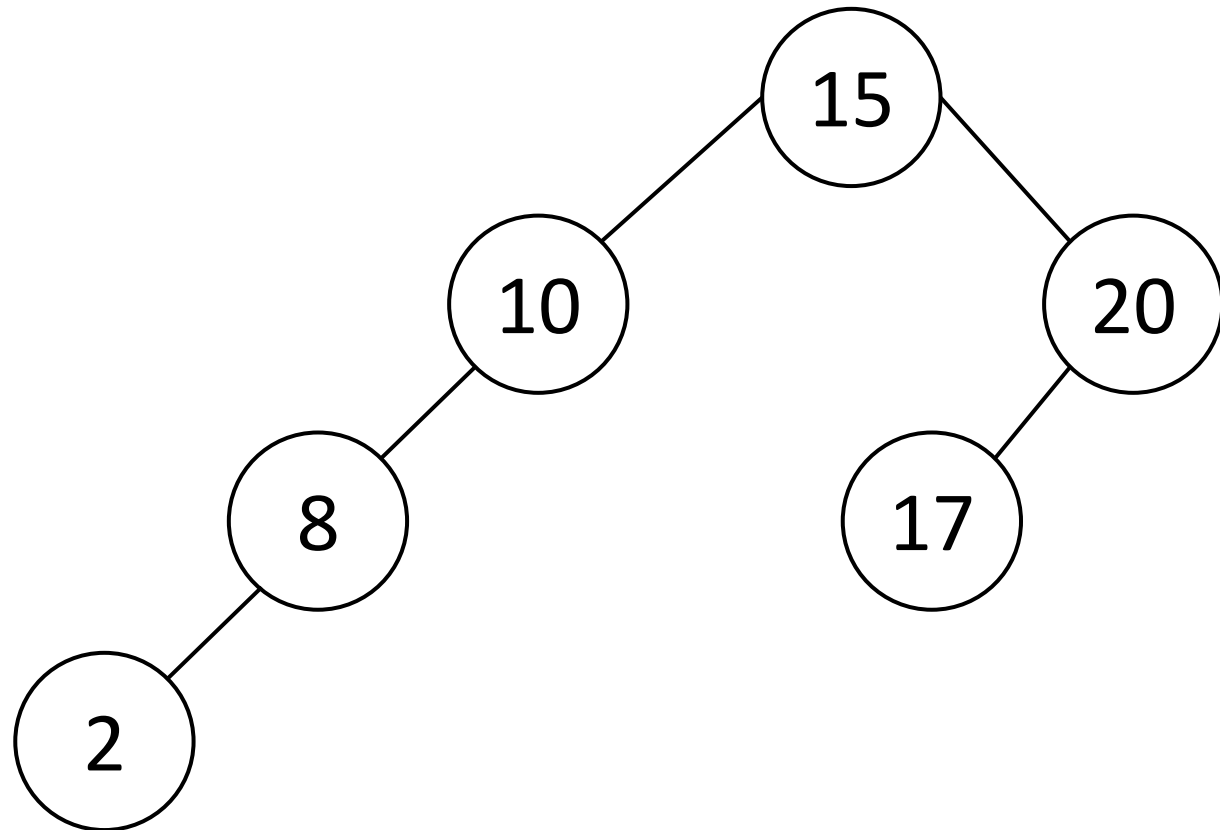
Insert (2)

Insert (25)

Insert (23)

Insert (5)

Insert (12)



Insert (15)

Insert (20)

Insert (17)

Insert (10)

Insert (8)

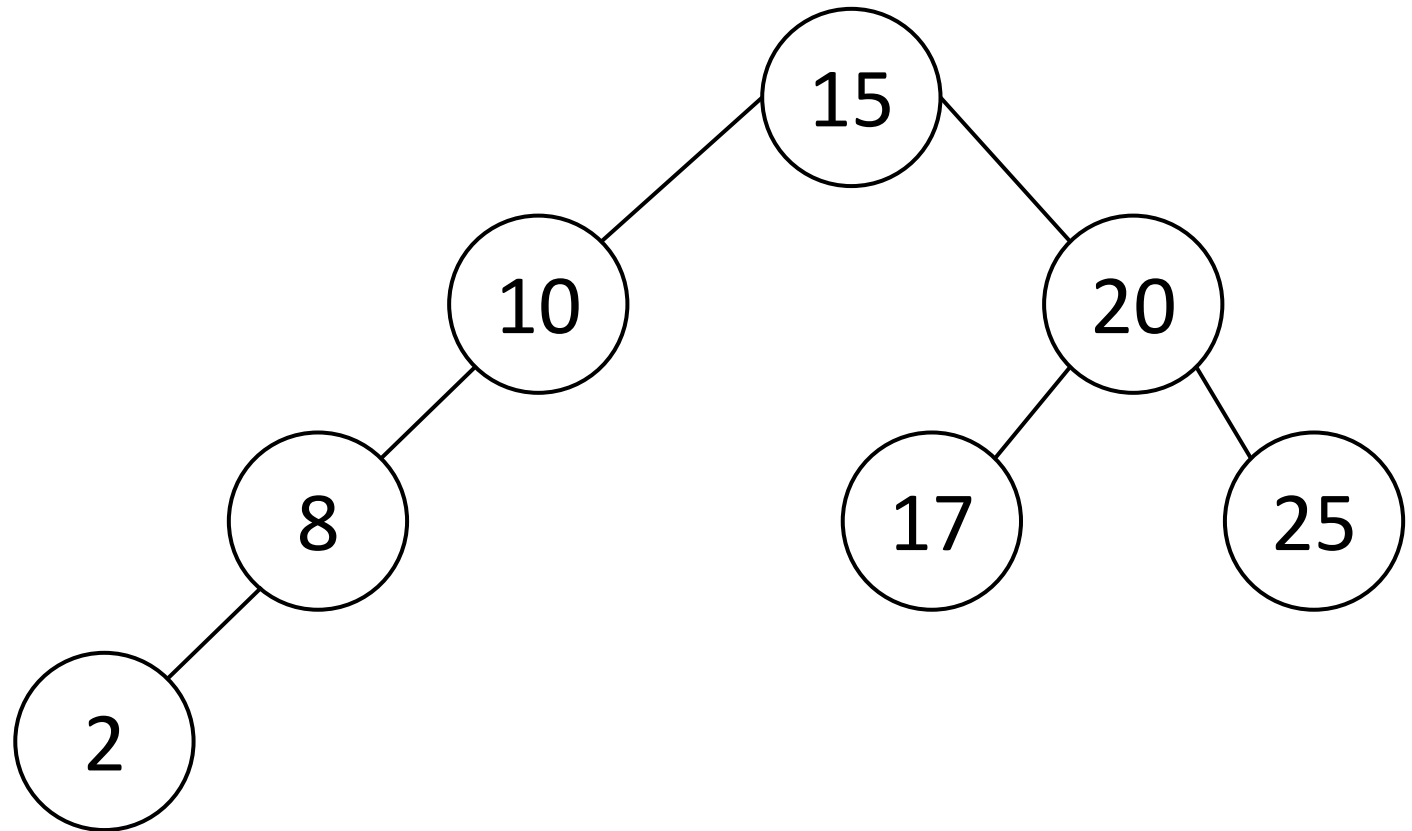
Insert (2)

Insert (25)

Insert (23)

Insert (5)

Insert (12)



Insert (15)

Insert (20)

Insert (17)

Insert (10)

Insert (8)

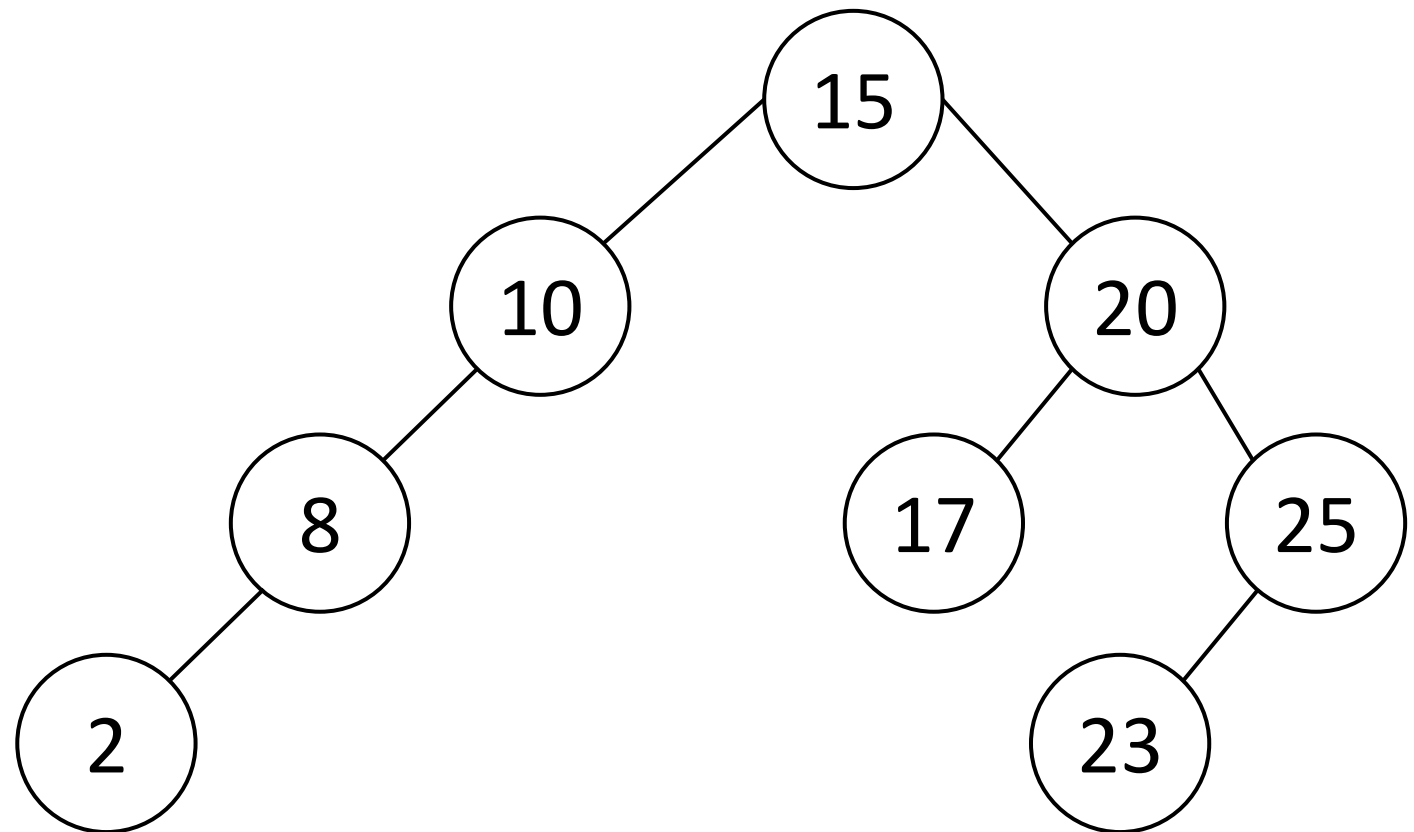
Insert (2)

Insert (25)

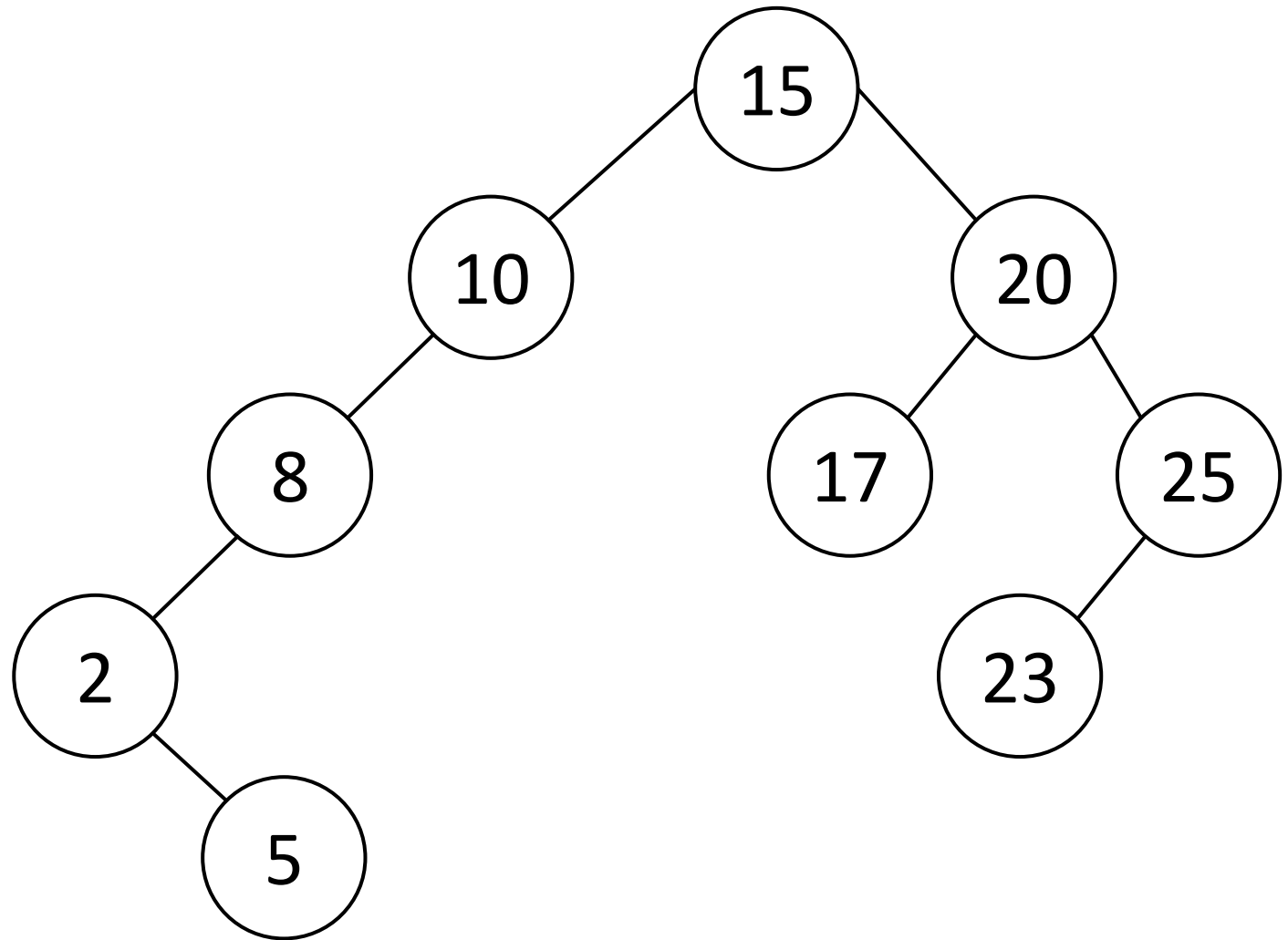
Insert (23)

Insert (5)

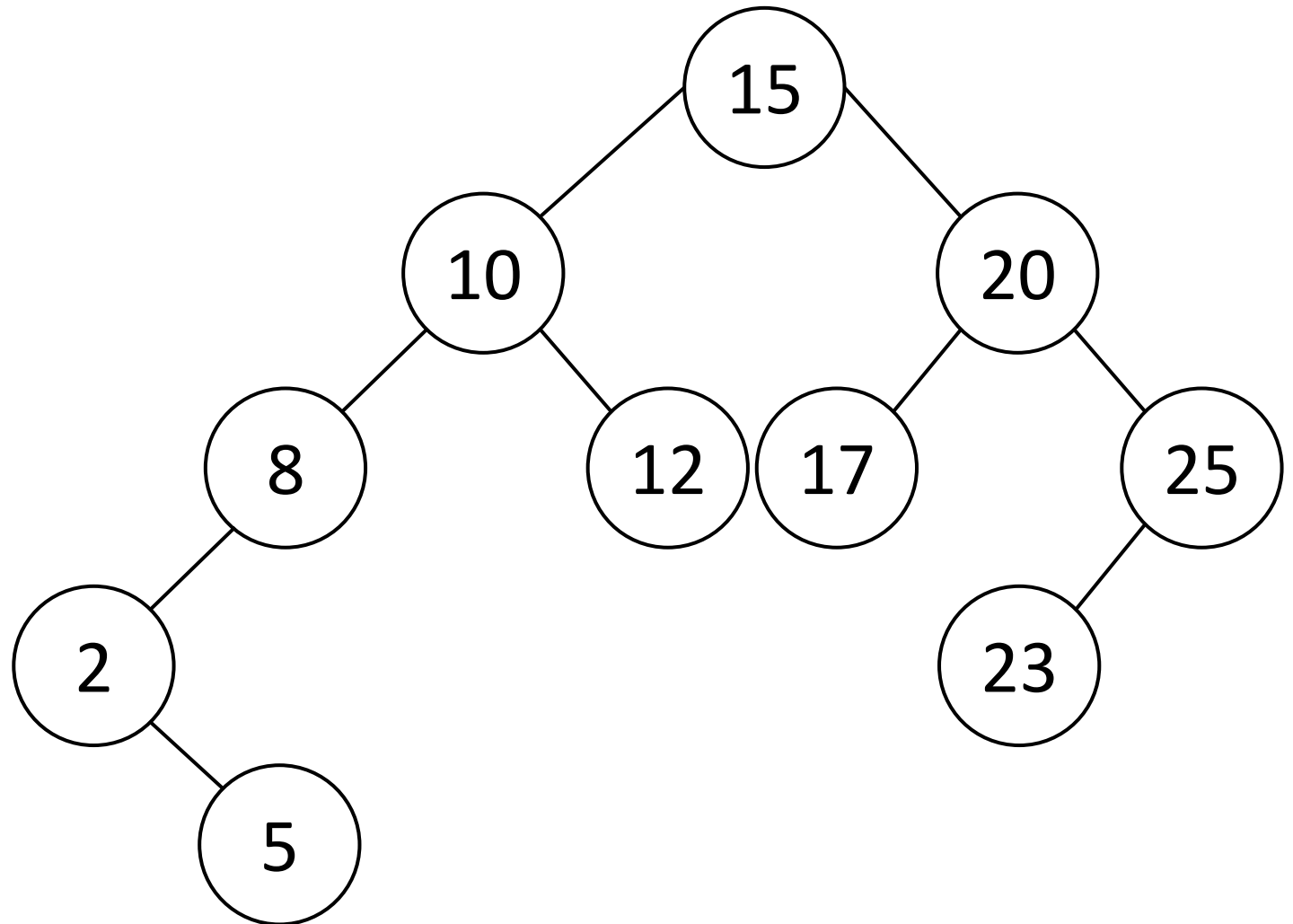
Insert (12)



Insert (15)
Insert (20)
Insert (17)
Insert (10)
Insert (8)
Insert (2)
Insert (25)
Insert (23)
Insert (5)
Insert (12)



Insert (15)
Insert (20)
Insert (17)
Insert (10)
Insert (8)
Insert (2)
Insert (25)
Insert (23)
Insert (5)
Insert (12)



```
struct Node{  
    int data  
    struct Node* left  
    struct Node* right  
    struct Node* parent  
}
```

```
Insert (int data){
```

```
Insert (int data){  
    Node* newNode = create_node(data)  
    Node* temp = tree_root
```

```
Insert (int data){  
    Node* newNode = create_node(data)  
    Node* temp = tree_root  
  
    if (temp == Null)  
        tree_root = newNode  
}
```

```

Insert (int data){
    Node* newNode = create_node(data)
    Node* temp = tree_root

    if (temp == Null)
        tree_root = newNode

    while (      ?      )

        if (newNode->data < temp->data) temp = temp->left
        else                           temp = temp->right

```

```
Insert (int data){  
    Node* newNode = create_node(data)  
    Node* temp = tree_root  
  
    if (temp == Null)  
        tree_root = newNode  
  
    while (temp!= Null)  
  
        if (newNode->data < temp->data) temp = temp->left  
        else temp = temp->right  
}
```

```
Insert (int data){  
    Node* newNode = create_node(data)  
    Node* temp = tree_root  
  
    if (temp == Null)  
        tree_root = newNode  
  
    while (temp!= Null)  
  
        if (newNode->data < temp->data) temp = temp->left  
        else temp = temp->right
```

Connect temp with newNode

```

Insert (int data){
    Node* newNode = create_node(data)
    Node* temp = tree_root

    if (temp == Null)
        tree_root = newNode

    while (temp!= Null)

        if (newNode->data < temp->data) temp = temp->left
        else temp = temp->right

```

*Connect temp with newNode
But temp has become Null !*

```
Insert (int data){  
    Node* newNode = create_node(data)  
    Node* temp = tree_root  
  
    if (temp == Null)  
        tree_root = newNode  
  
    while (temp!= Null)  
        target = temp // holds the most recent value of temp  
        if (newNode->data < temp->data) temp = temp->left  
        else temp = temp->right  
}
```

```
Insert (int data){
    Node* newNode = create_node(data)
    Node* temp = tree_root

    if (temp == Null)
        tree_root = newNode

    while (temp!= Null)
        target = temp // holds the most recent value of temp
        if (newNode->data < temp->data) temp = temp->left
        else temp = temp->right
}
```

Connect target with newNode

```

Insert (int data){
    Node* newNode = create_node(data)
    Node* temp = tree_root

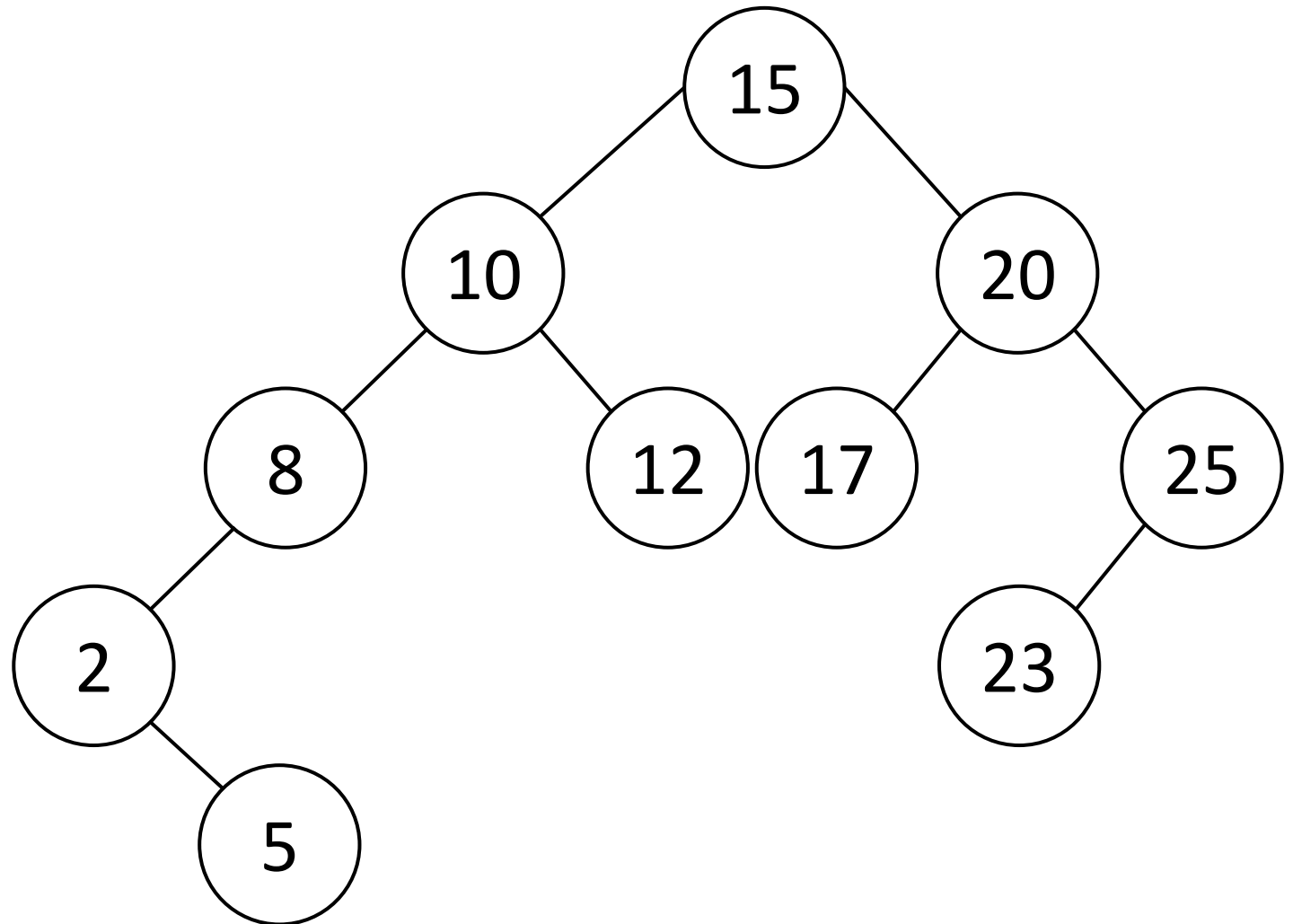
    if (temp == Null)
        tree_root = newNode

    while (temp!= Null)
        target = temp // holds the most recent value of temp
        if (newNode->data < temp->data) temp = temp->left
        else temp = temp->right

    newNode->parent = target
    if (newNode->data < target->data) target->left = newNode
    else target->right= newNode
}

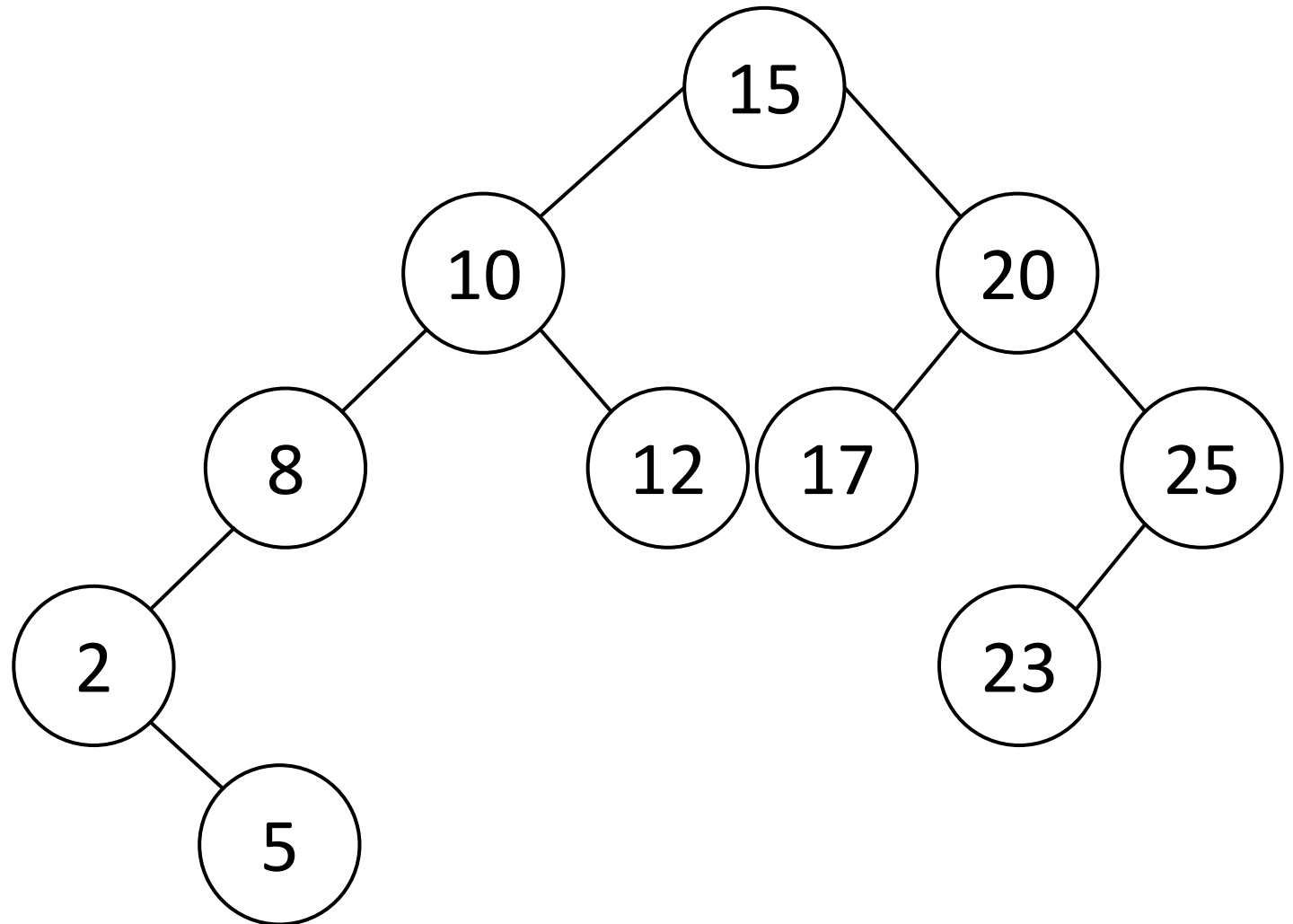
```

Search value in BST:



Search value in BST:

Is 23 present?



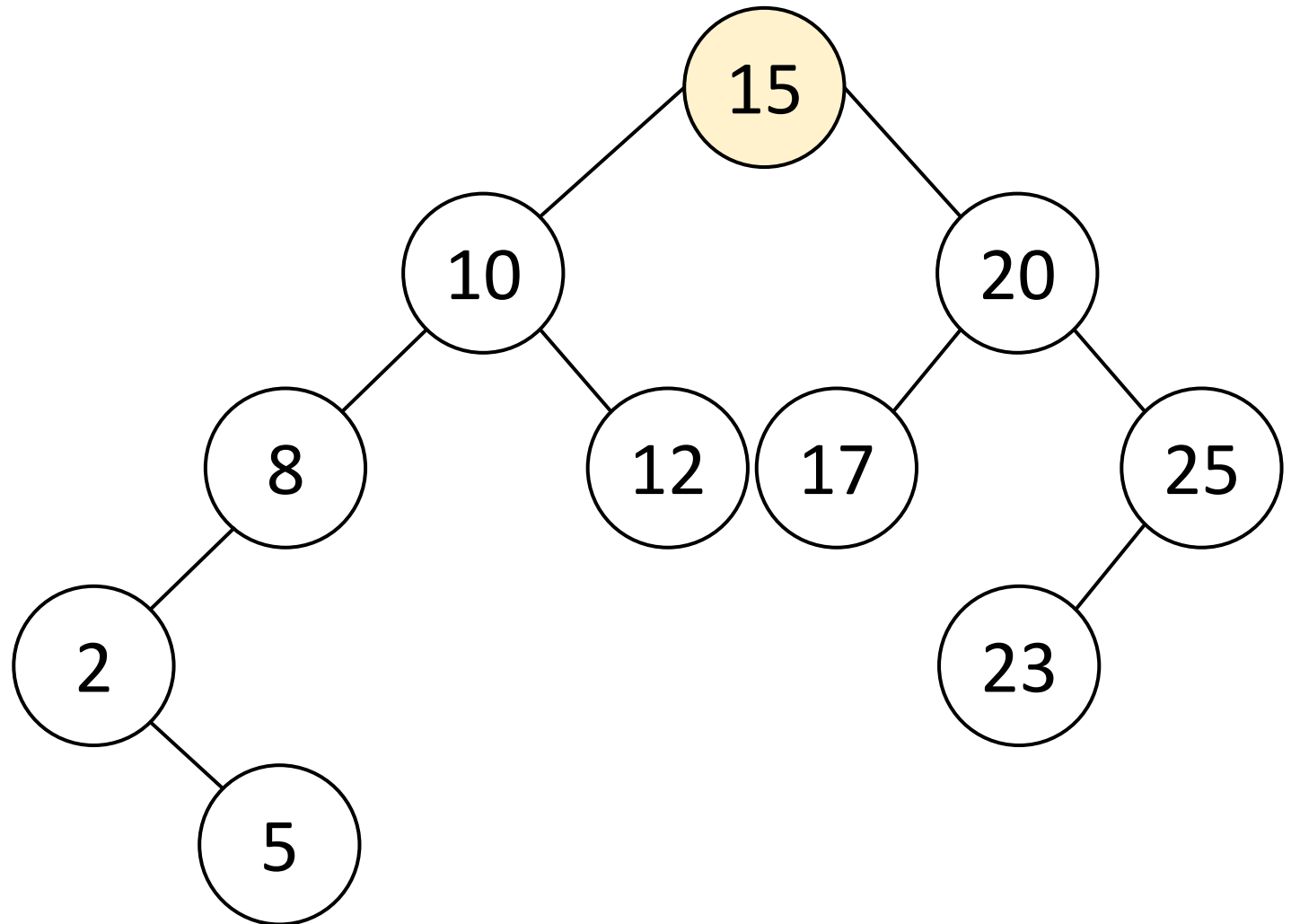
Search value in BST:

Is 23 present?

Idea:

Start at root

$23 > 15$: go right!



Search value in BST:

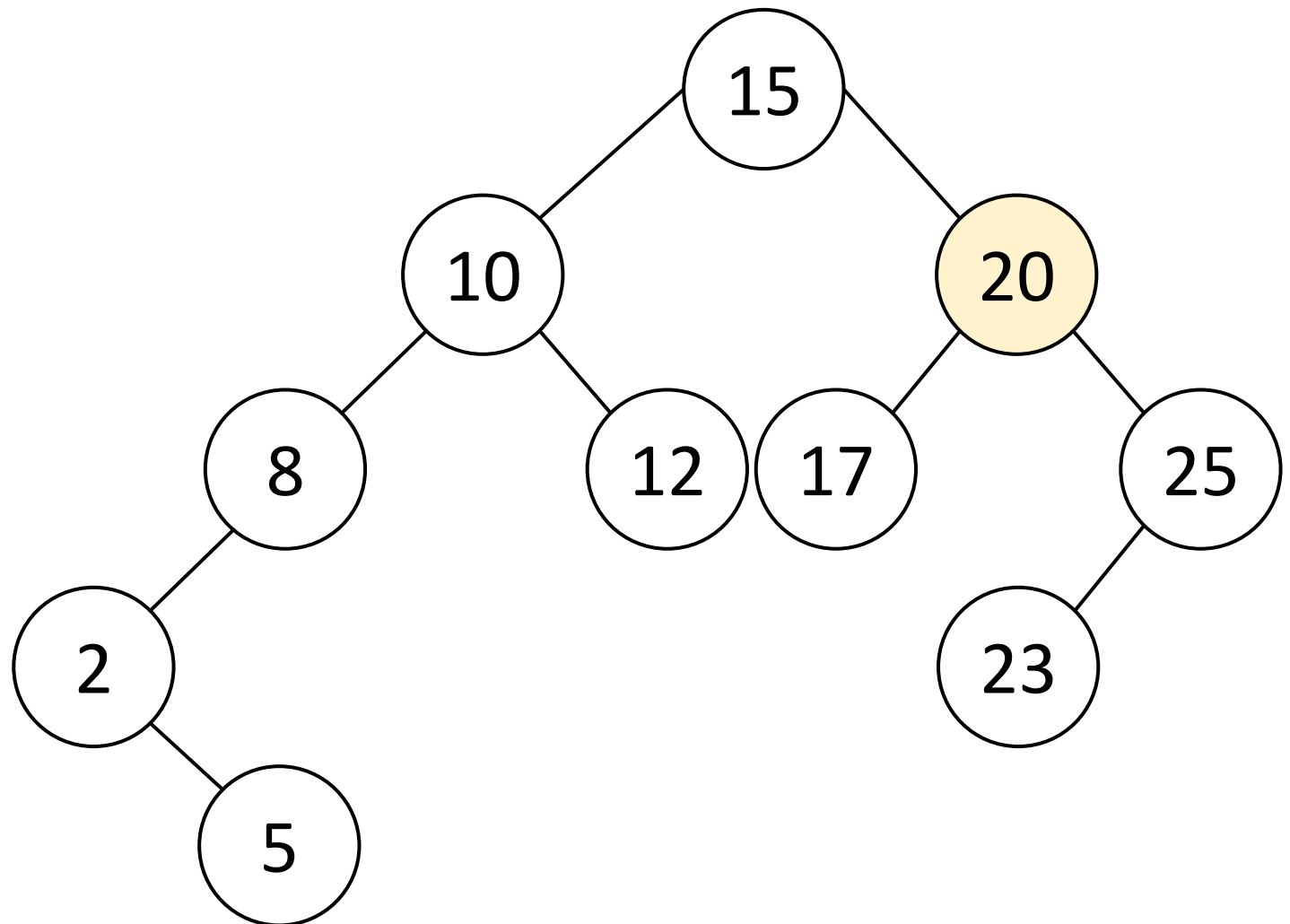
Is 23 present?

Idea:

Start at root

$23 > 15$: go right

$23 > 20$: go right



Search value in BST:

Is 23 present?

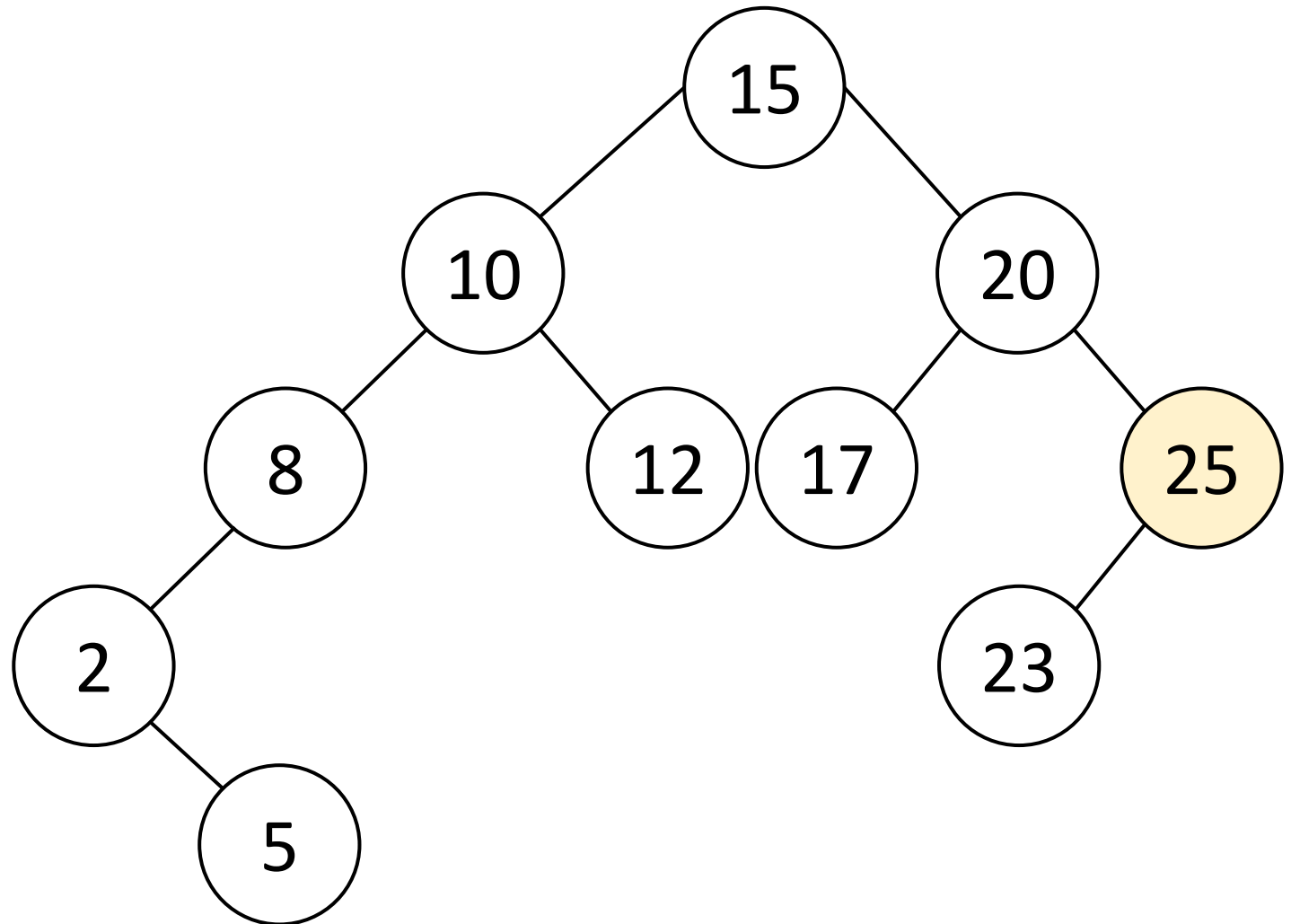
Idea:

Start at root

$23 > 15$: go right

$23 > 20$: go right

$23 < 25$: go left



Search value in BST:

Is 23 present?

Idea:

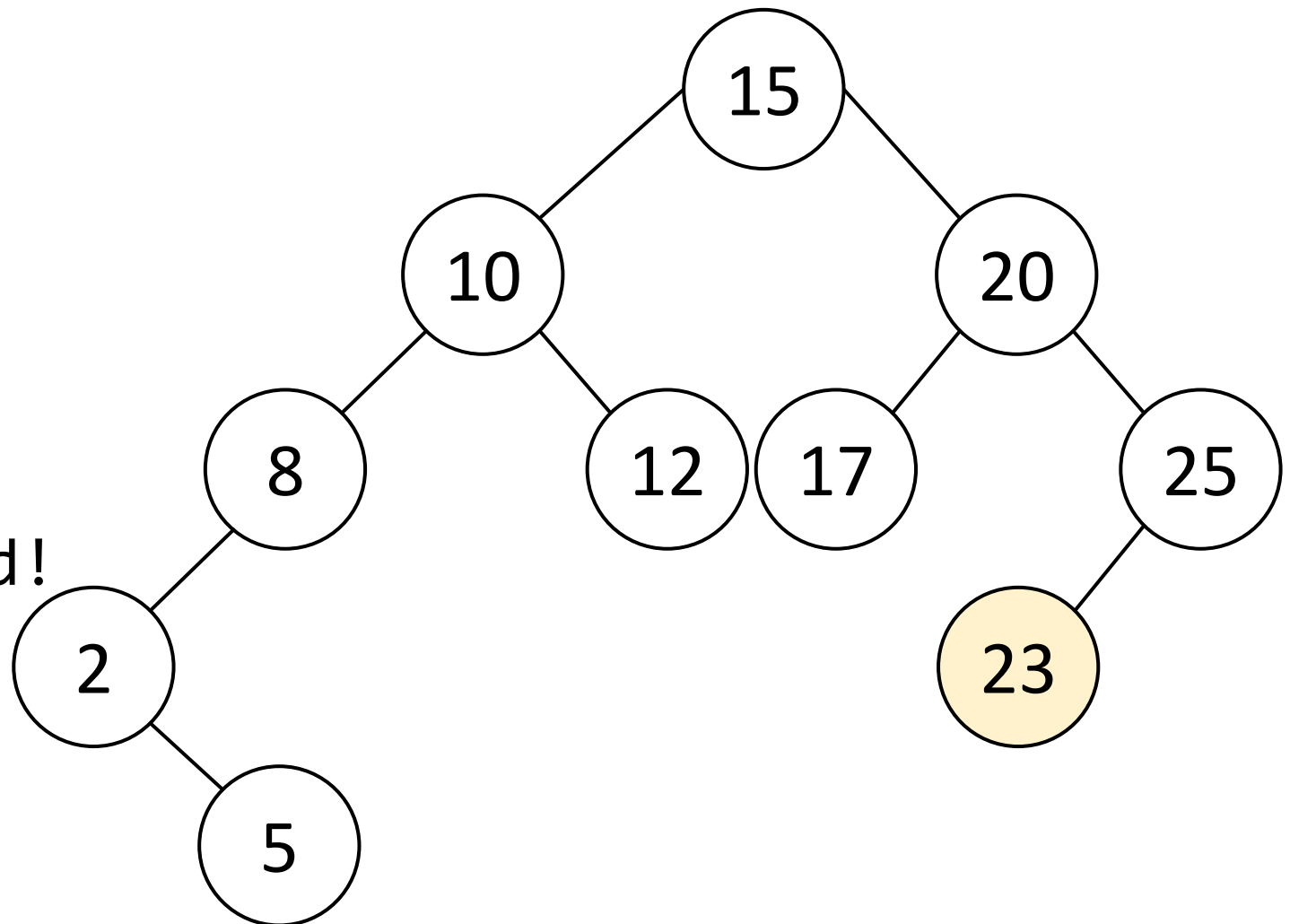
Start at root

$23 > 15$: go right

$23 > 20$: go right

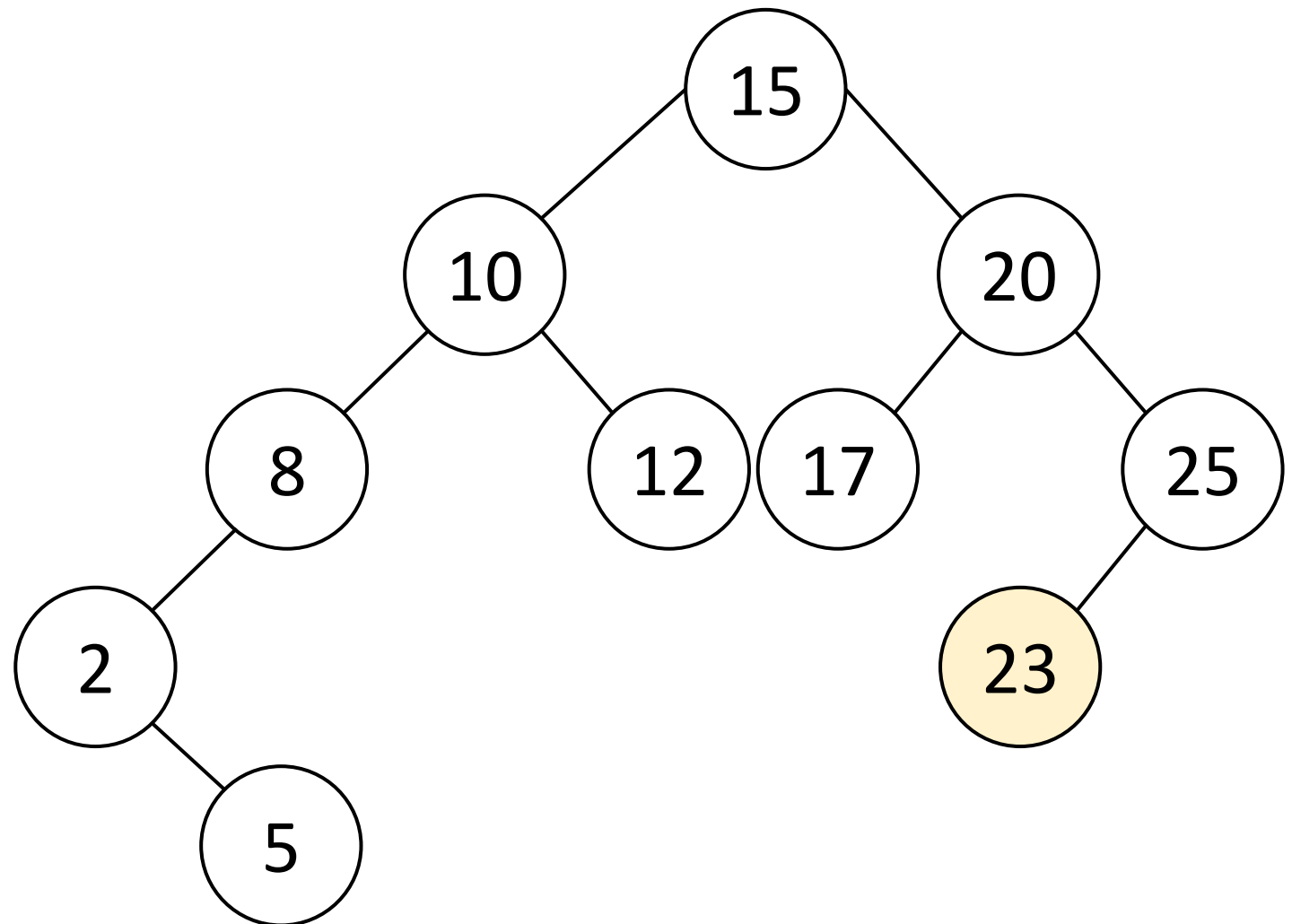
$23 < 25$: go left

$23 == 23$: value found!



Search value in BST:

Is 24 present?

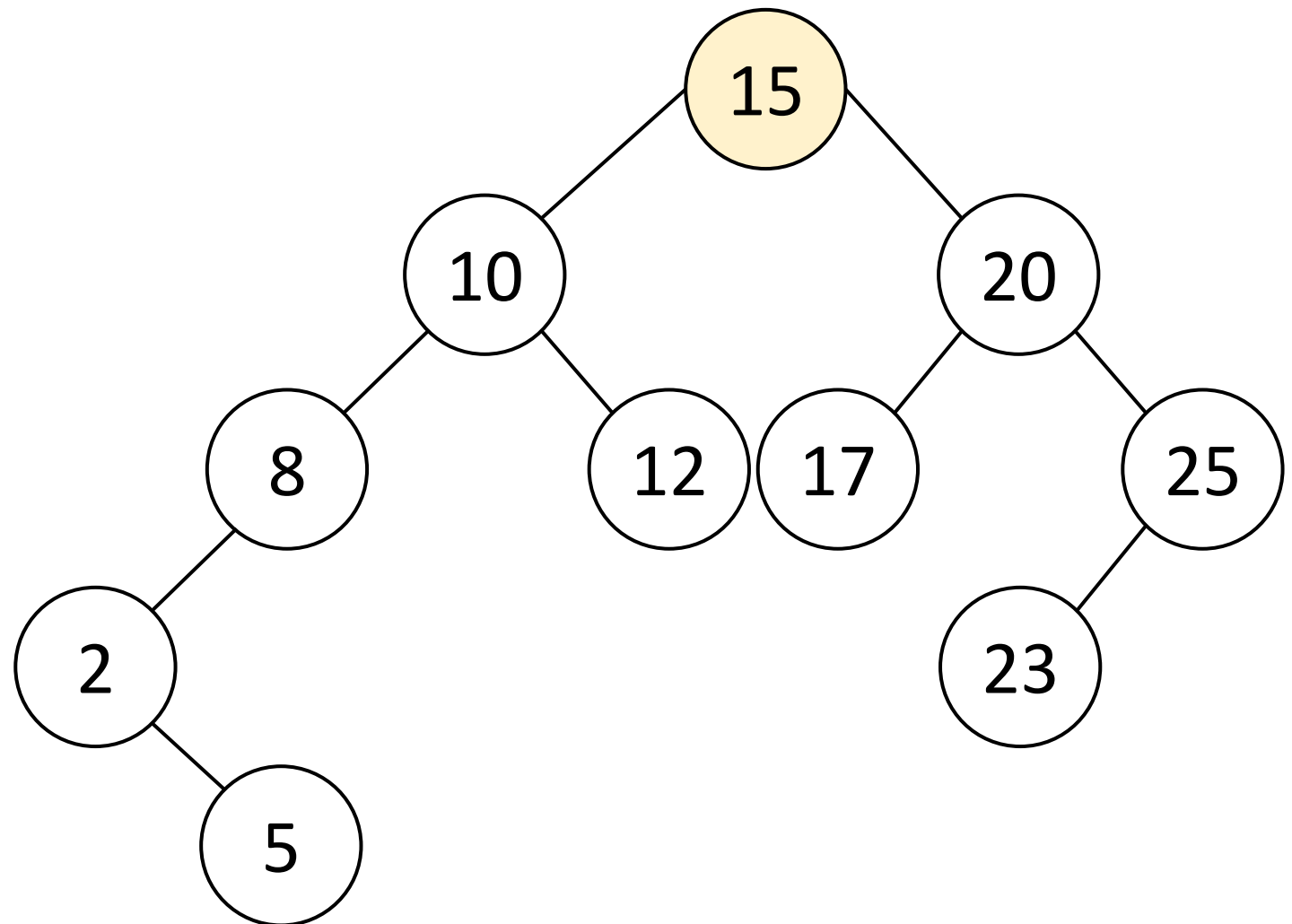


Search value in BST:

Is 24 present?

Idea:

$24 > 15$: go right



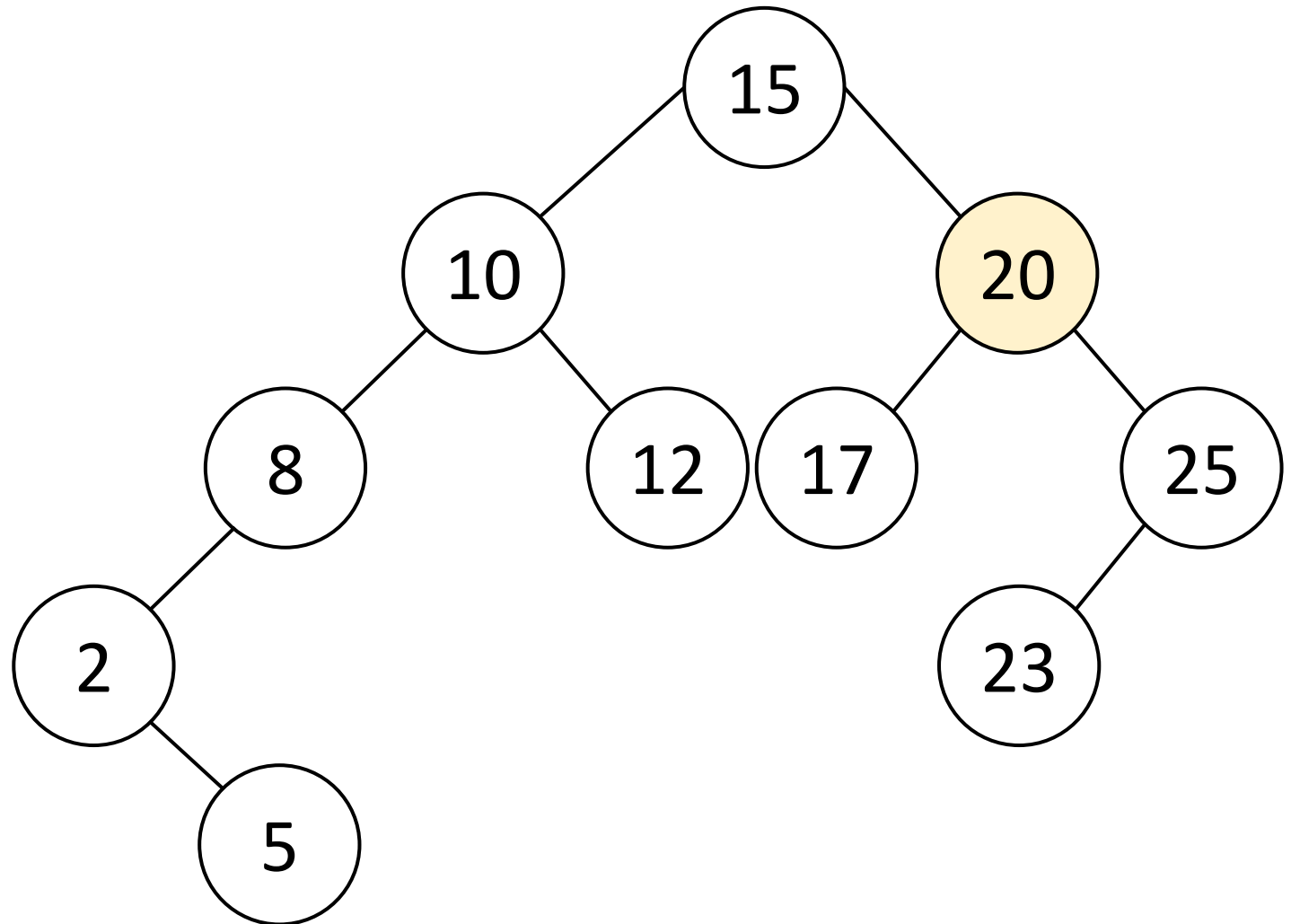
Search value in BST:

Is 24 present?

Idea:

$24 > 15$: go right

$24 > 20$: go right



Search value in BST:

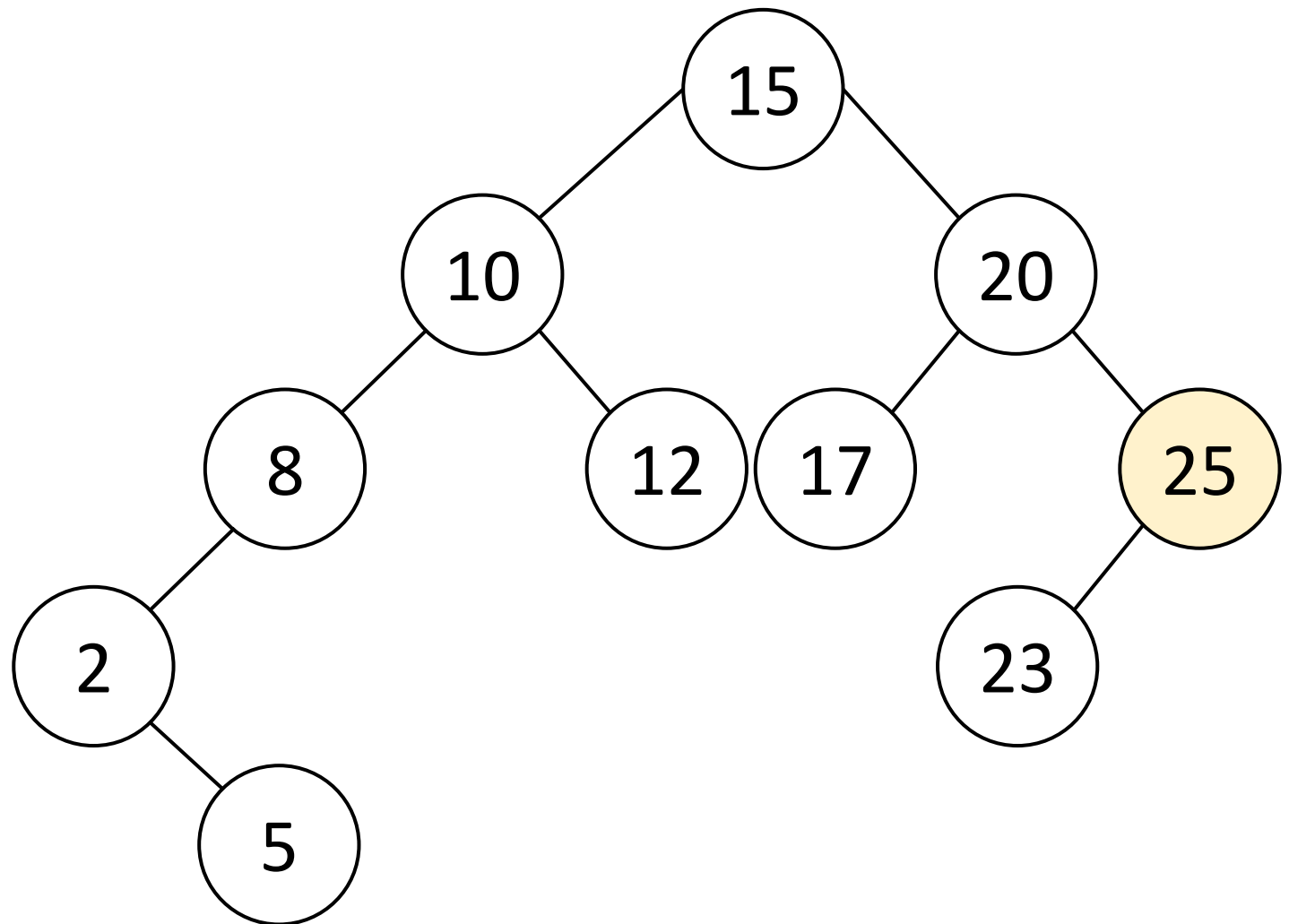
Is 24 present?

Idea:

$24 > 15$: go right

$24 > 20$: go right

$24 < 25$: go left



Search value in BST:

Is 24 present?

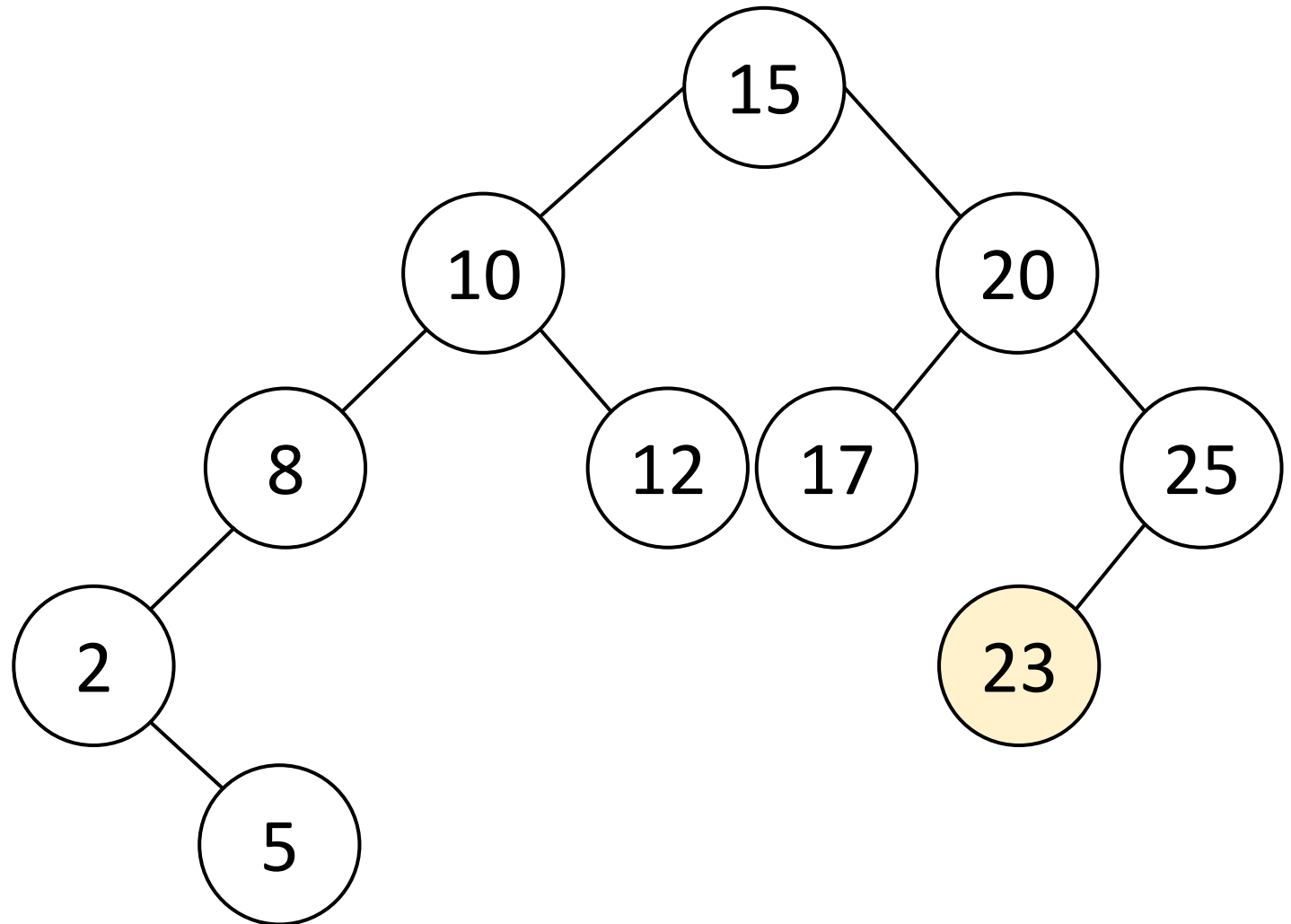
Idea:

$24 > 15$: go right

$24 > 20$: go right

$24 < 25$: go left

$24 > 23$: go right



Search value in BST:

Is 24 present?

Idea:

24 > 15: go right

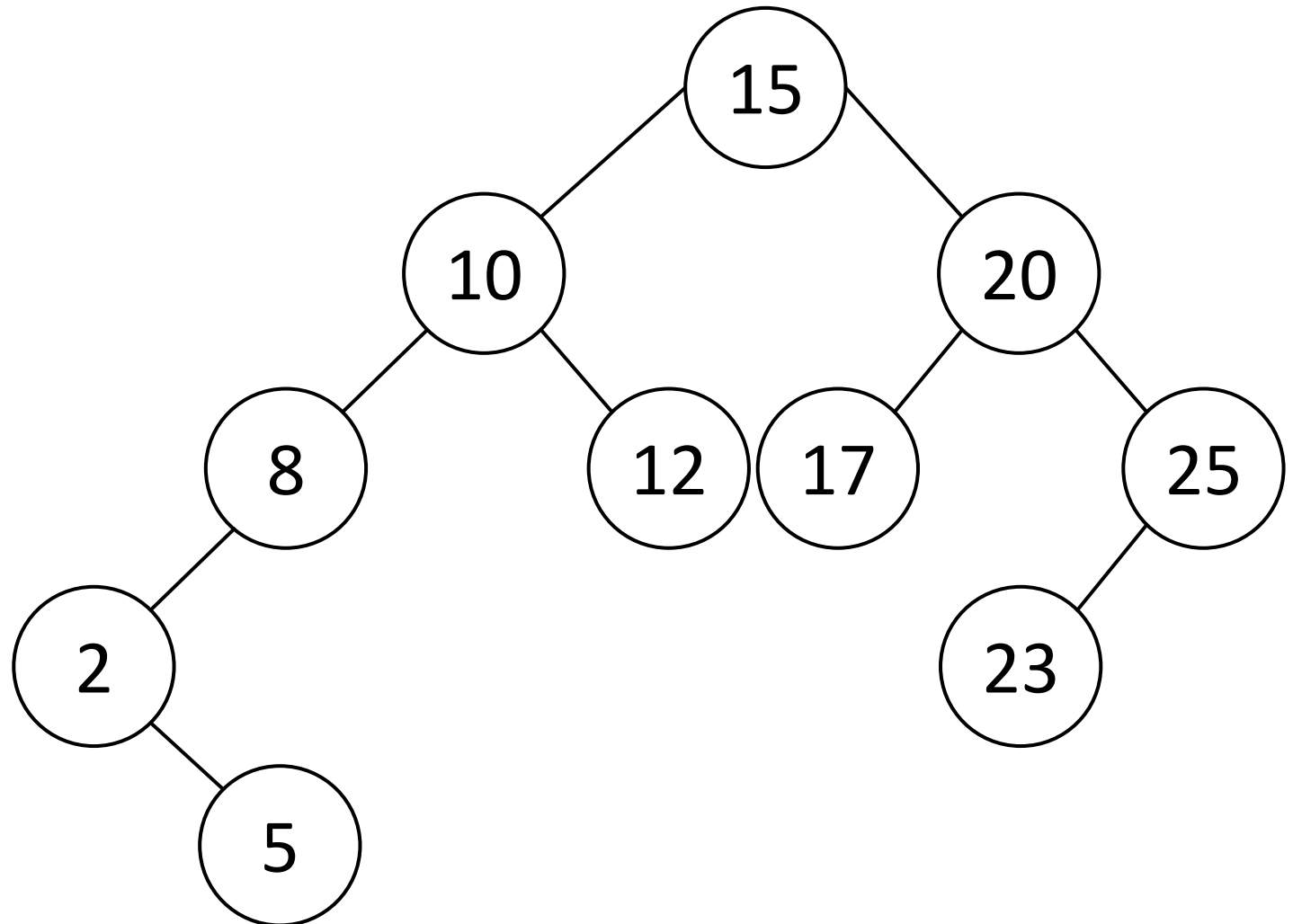
24 > 20: go right

24 < 25: go left

24 > 23: go right

Comparing with Null!

Means value absent!



```
Node* Tree_search(Node *x, int key){
```

?

```
}
```

```
Node* Tree_search(Node *x, int key){  
  
    if x->data < key  
        x = x->right  
  
}
```

```
Node* Tree_search(Node *x, int key){  
  
    if x->data < key  
        x = x->right  
    else  
        x = x->left  
  
}
```

```
Node* Tree_search(Node *x, int key){  
  
    while(  
        if x->data < key  
            x = x->right  
        else  
            x = x->left  
  
    }  
}
```

```
Node* Tree_search(Node *x, int key){  
    while(x!= Null  
        if x->data < key  
            x = x->right  
        else  
            x = x->left  
    }  
}
```

```
Node* Tree_search(Node *x, int key){  
    while(x!= Null and x->data!=key)  
        if x->data < key  
            x = x->right  
        else  
            x = x->left  
}
```

```
Node* Tree_search(Node *x, int key){  
    while(x!= Null and x->data!=key)  
        if x->data < key  
            x = x->right  
        else  
            x = x->left  
    return x  
}
```

```
Node* Tree_search(Node *x, int key){  
    while(x!= Null and x->data!=key)  
        if x->data < key  
            x = x->right  
        else  
            x = x->left  
    return x  
}
```

*Time
complexity?*

```
Node* Tree_search(Node *x, int key){  
    while(x!= Null and x->data!=key)  
        if x->data < key  
            x = x->right  
        else  
            x = x->left  
    return x  
}
```

*Time
complexity?
 $O(\text{height})$*

```

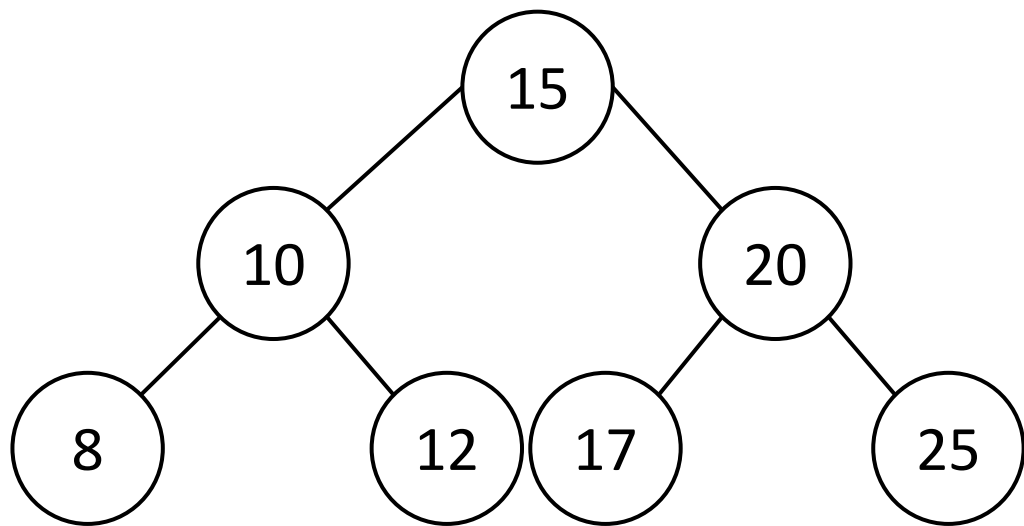
Node* Tree_search(Node *x, int key){

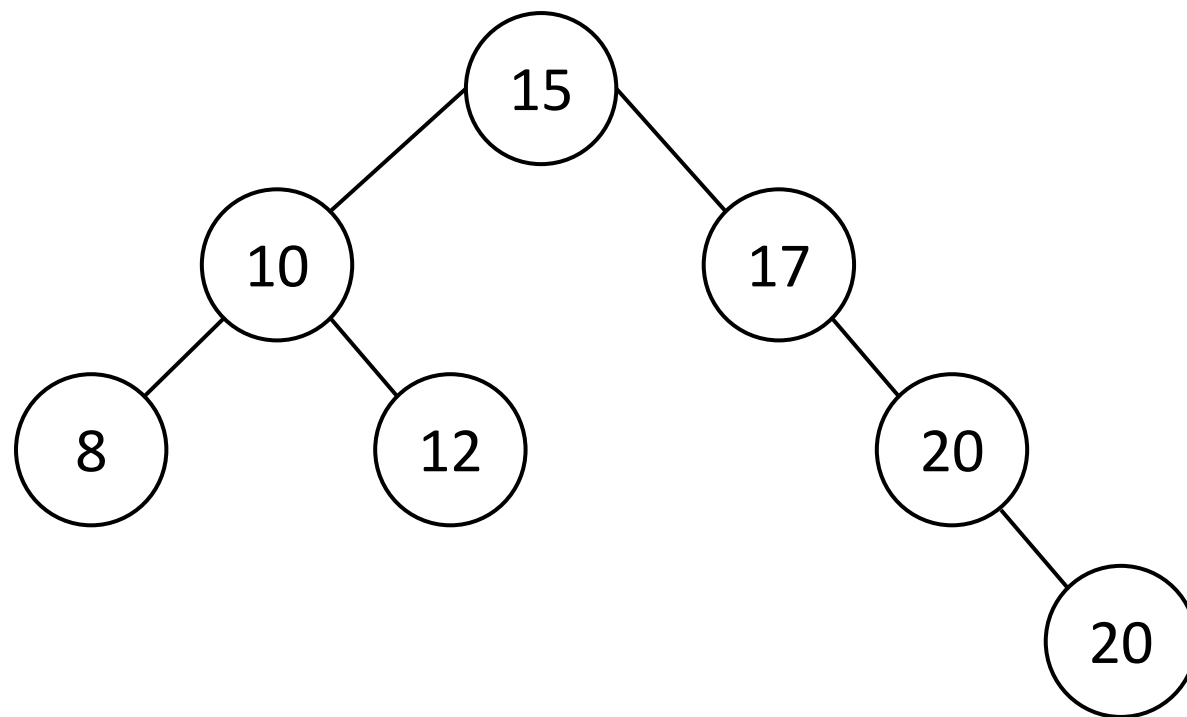
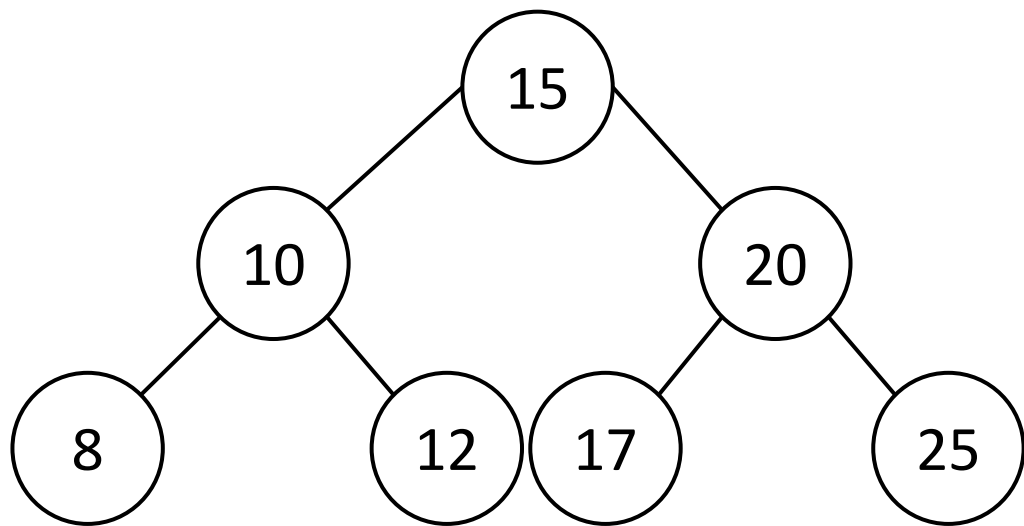
    while(x!= Null and x->data!=key)
        if x->data < key
            x = x->right
        else
            x = x->left
    return x
}

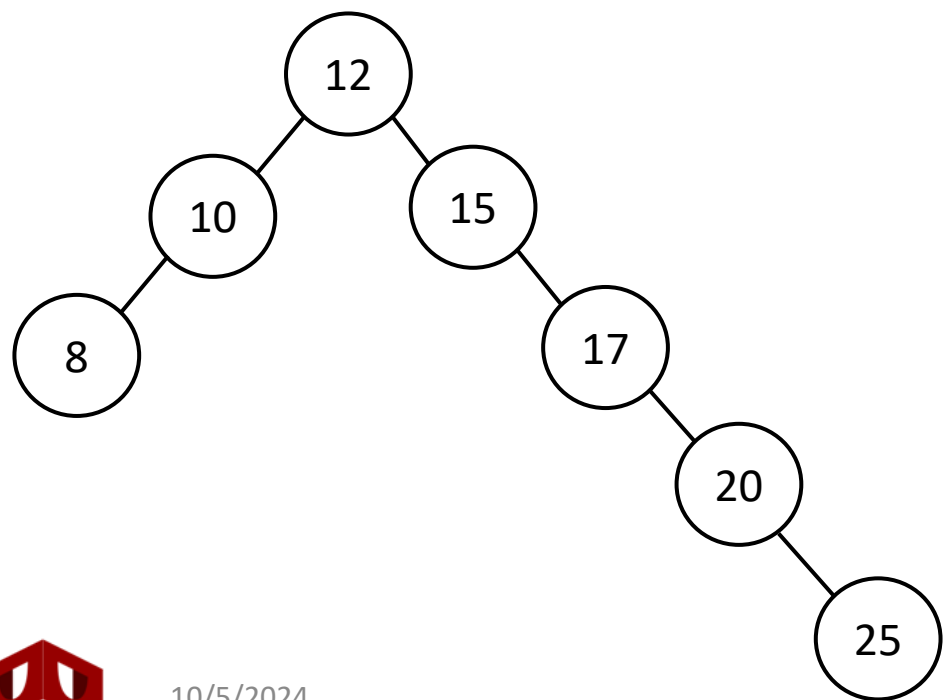
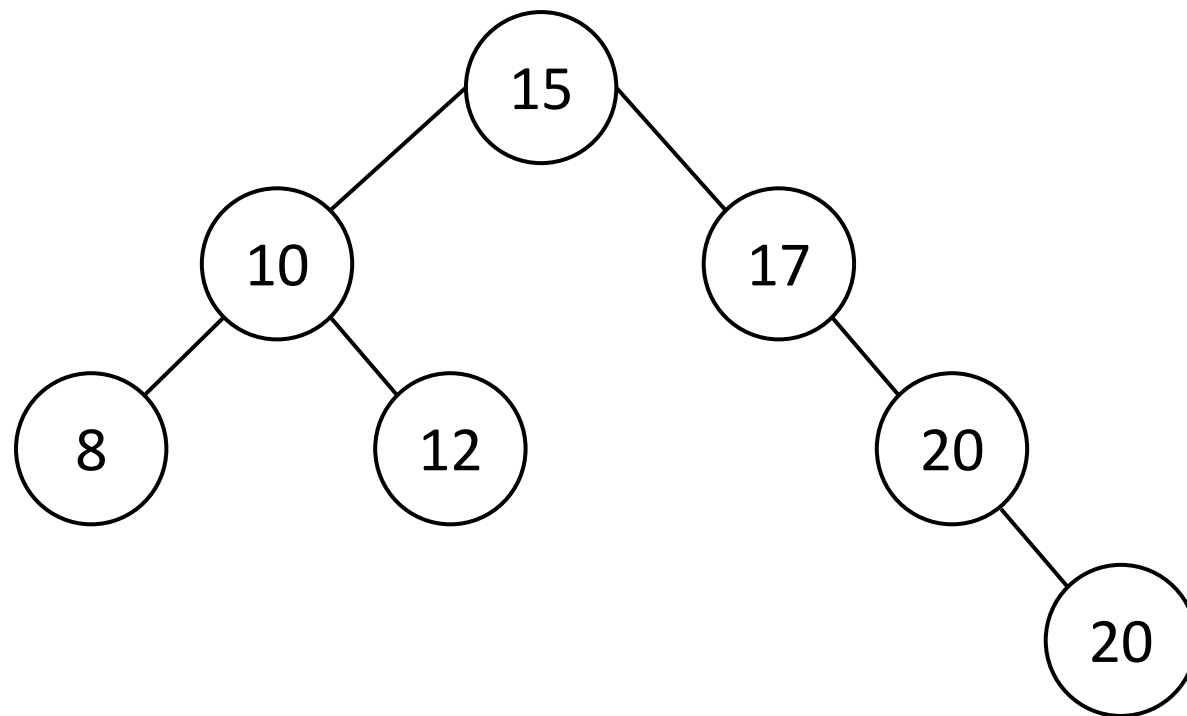
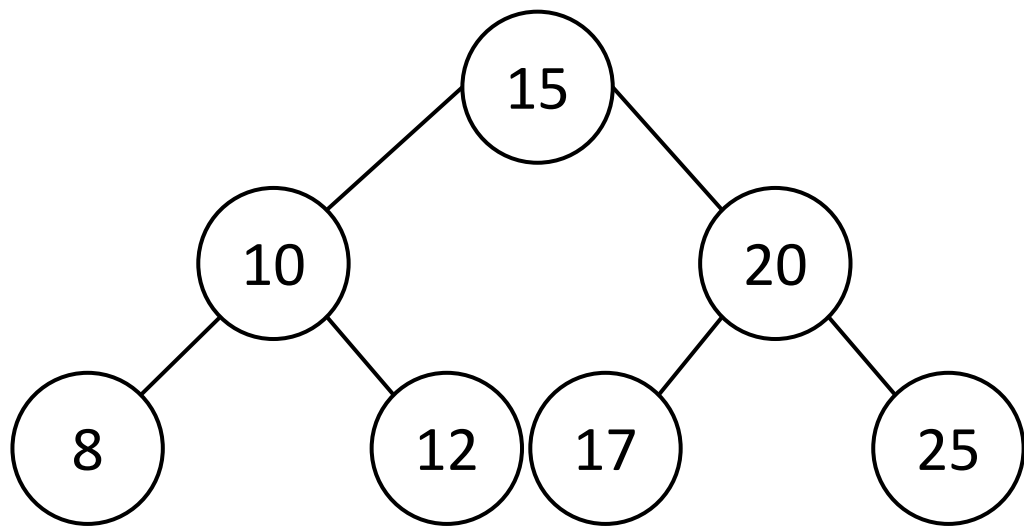
```

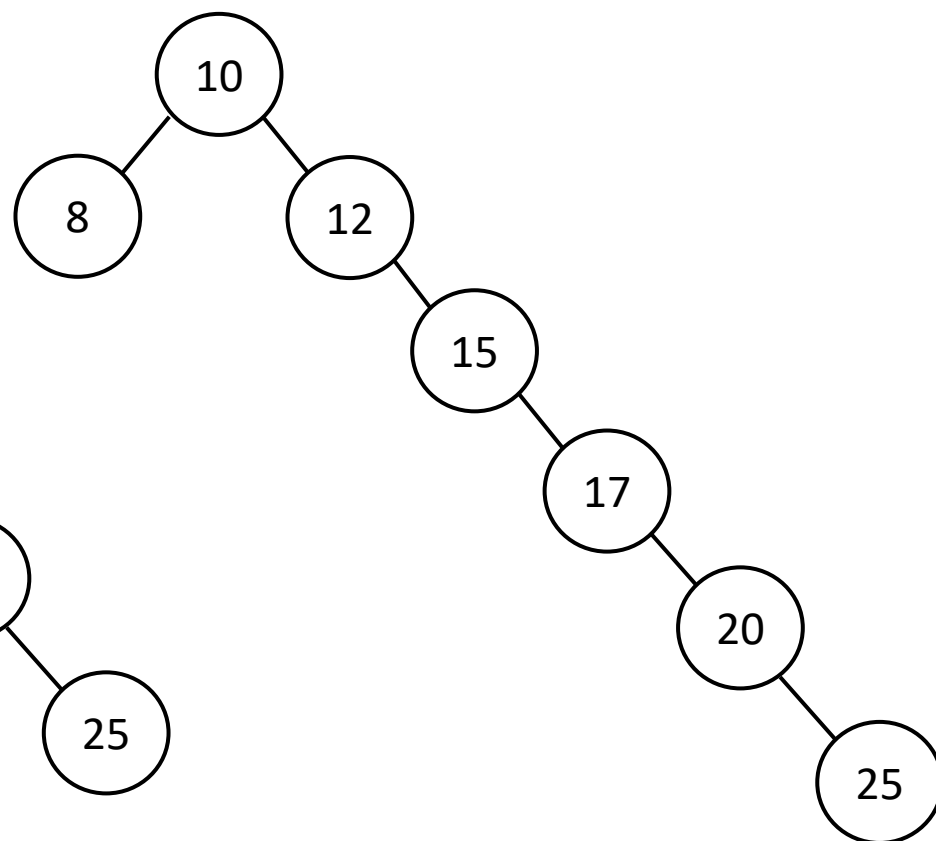
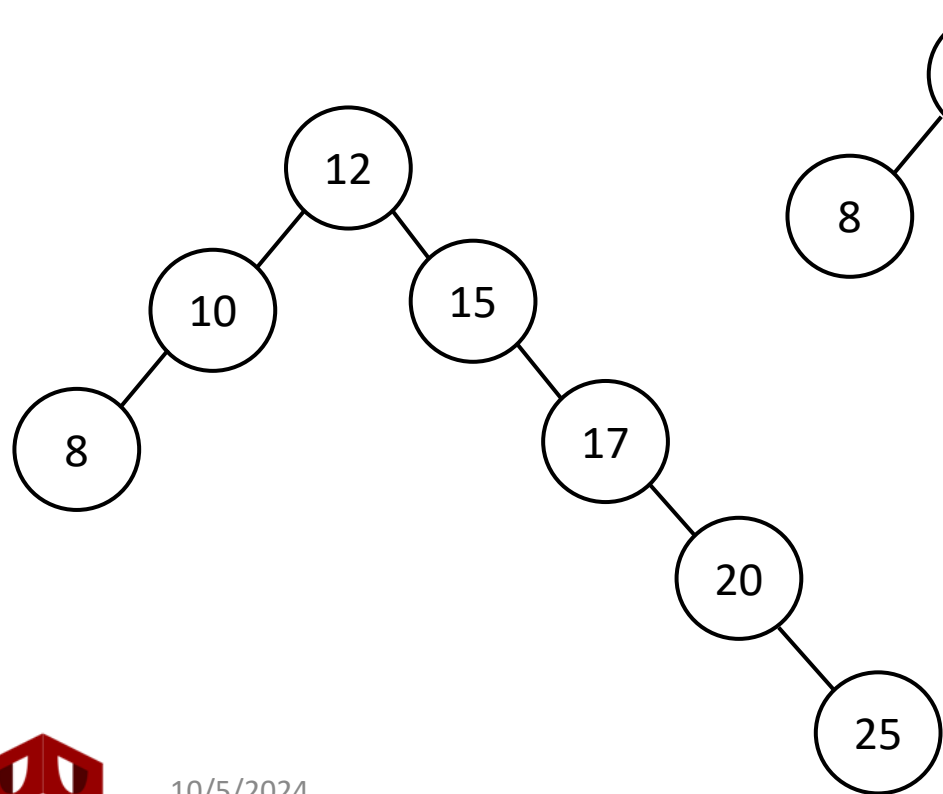
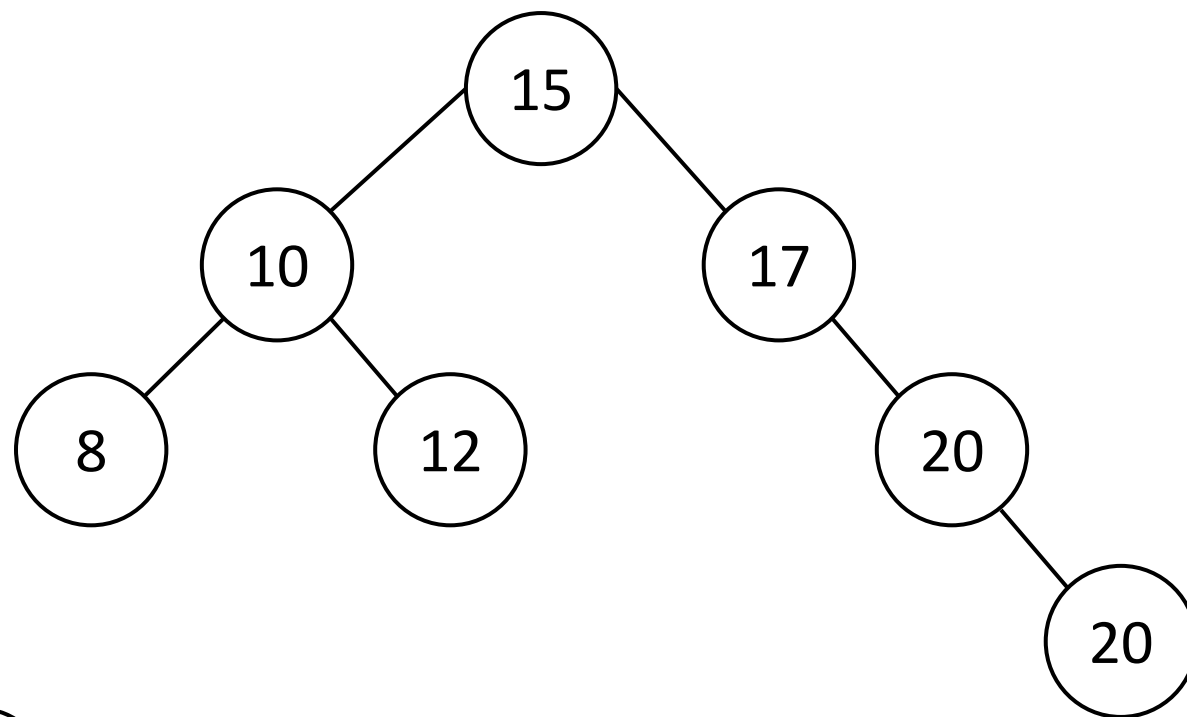
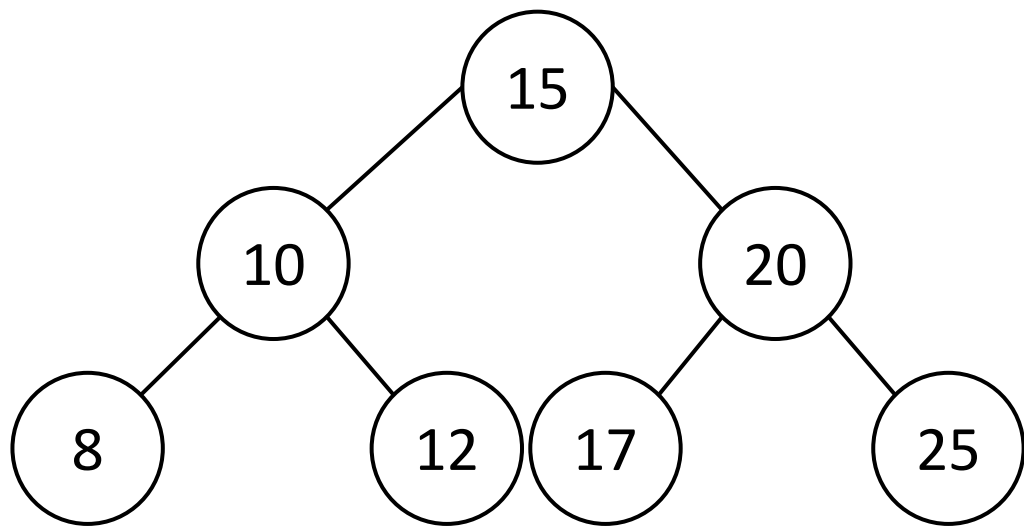
*Time
complexity?
 $O(\text{height})$*

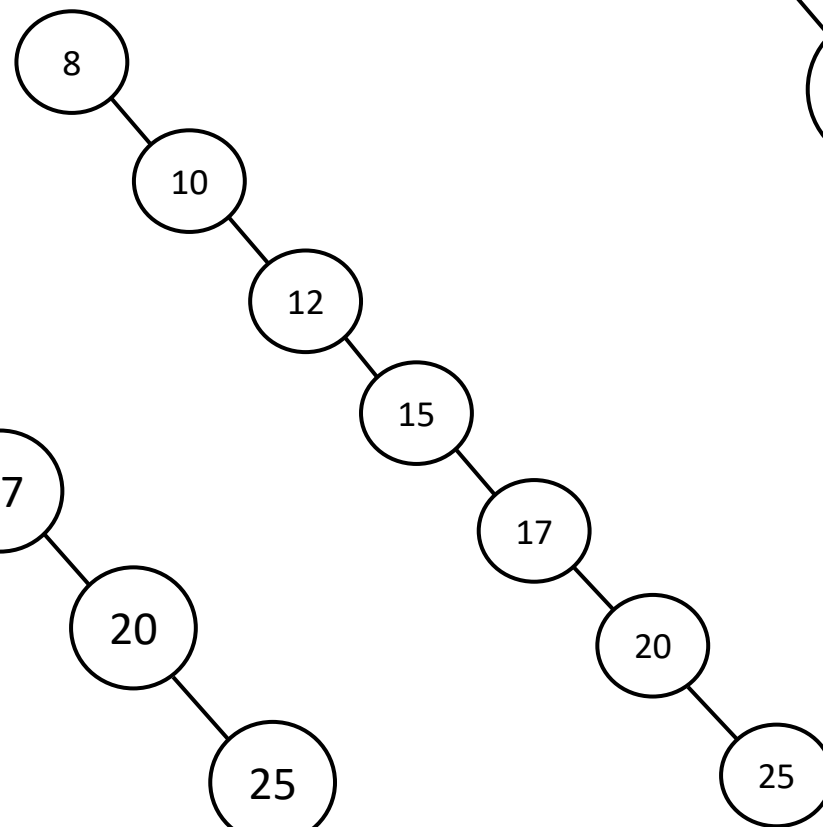
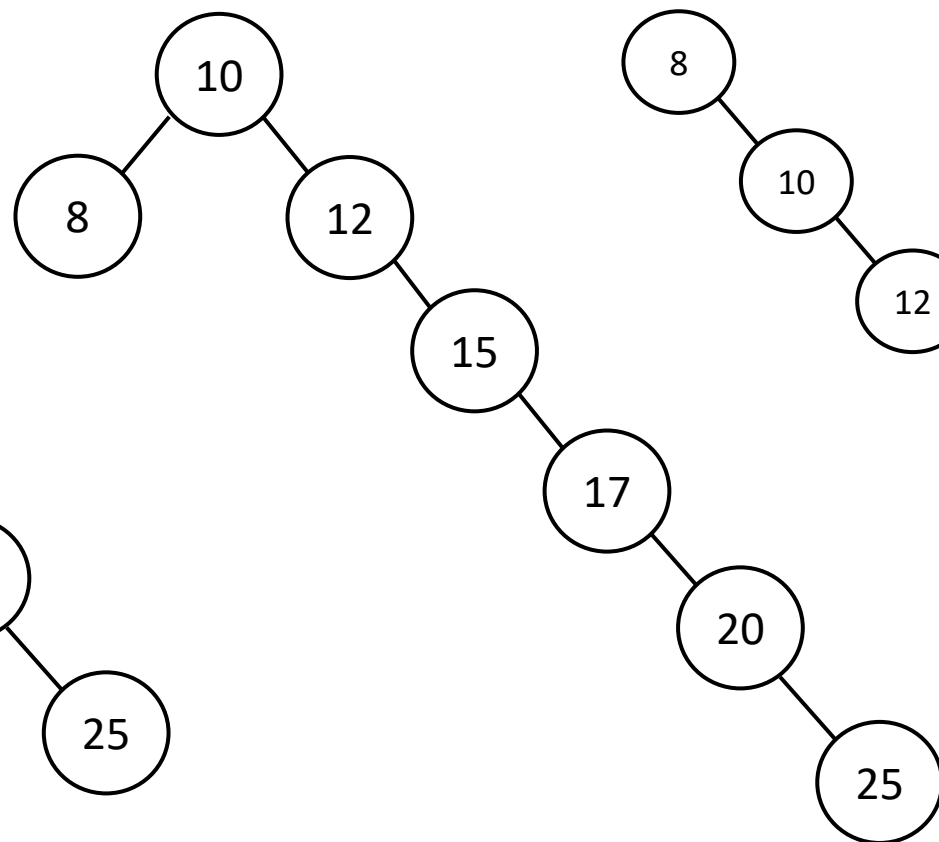
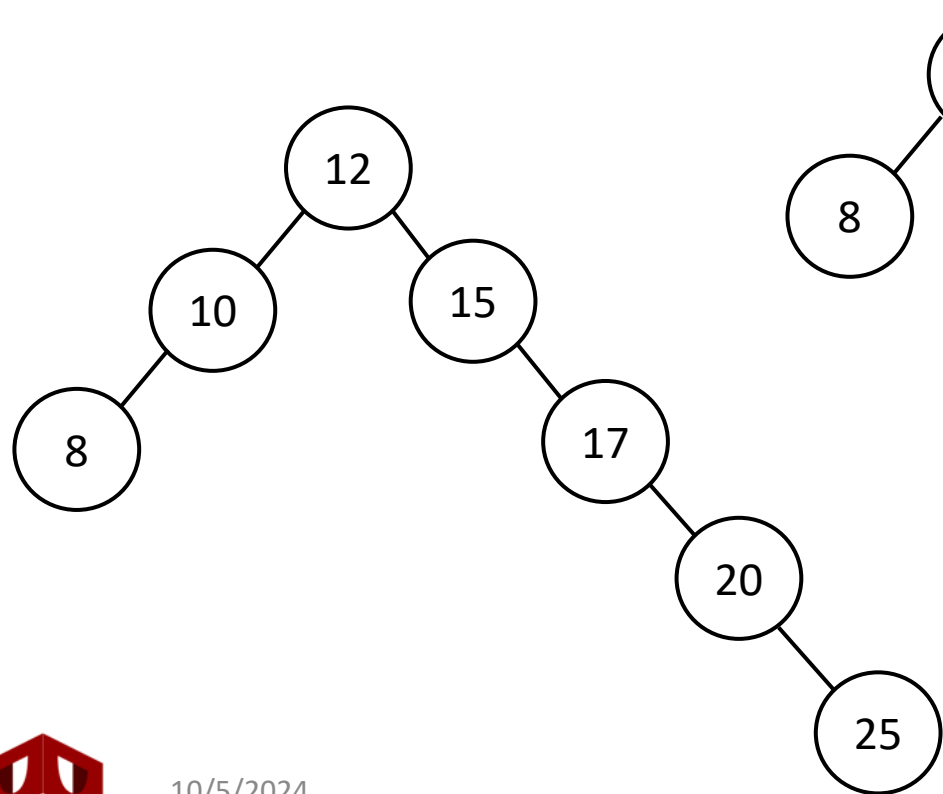
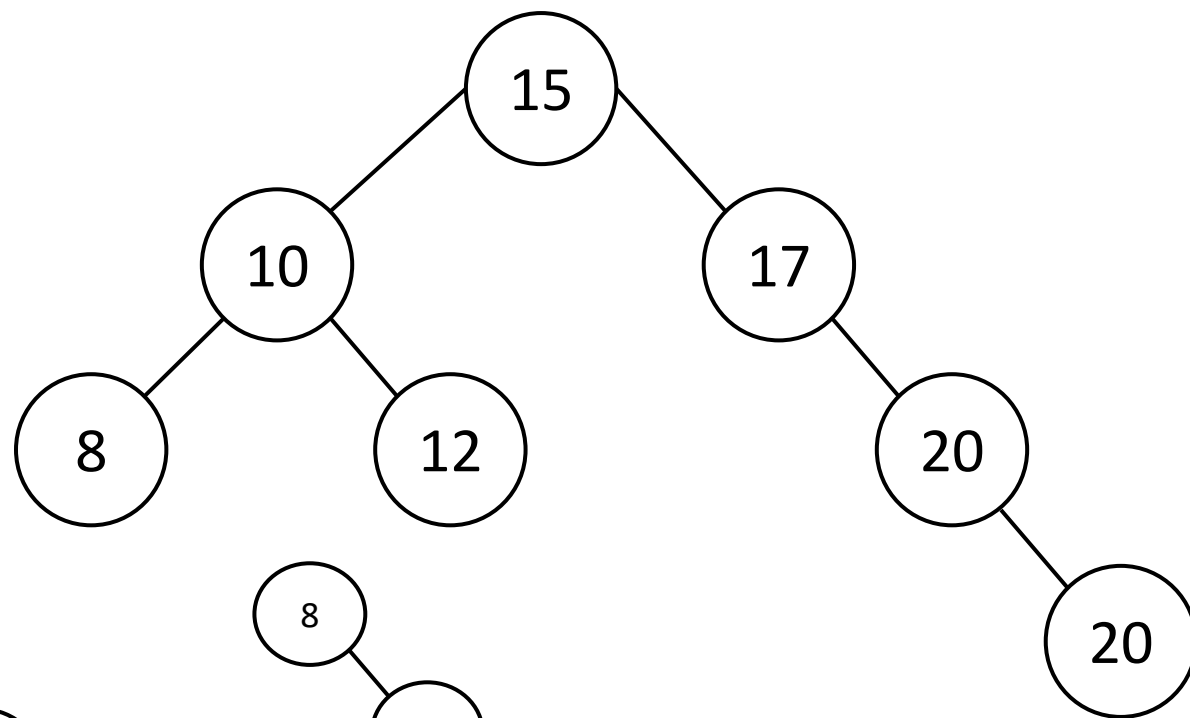
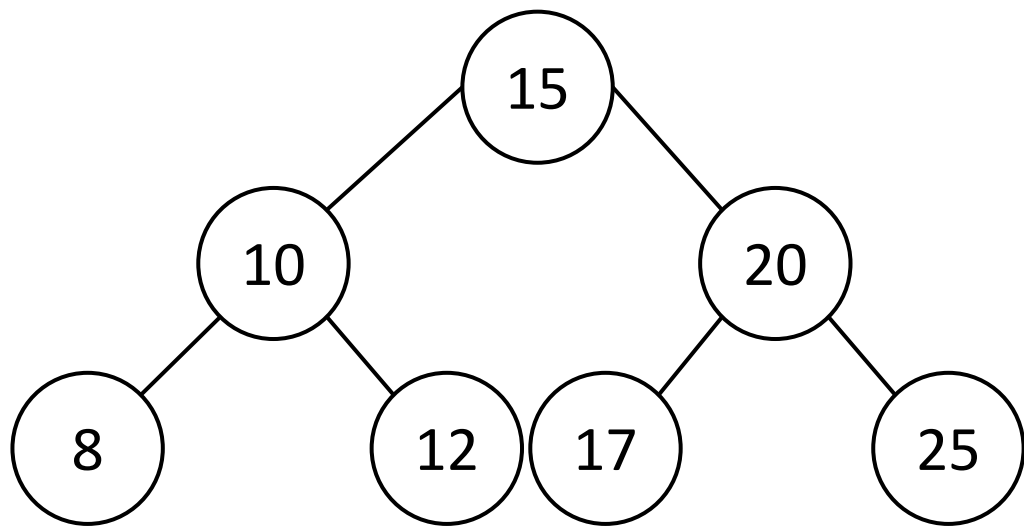
*(Balanced BSTs
can guarantee
 $\text{height} = \log N$)*



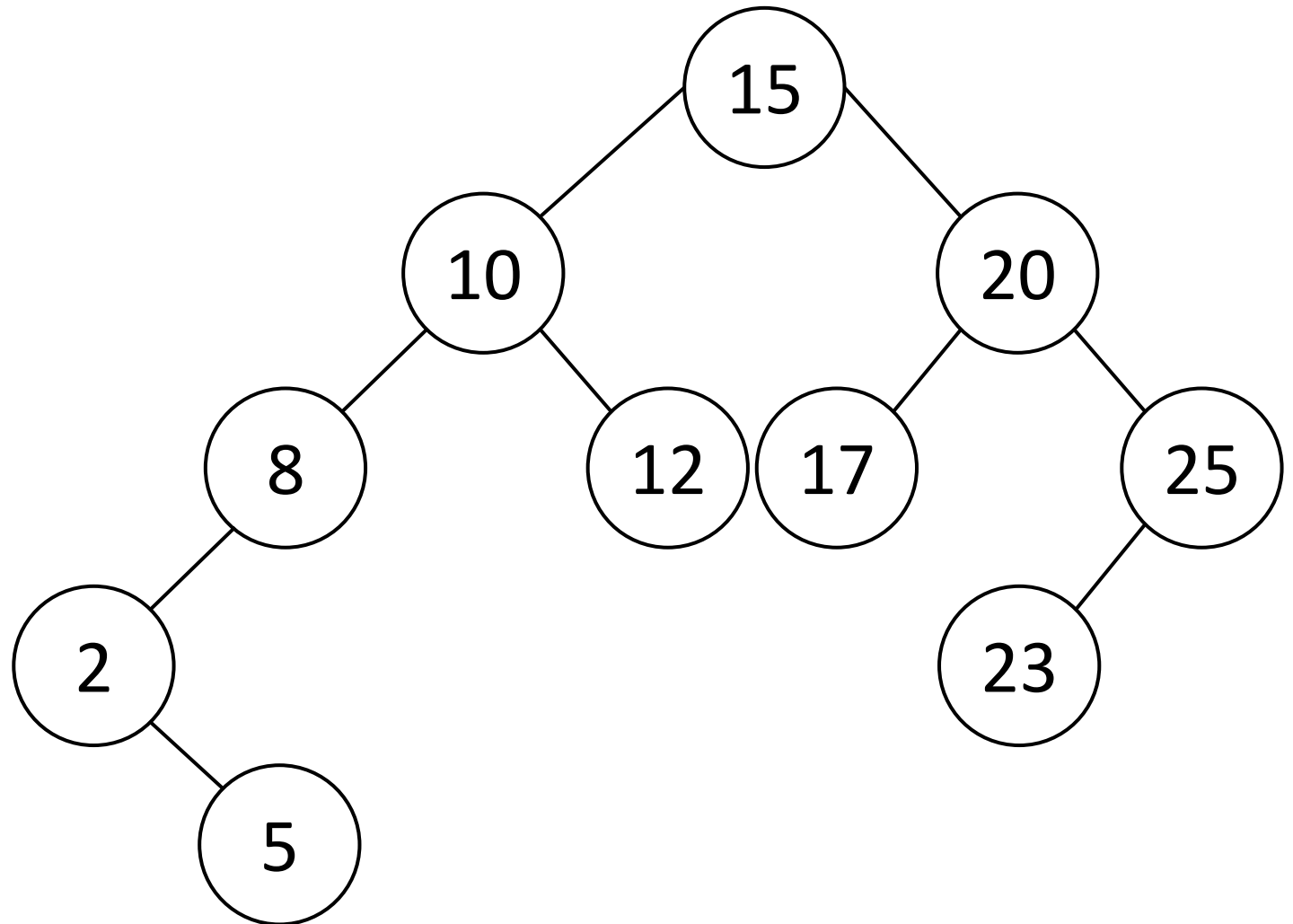








Maximum value in BST:

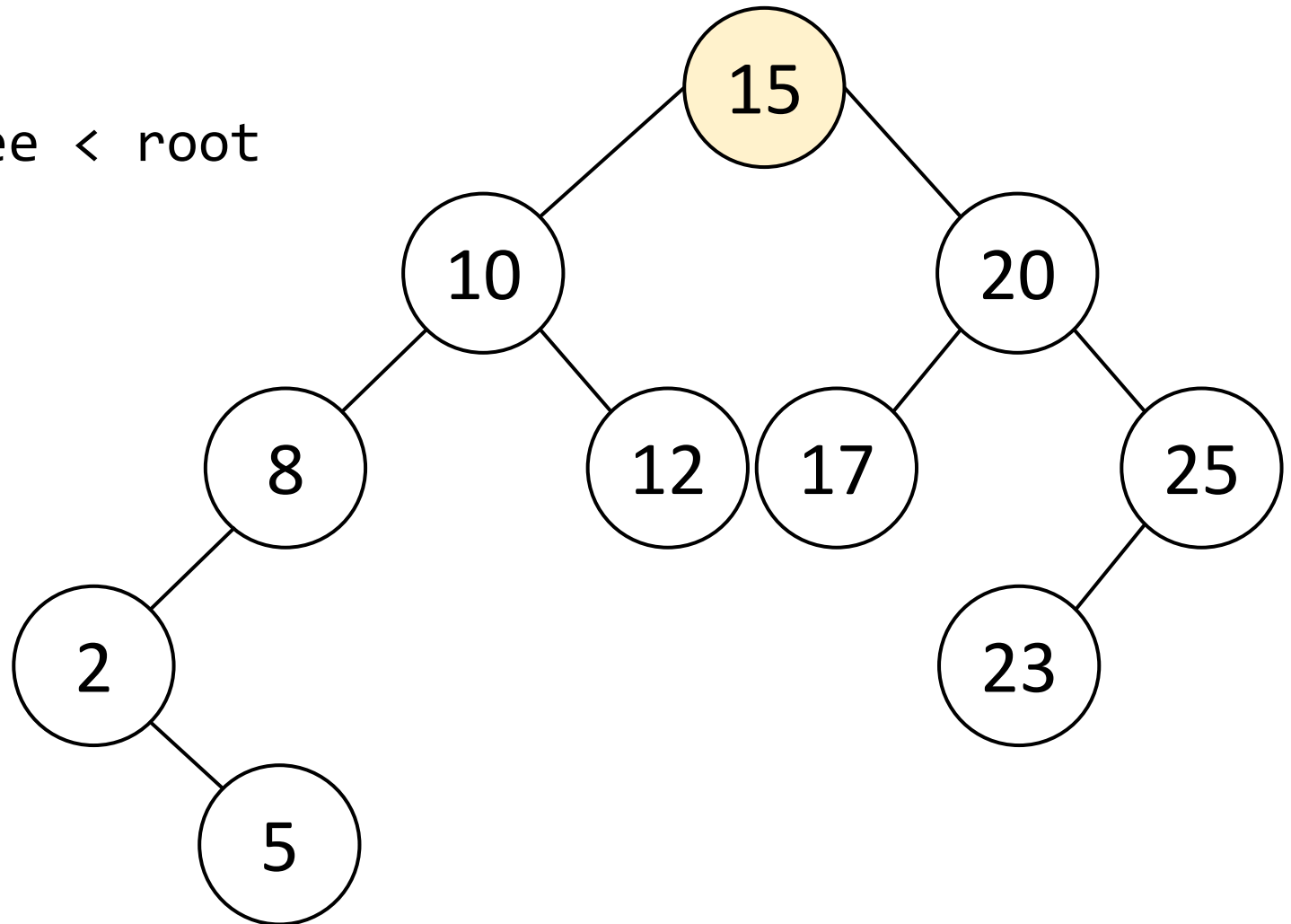


Maximum value in BST:

Idea:

Start from root (15)

All values in leftSubtree $<$ root



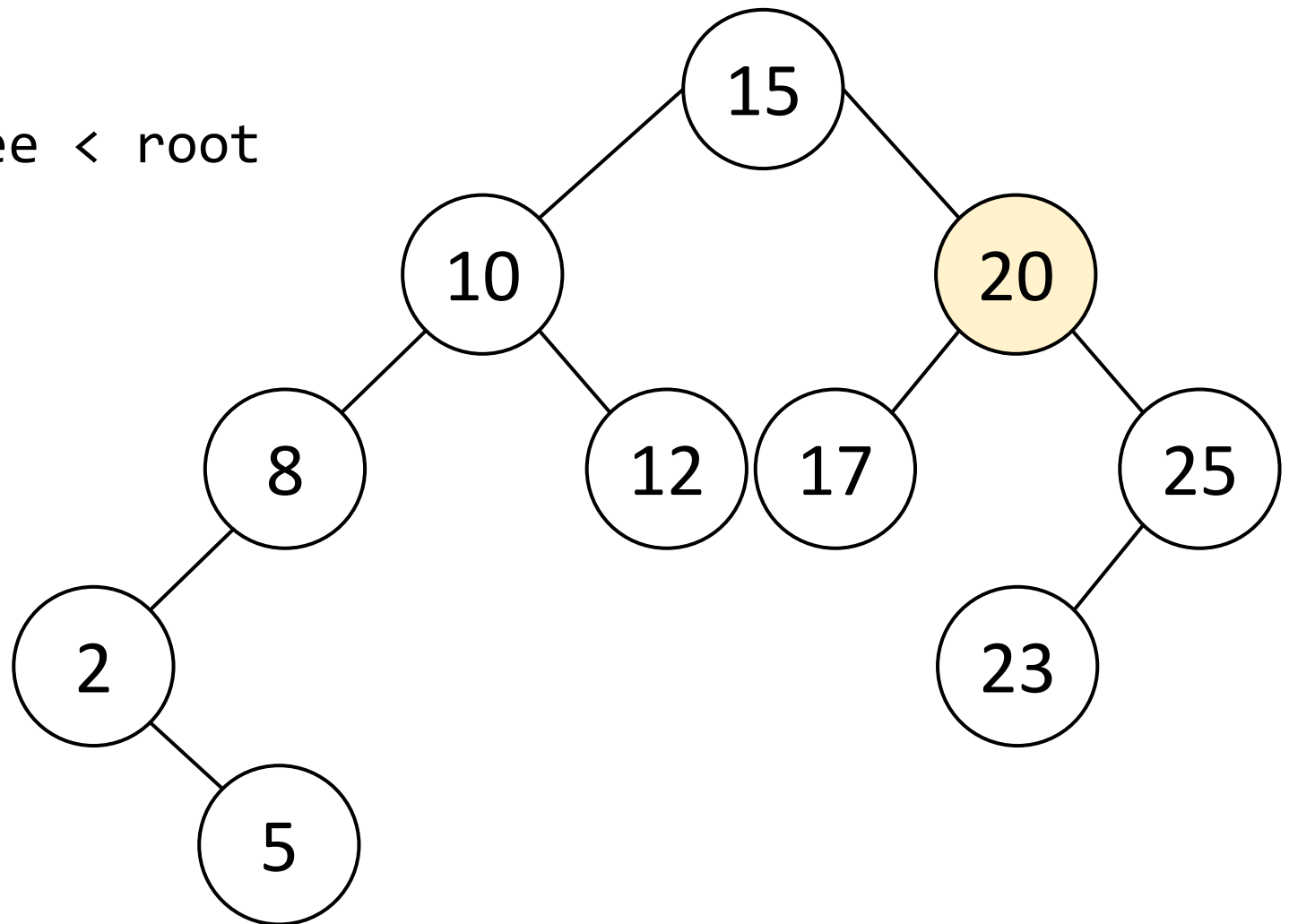
Maximum value in BST:

Idea:

Start from root (15)

All values in leftSubtree < root

Keep moving right!



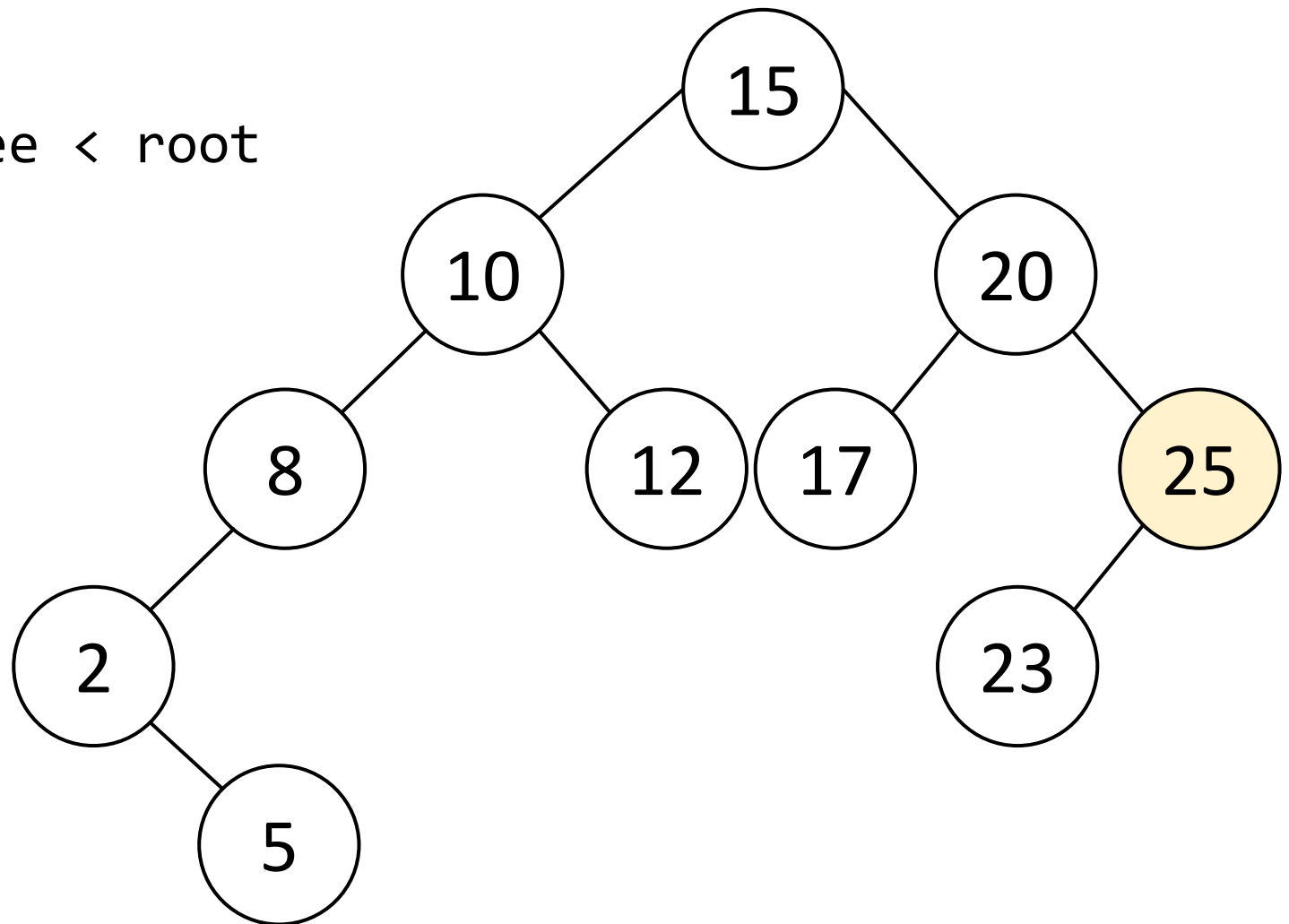
Maximum value in BST:

Idea:

Start from root (15)

All values in leftSubtree < root

Keep moving right!



Maximum value in BST:

Idea:

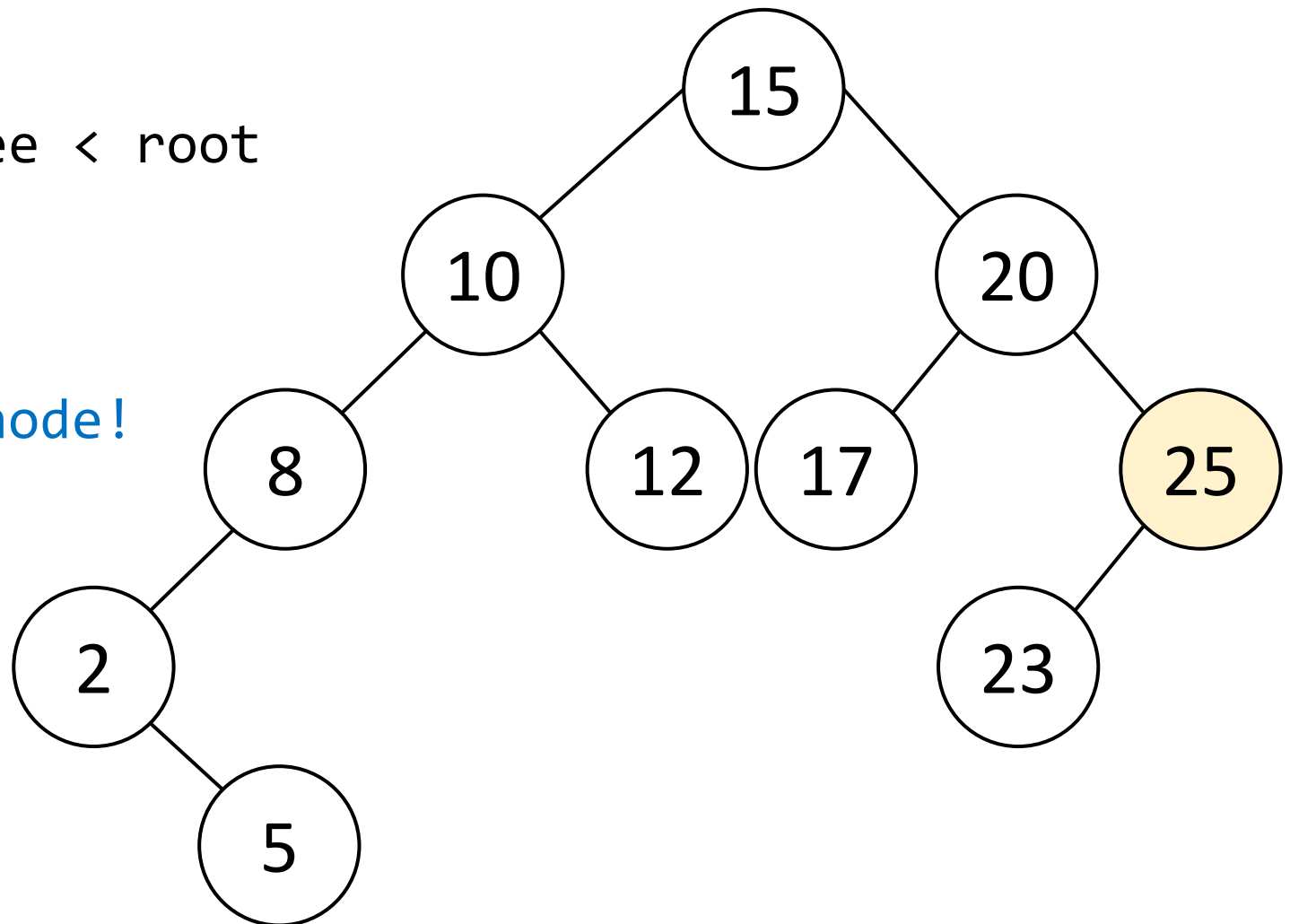
Start from root (15)

All values in leftSubtree < root

Keep moving right!

Can't move further

It must be the maximum node!



```
Node* tree_maximum(Node *x){
```

```
}
```

```
Node* tree_maximum(Node *x){  
  
    x = x->right  
  
    return x  
}
```

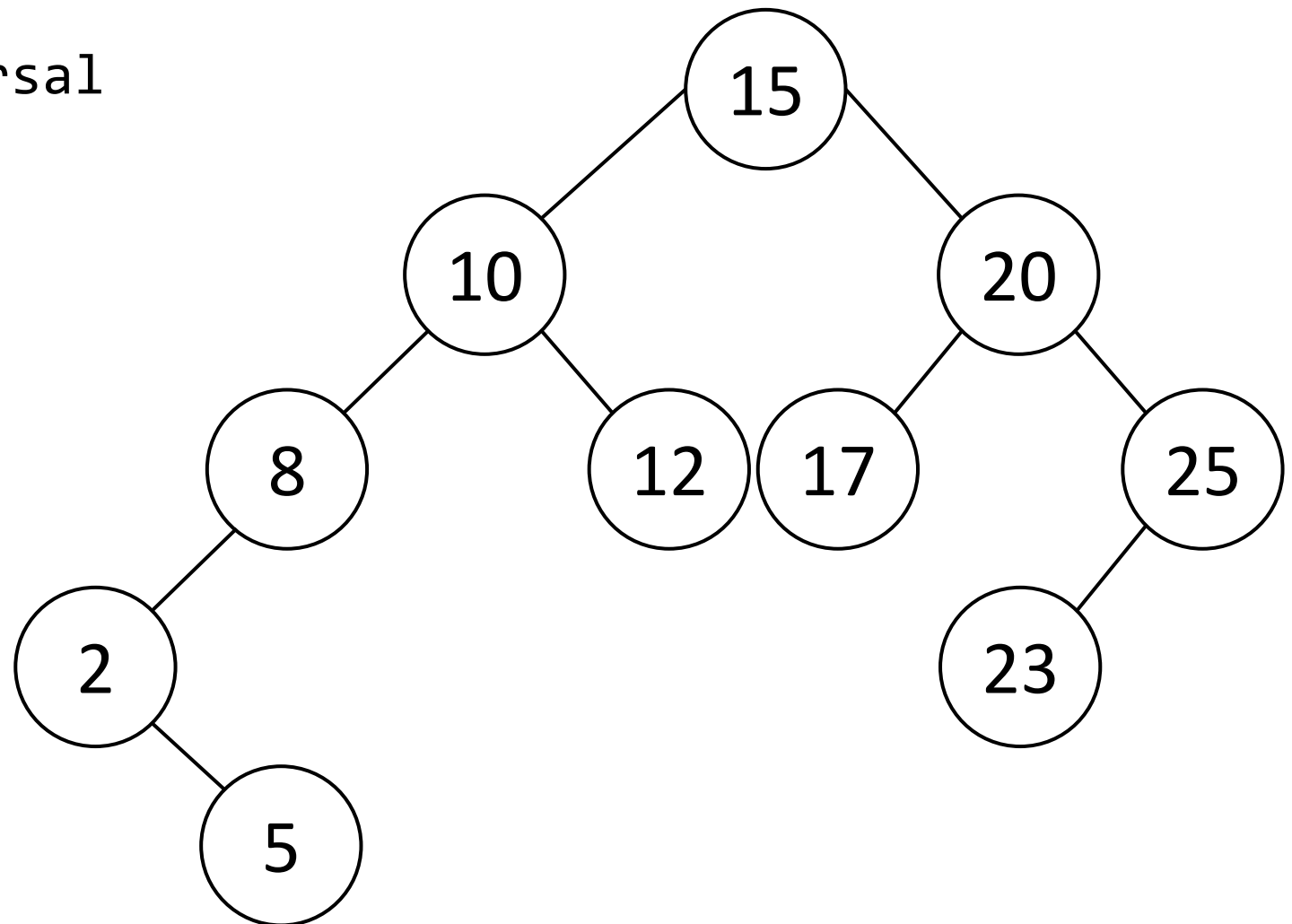
```
Node* tree_maximum(Node *x){  
  
    while(  
        x = x->right  
    )  
  
    return x  
}
```

```
Node* tree_maximum(Node *x){  
  
    while(x->right!= Null)  
        x = x->right  
  
    return x  
}
```

Print items of BST:

Idea:

Let's use inorder traversal



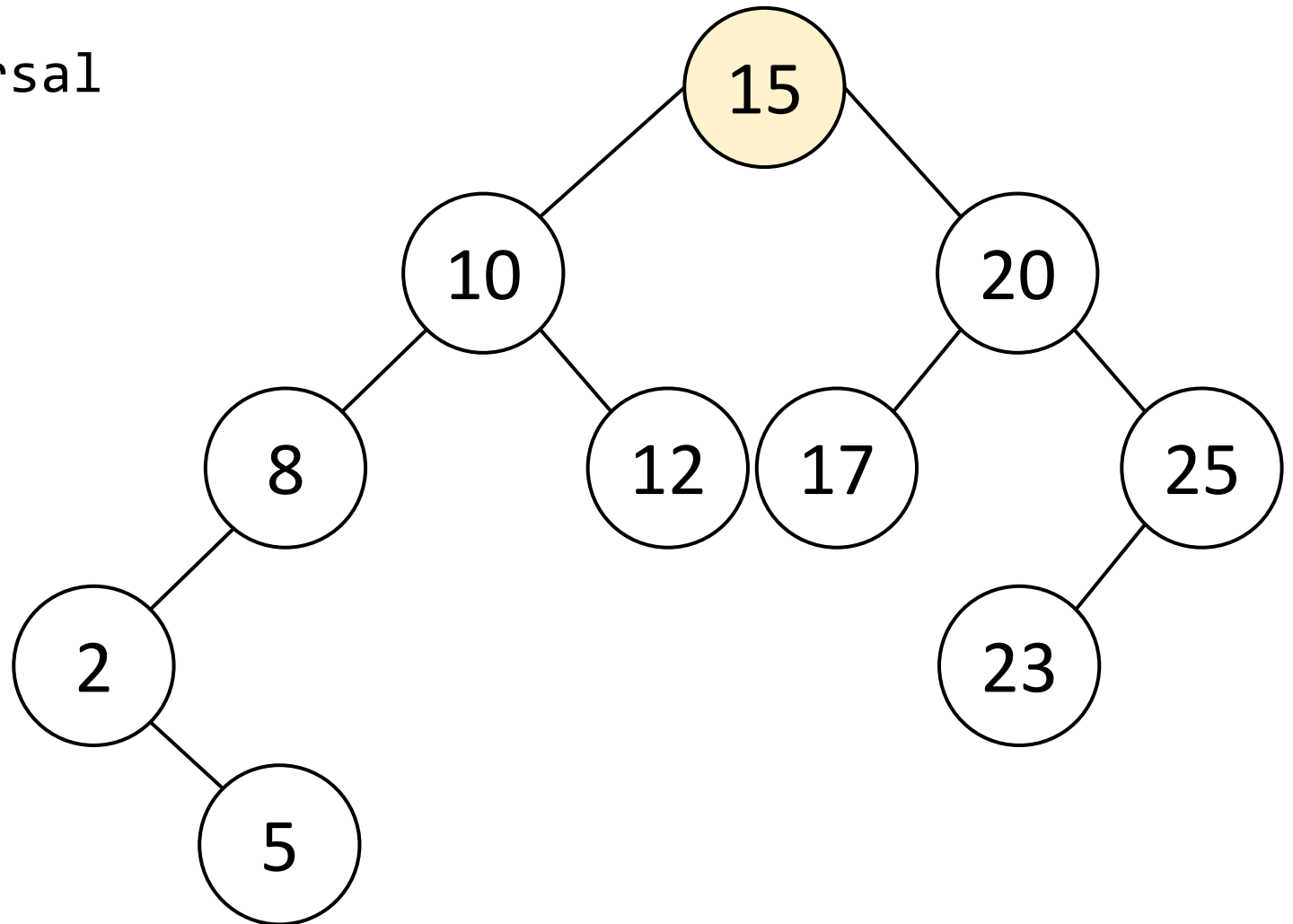
Print items of BST:

Idea:

Let's use inorder traversal

Printing order will be:
?

Step:
 $f(15)$



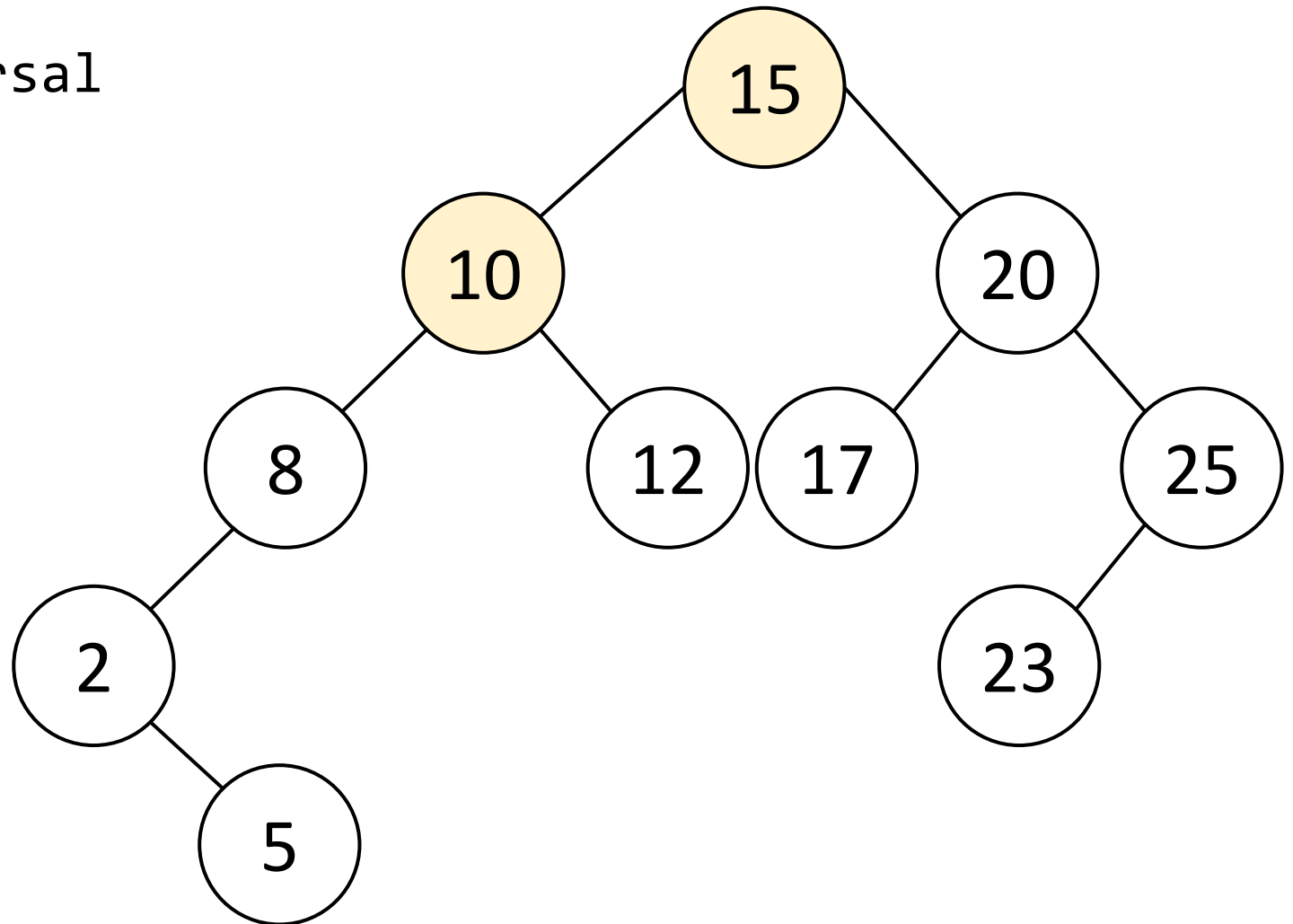
Print items of BST:

Idea:

Let's use inorder traversal

Printing order will be:
?

Step:
 $f(10)$



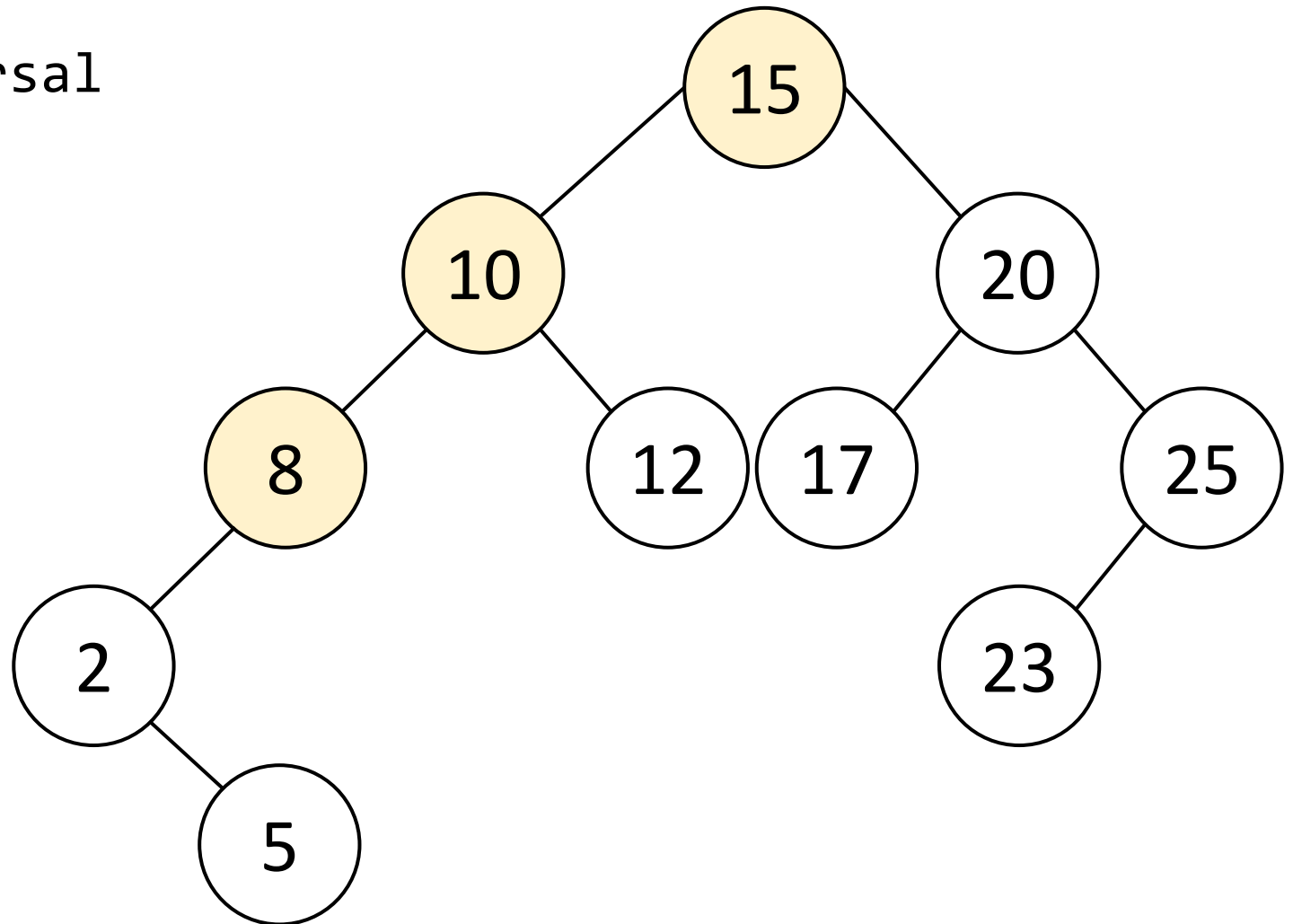
Print items of BST:

Idea:

Let's use inorder traversal

Printing order will be:
?

Step:
 $f(8)$



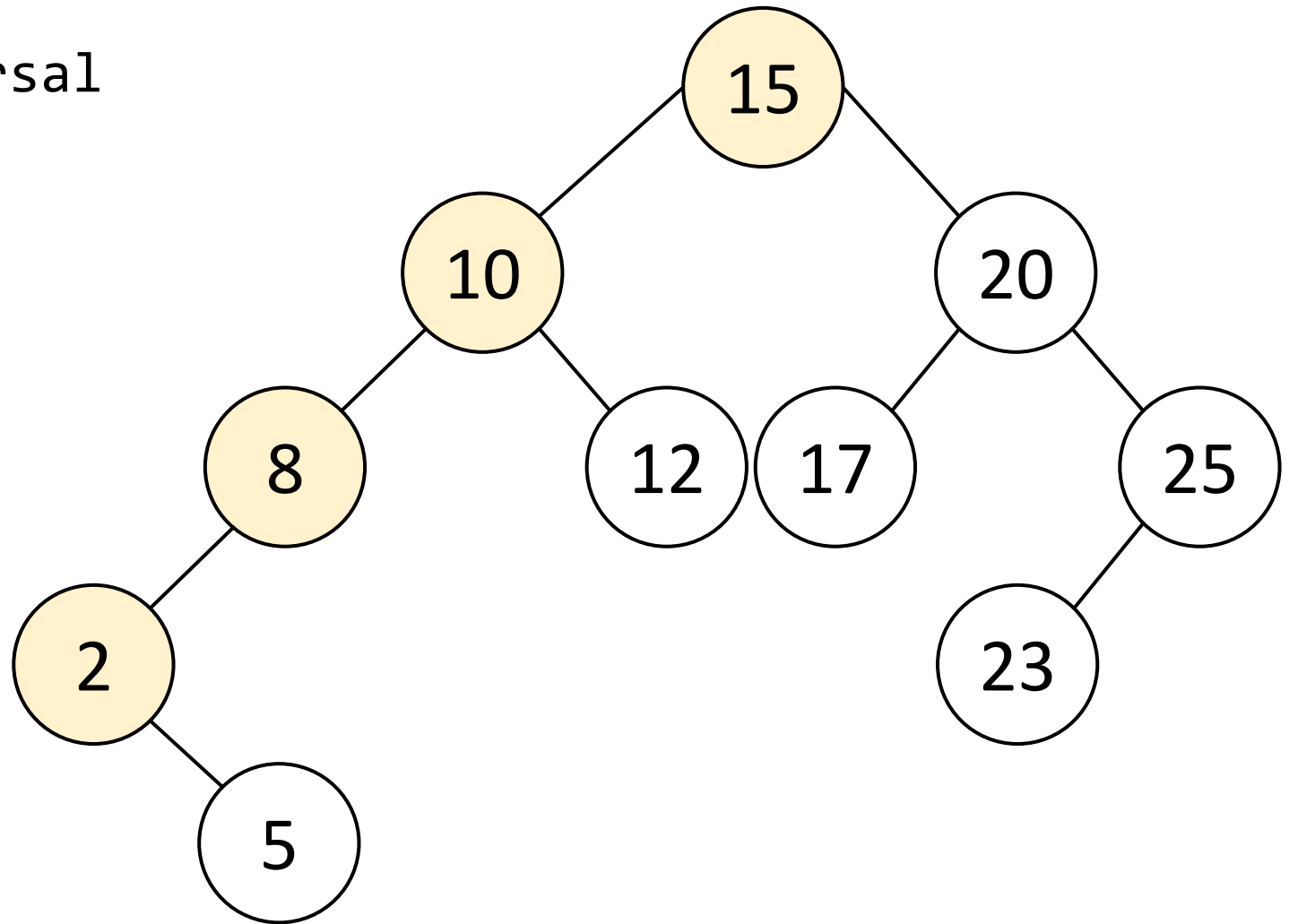
Print items of BST:

Idea:

Let's use inorder traversal

Printing order will be:
?

Step:
f(2)



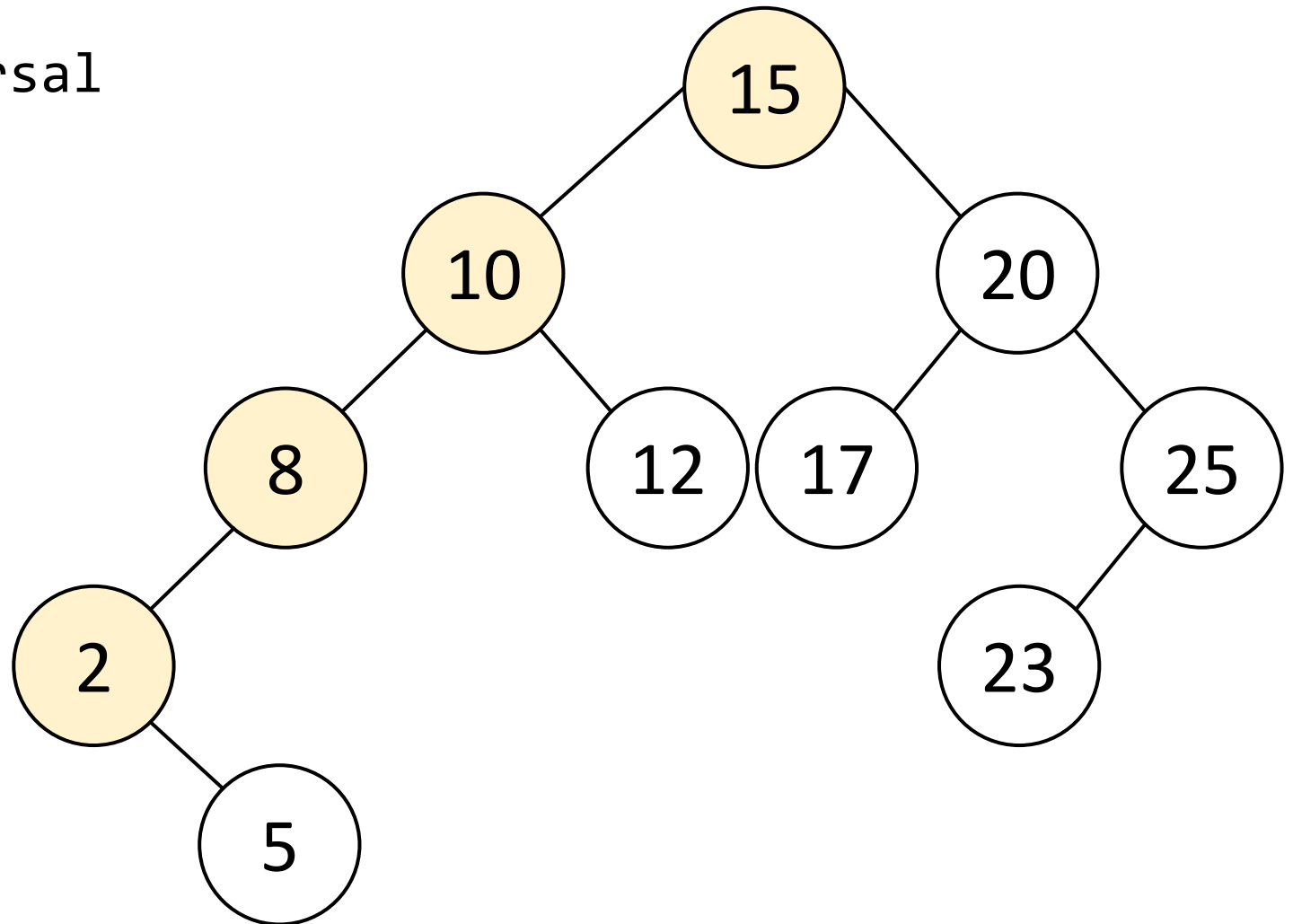
Print items of BST:

Idea:

Let's use inorder traversal

Printing order will be:
?

Step:
f(Null)



Print items of BST:

Idea:

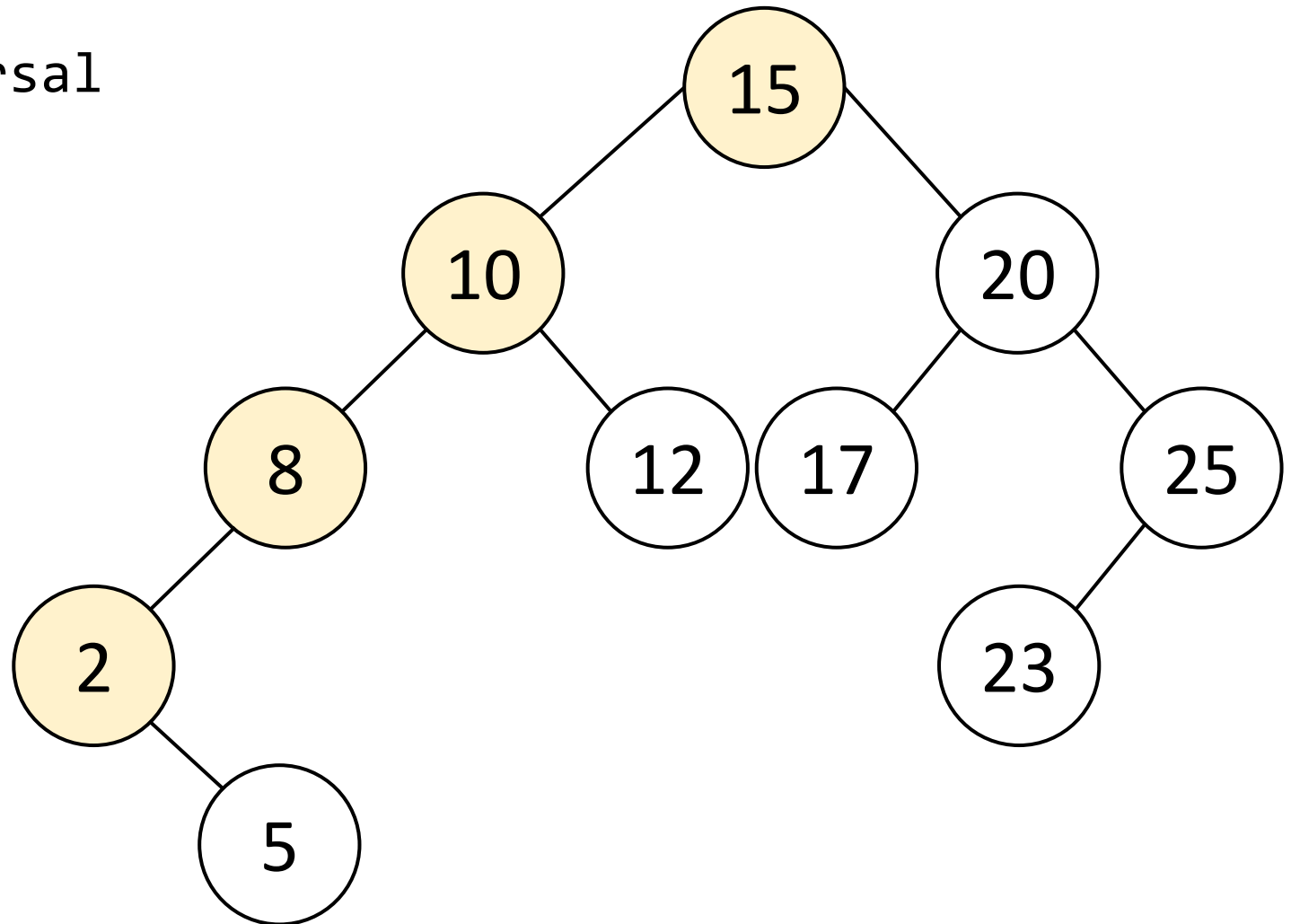
Let's use inorder traversal

Printing order will be:

2

Step:

Back to $f(2)$



Print items of BST:

Idea:

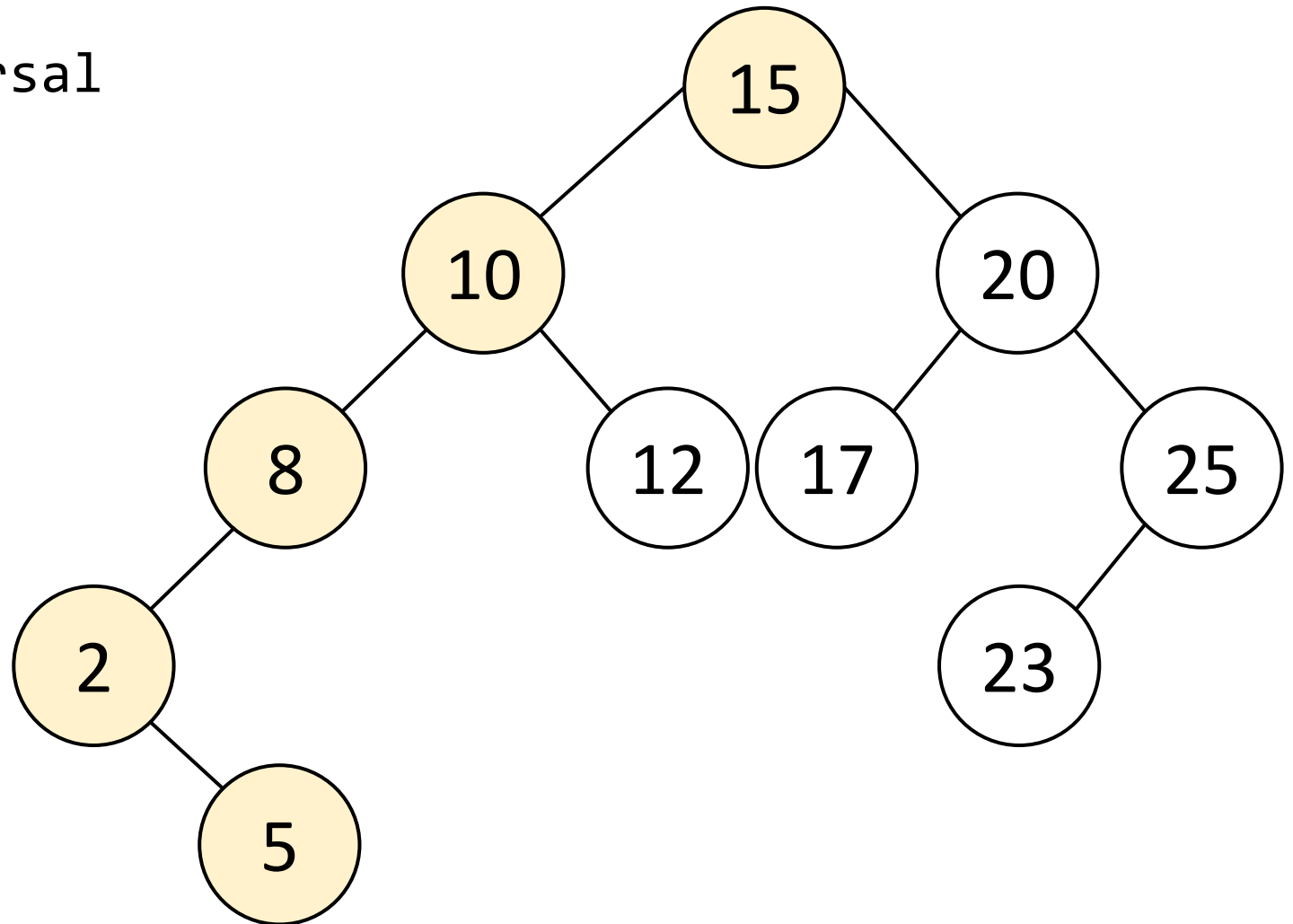
Let's use inorder traversal

Printing order will be:

2

Step:

f(5)



Print items of BST:

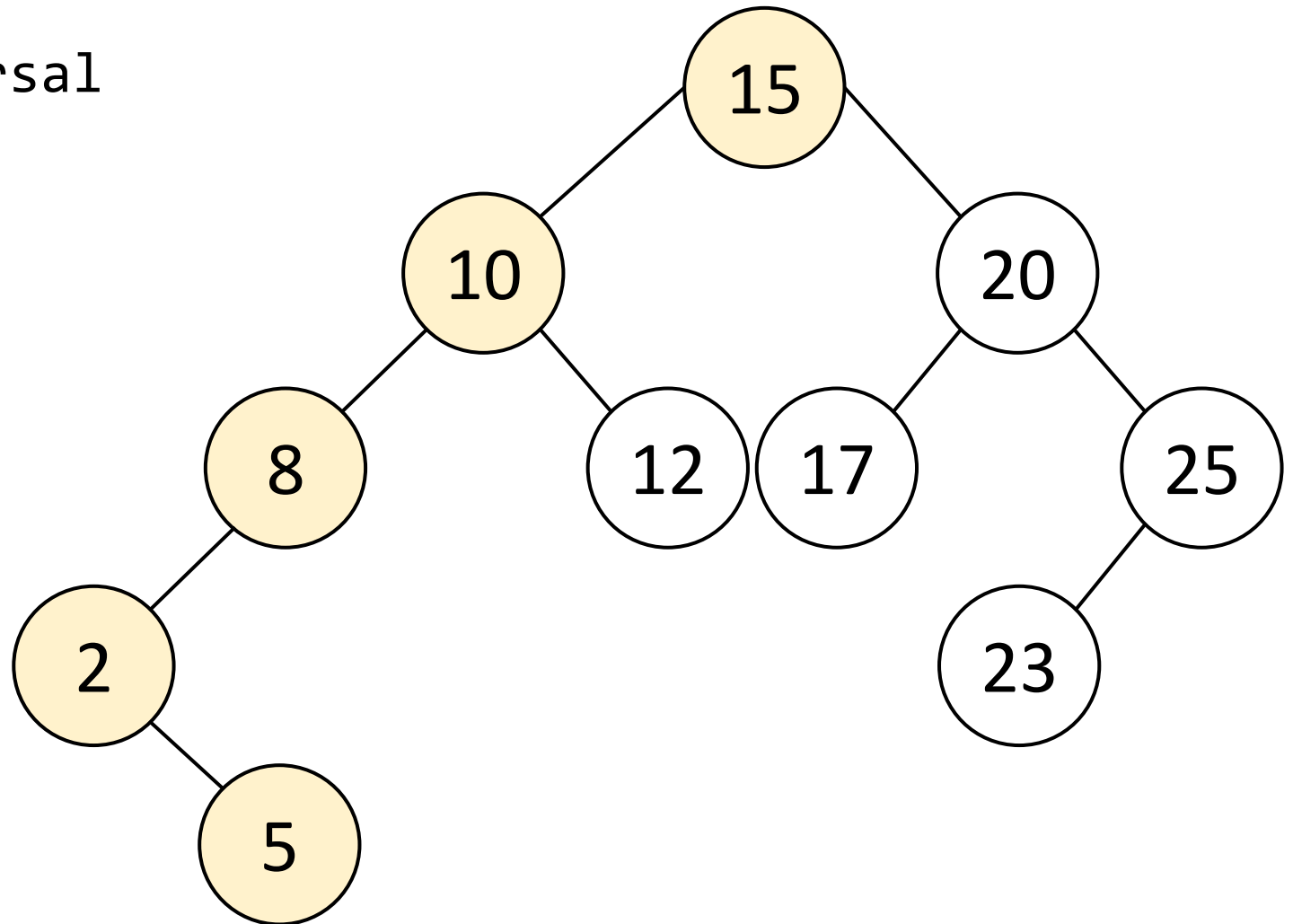
Idea:

Let's use inorder traversal

Printing order will be:

2

Step:
f(Null)



Print items of BST:

Idea:

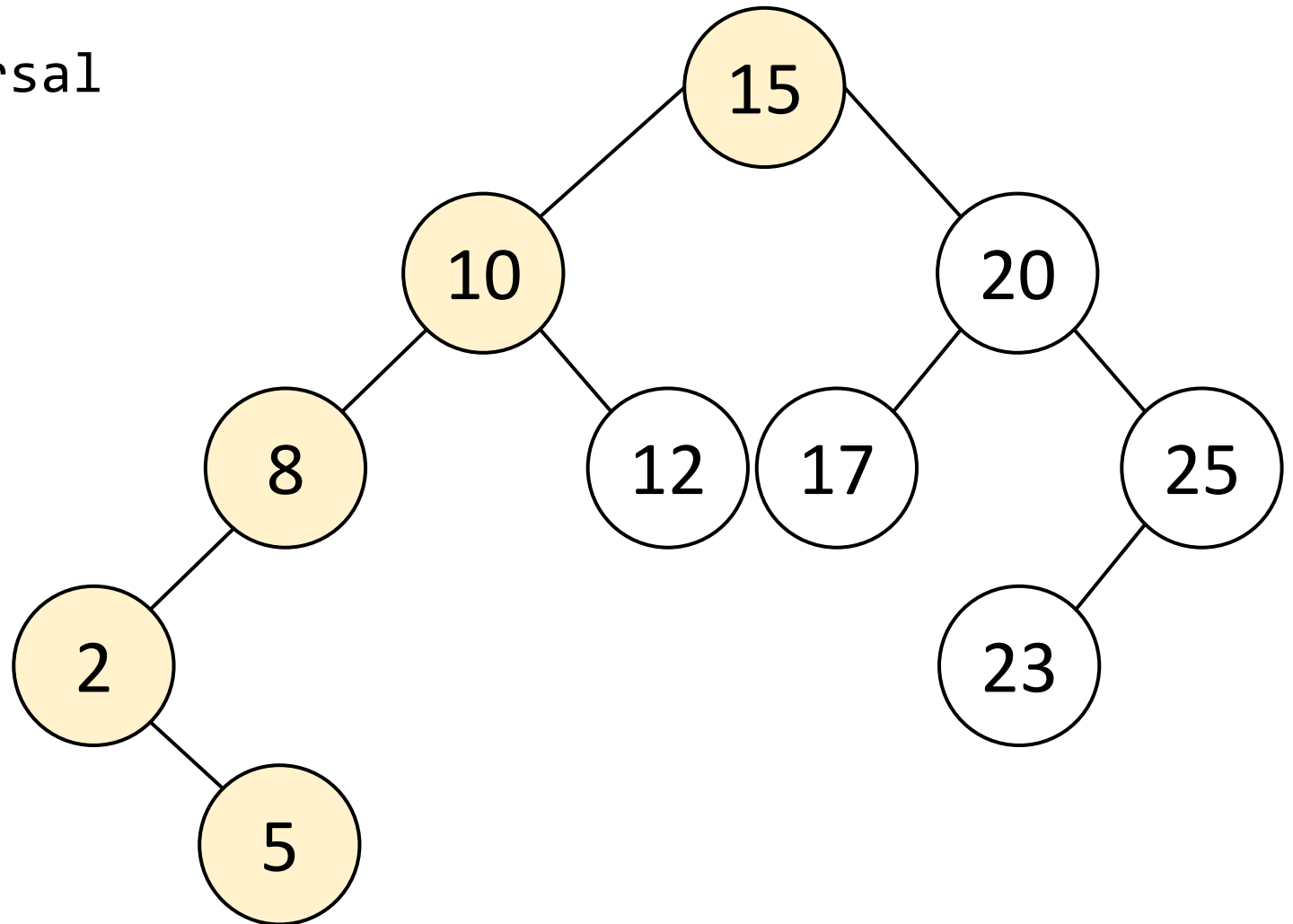
Let's use inorder traversal

Printing order will be:

2 5

Step:

Back to f(5)



Print items of BST:

Idea:

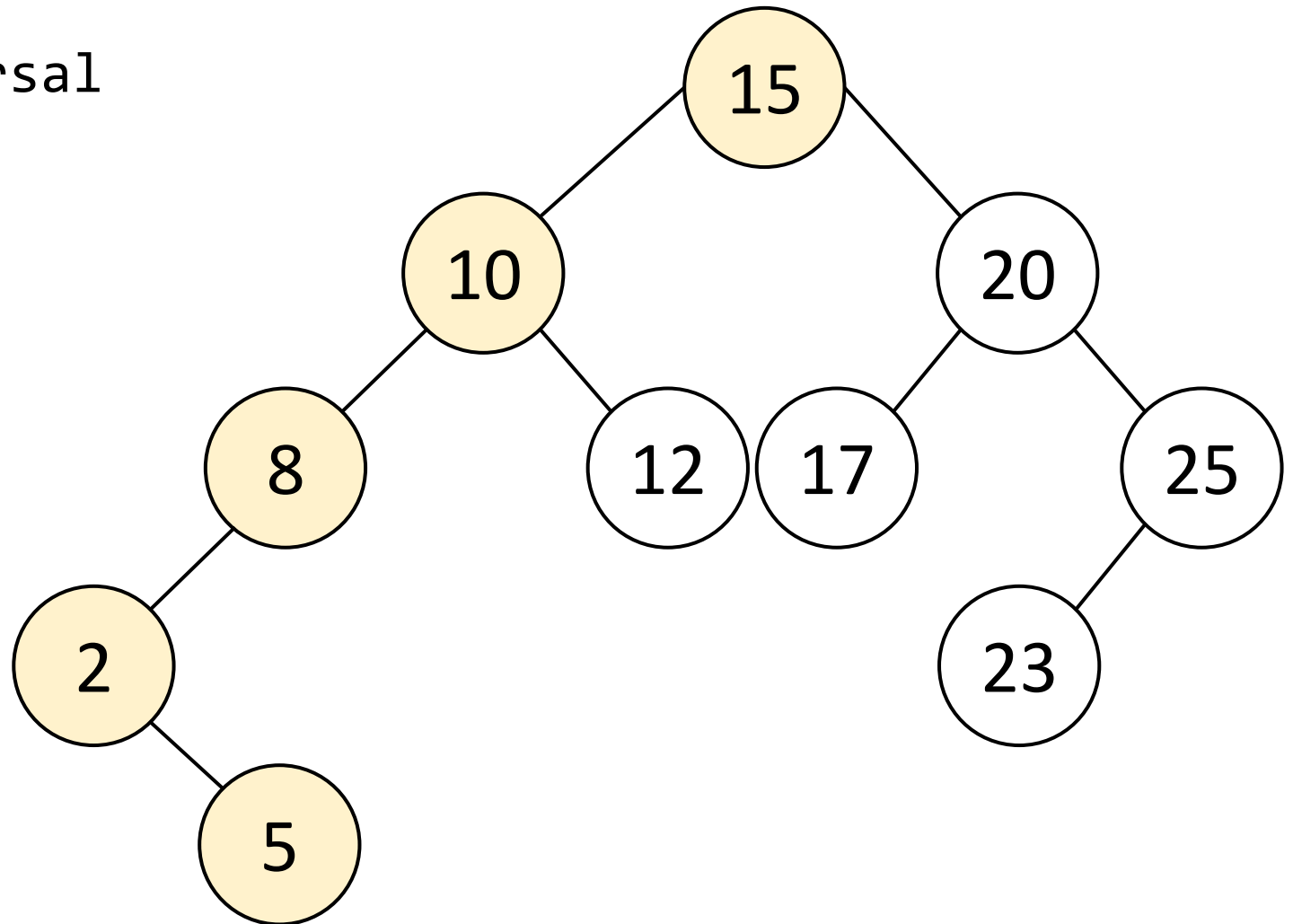
Let's use inorder traversal

Printing order will be:

2 5

Step:

f(Null)



Print items of BST:

Idea:

Let's use inorder traversal

Printing order will be:

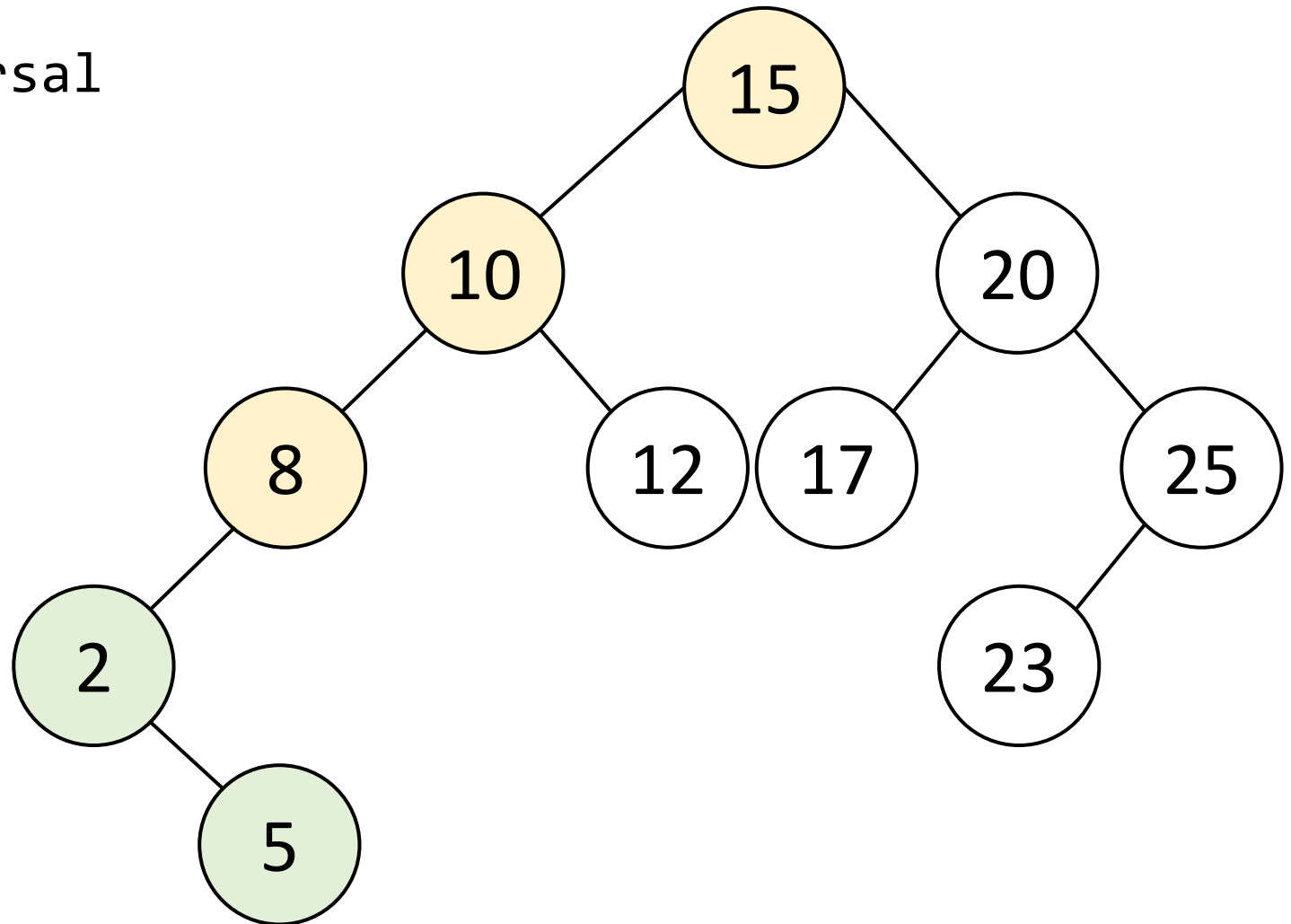
2 5

Step:

Back to f(5)

Back to f(2)

Back to f(8)



Print items of BST:

Idea:

Let's use inorder traversal

Printing order will be:

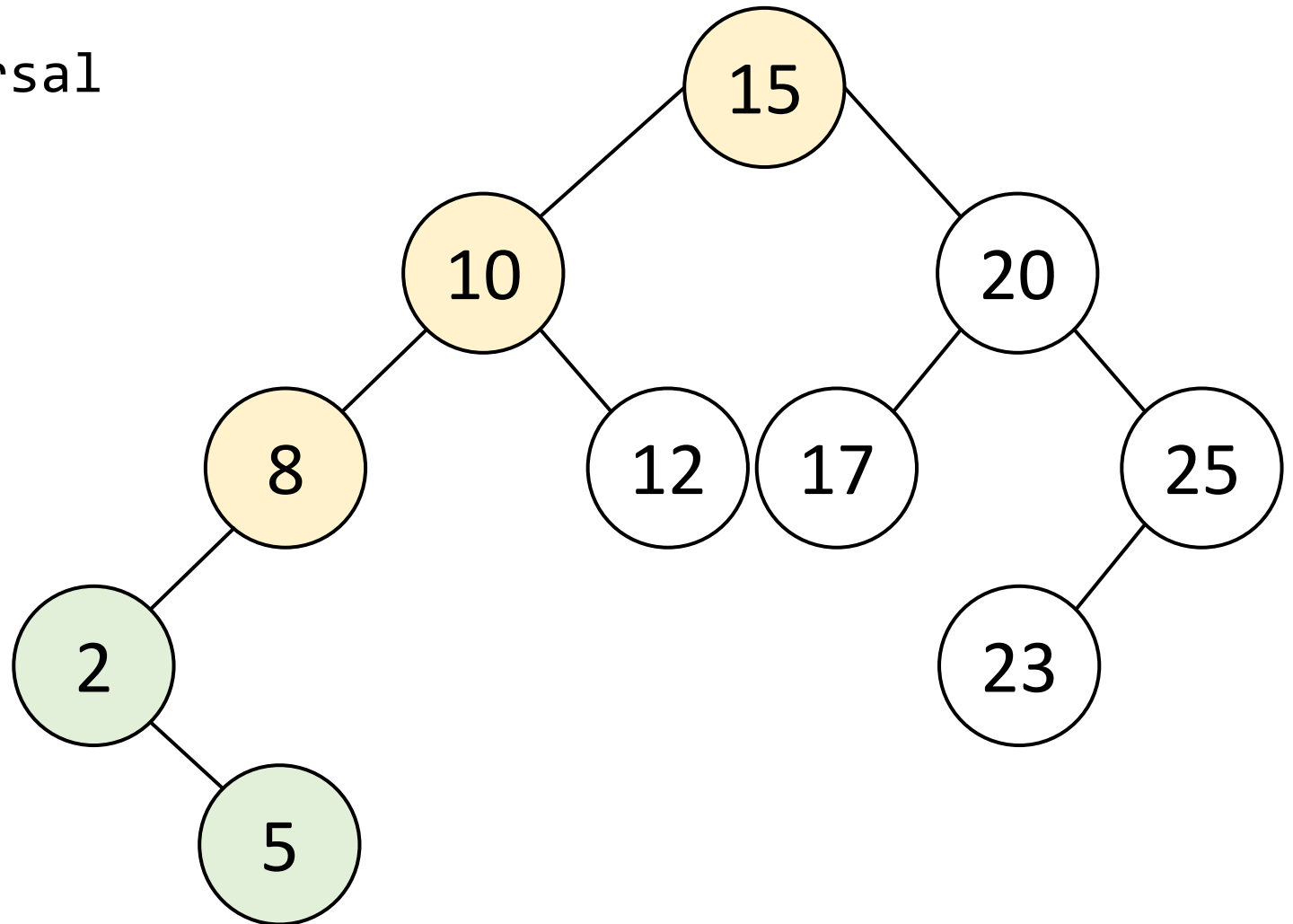
2 5 8

Step:

Back to f(5)

Back to f(2)

Back to f(8)



Print items of BST:

Idea:

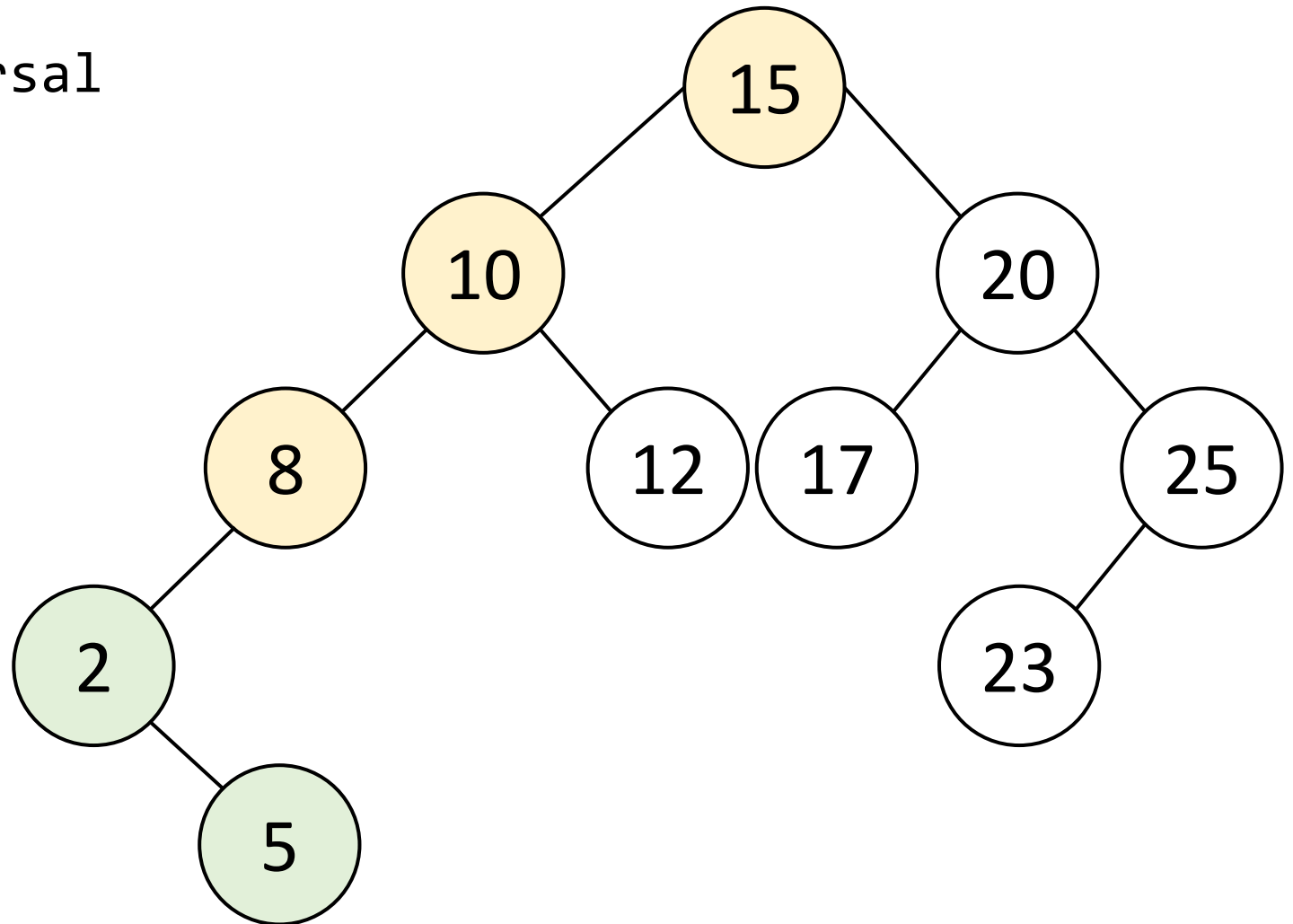
Let's use inorder traversal

Printing order will be:

2 5 8

Step:

f(8->right)



Print items of BST:

Idea:

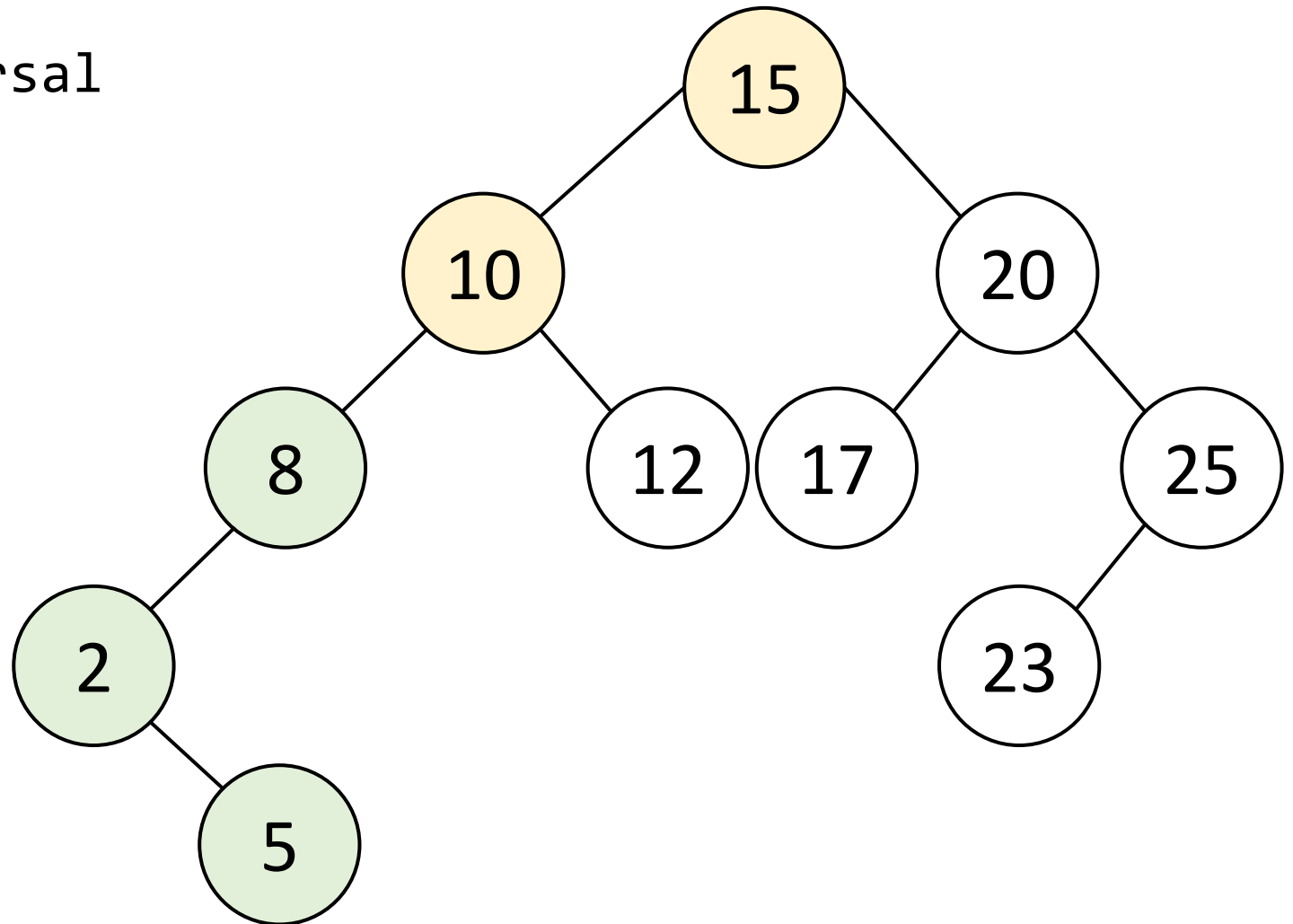
Let's use inorder traversal

Printing order will be:

2 5 8 10

Step:

Back to $f(10)$



Print items of BST:

Idea:

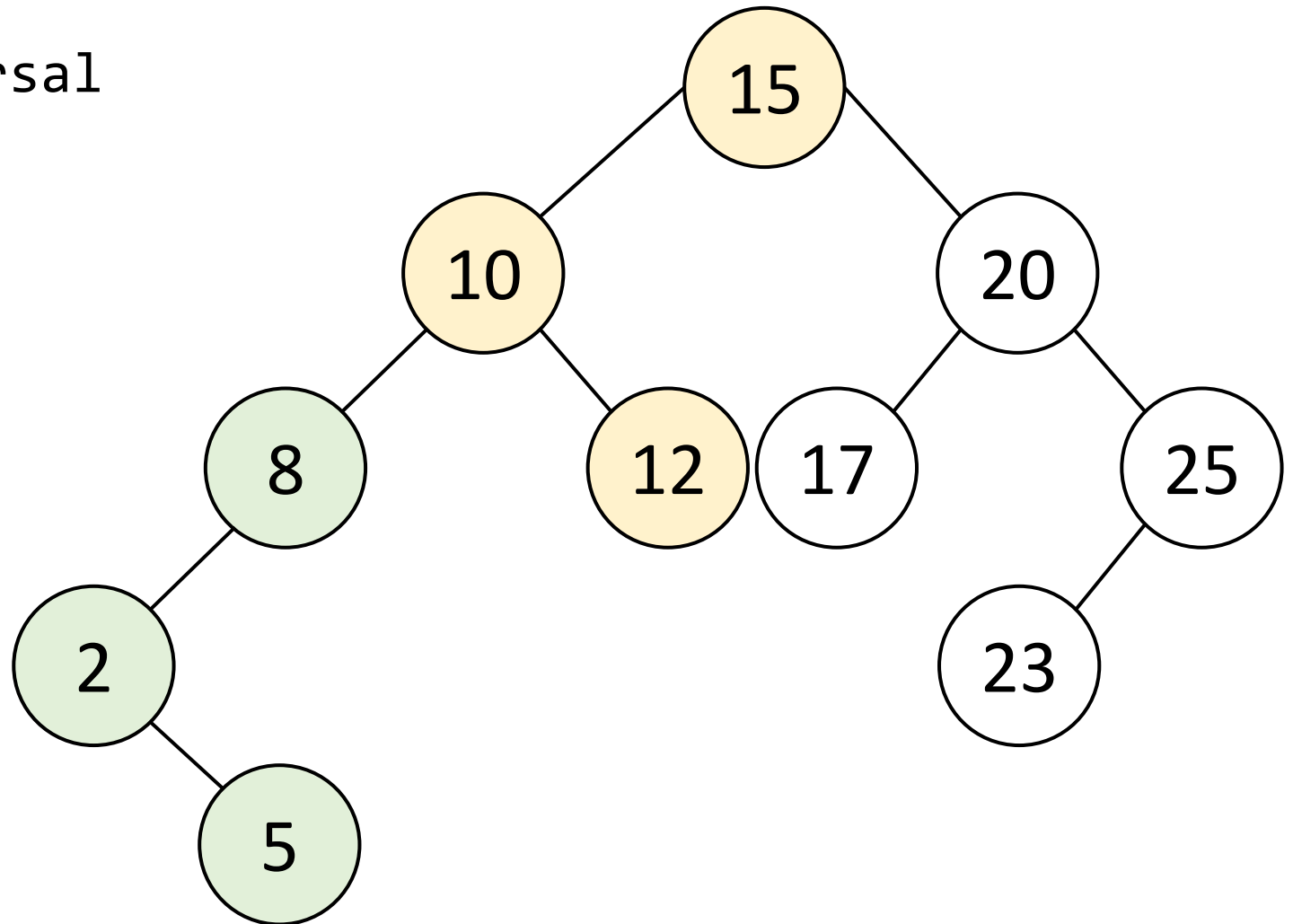
Let's use inorder traversal

Printing order will be:

2 5 8 10

Step:

f(12)



Print items of BST:

Idea:

Let's use inorder traversal

Printing order will be:

2 5 8 10 12

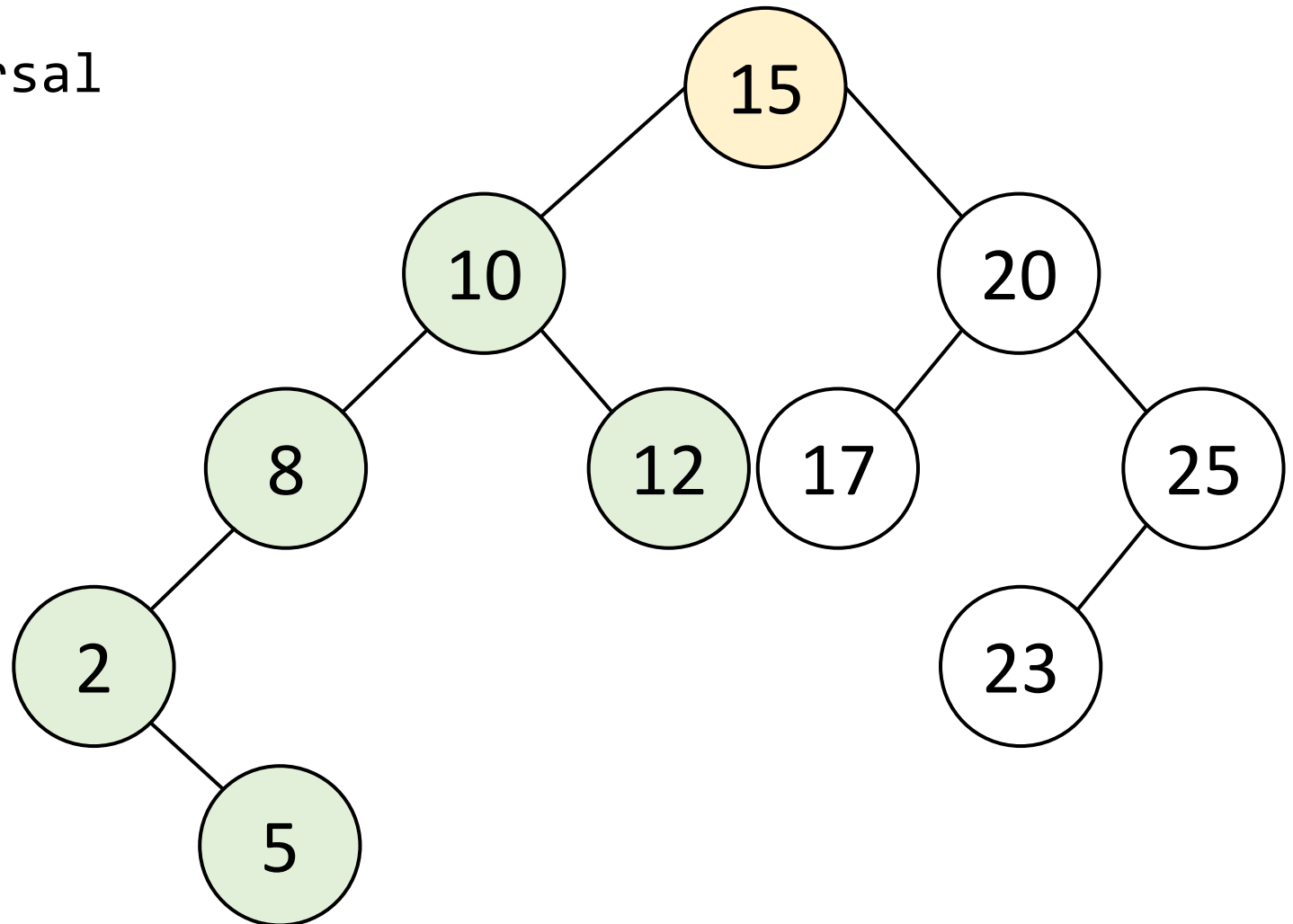
Step:

Check left,

Print 12

right of 12

Go back to 10



Print items of BST:

Idea:

Let's use inorder traversal

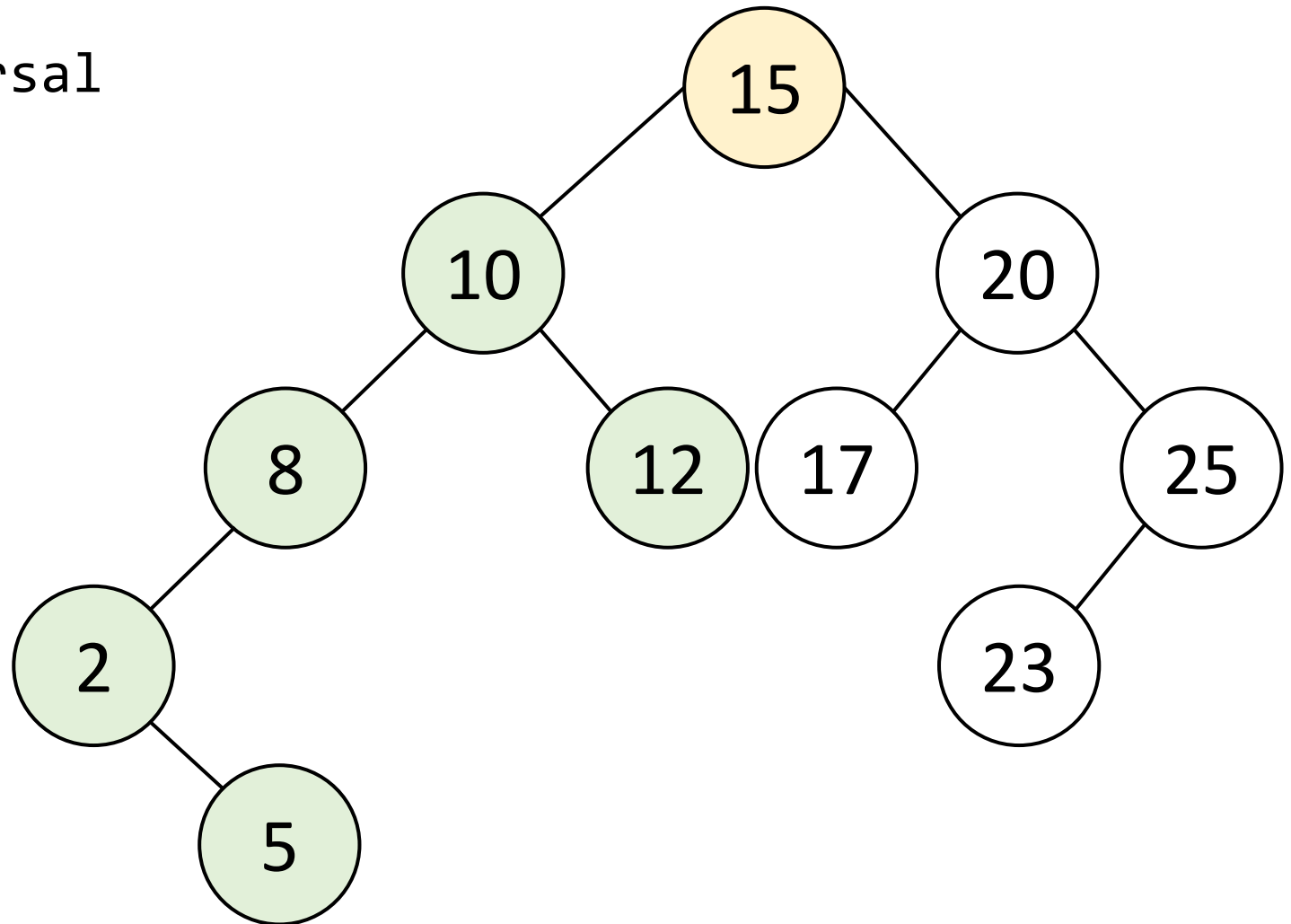
Printing order will be:

2 5 8 10 12 15

Step:

Go back to 15

Print 15



Print items of BST:

Idea:

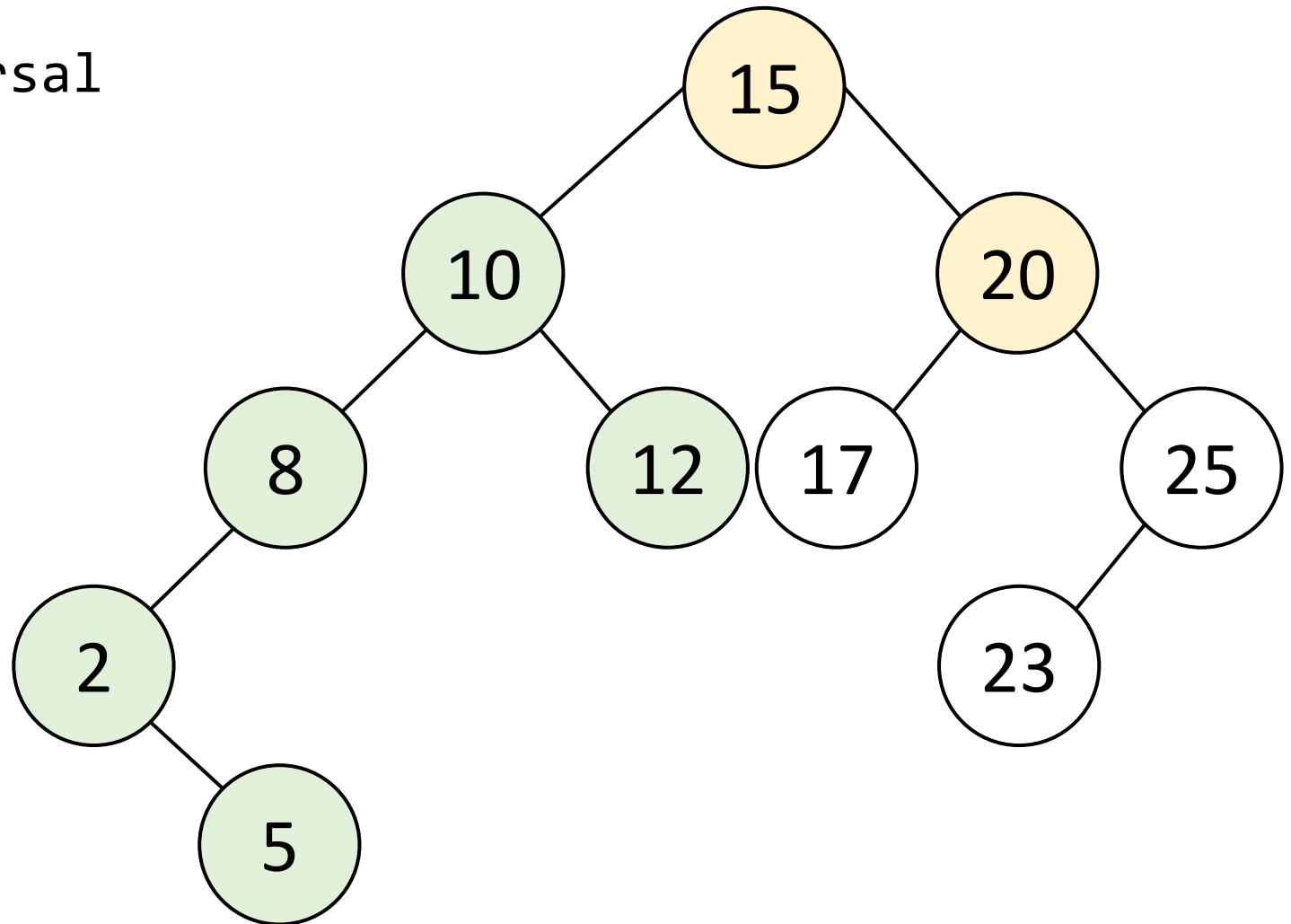
Let's use inorder traversal

Printing order will be:

2 5 8 10 12 15

Step:

F(20)



Print items of BST:

Idea:

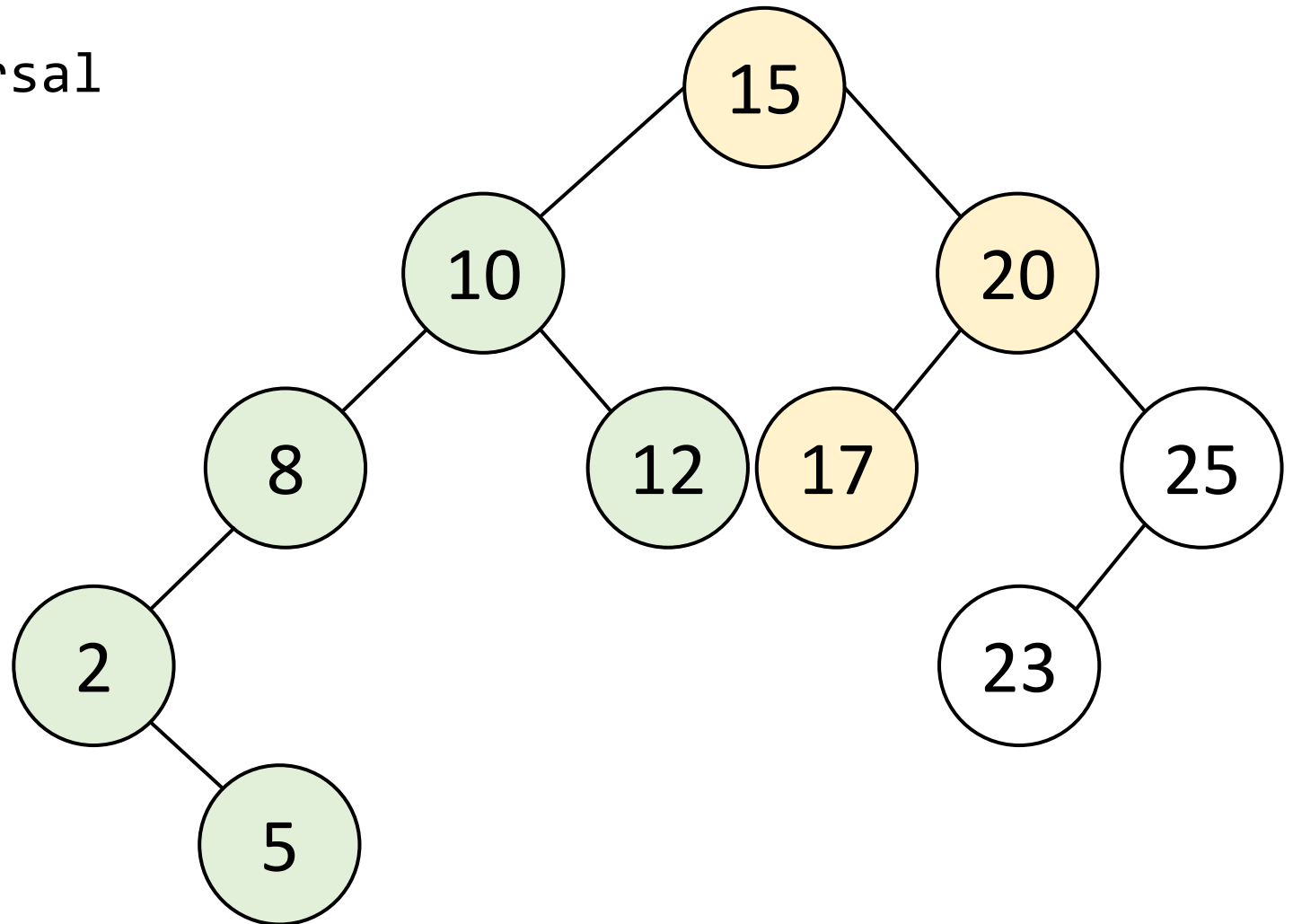
Let's use inorder traversal

Printing order will be:

2 5 8 10 12 15

Step:

F(17)



Print items of BST:

Idea:

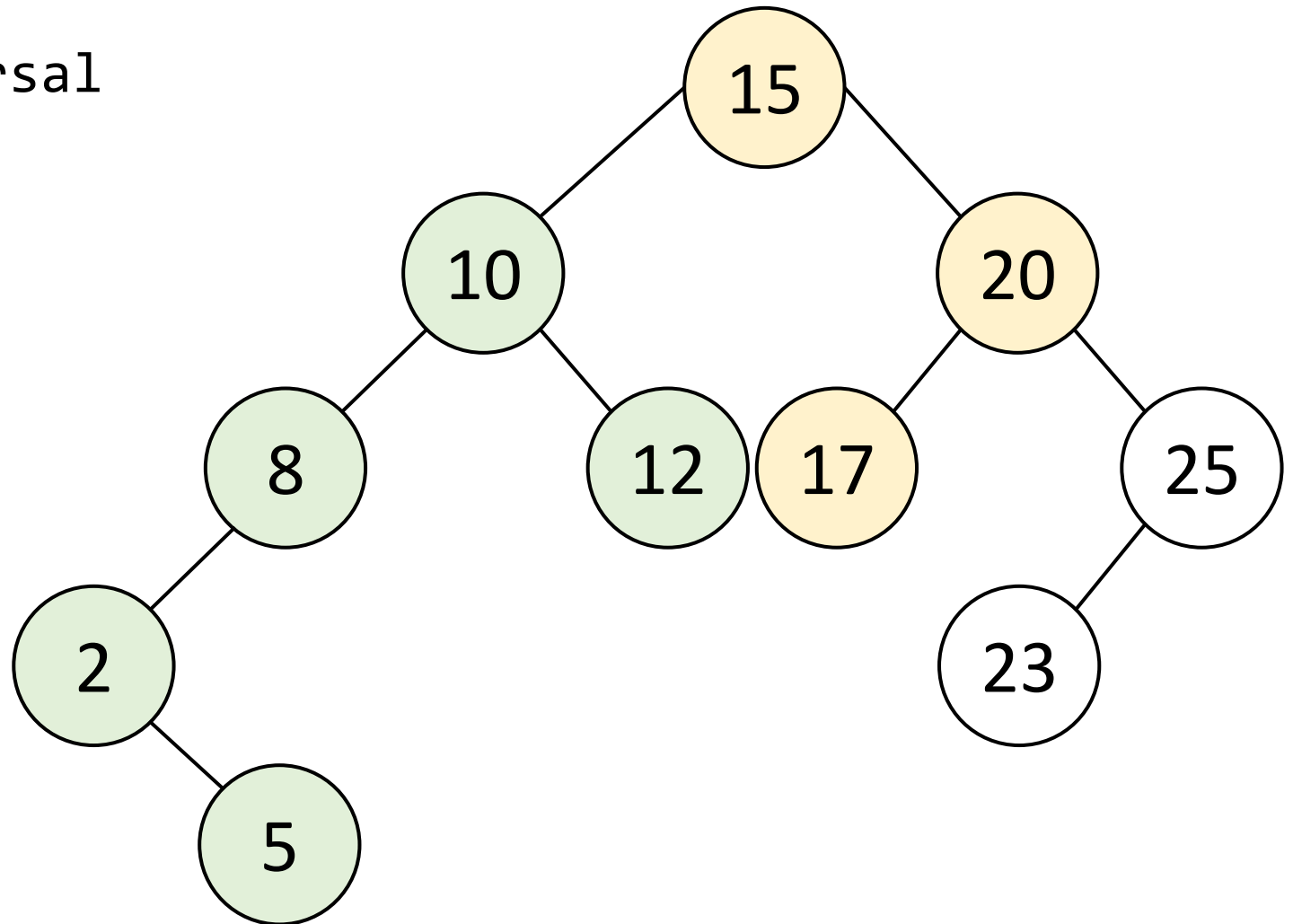
Let's use inorder traversal

Printing order will be:

2 5 8 10 12 15

Step:

Check 17->left



Print items of BST:

Idea:

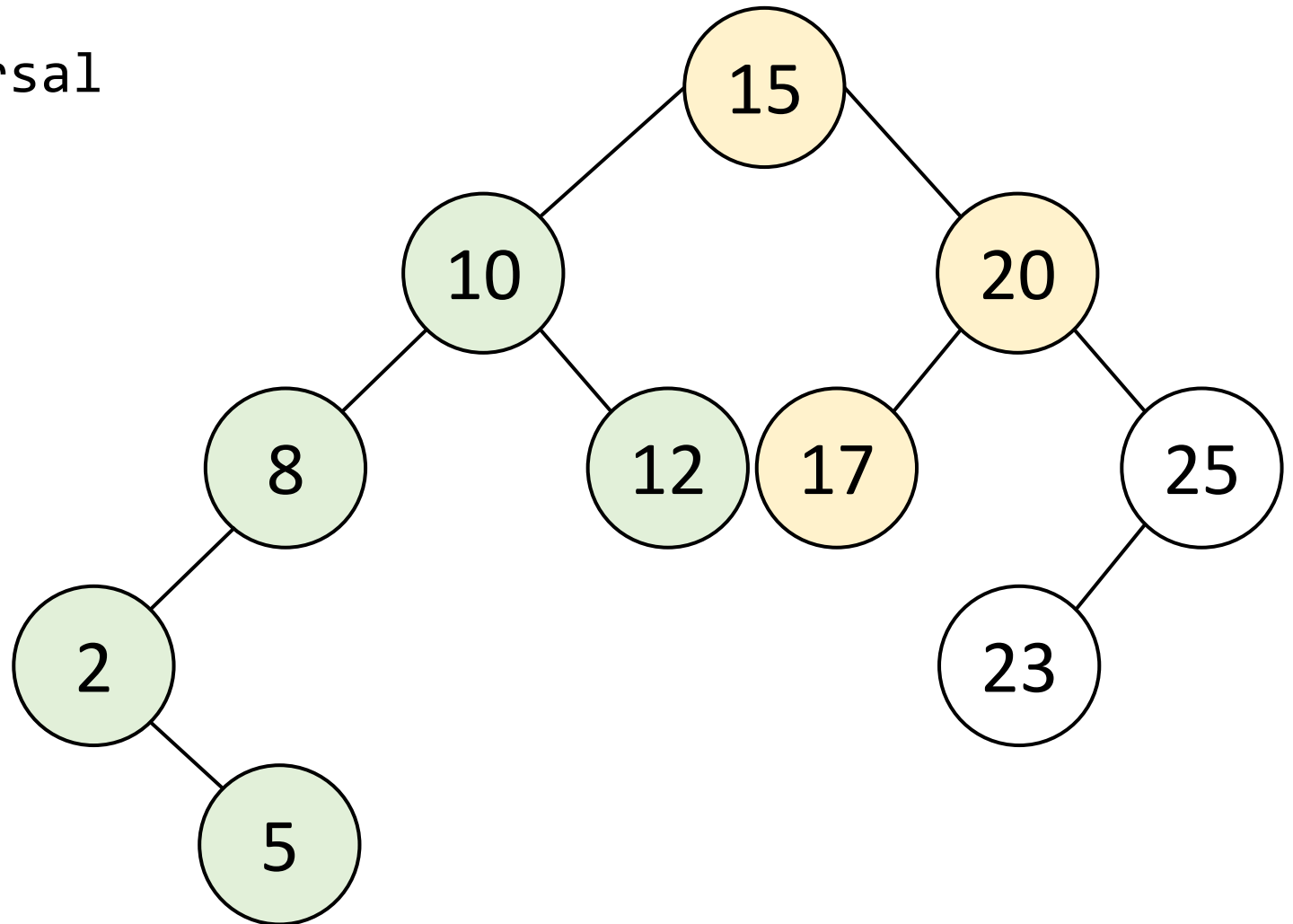
Let's use inorder traversal

Printing order will be:

2 5 8 10 12 15 17

Step:

Print 17



Print items of BST:

Idea:

Let's use inorder traversal

Printing order will be:

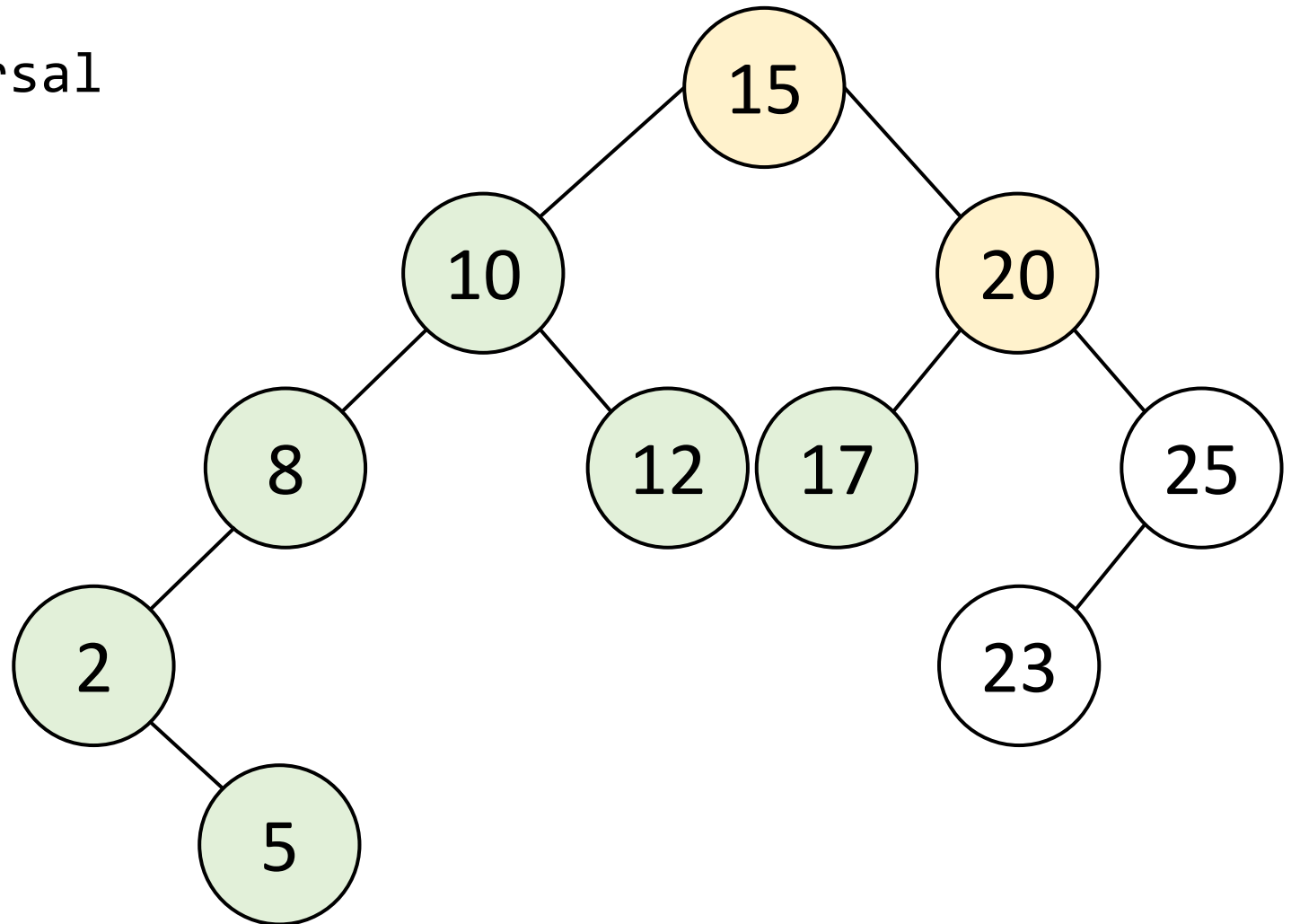
2 5 8 10 12 15 17 20

Step:

Check 17->right

Go back to 20

Print 20



Print items of BST:

Idea:

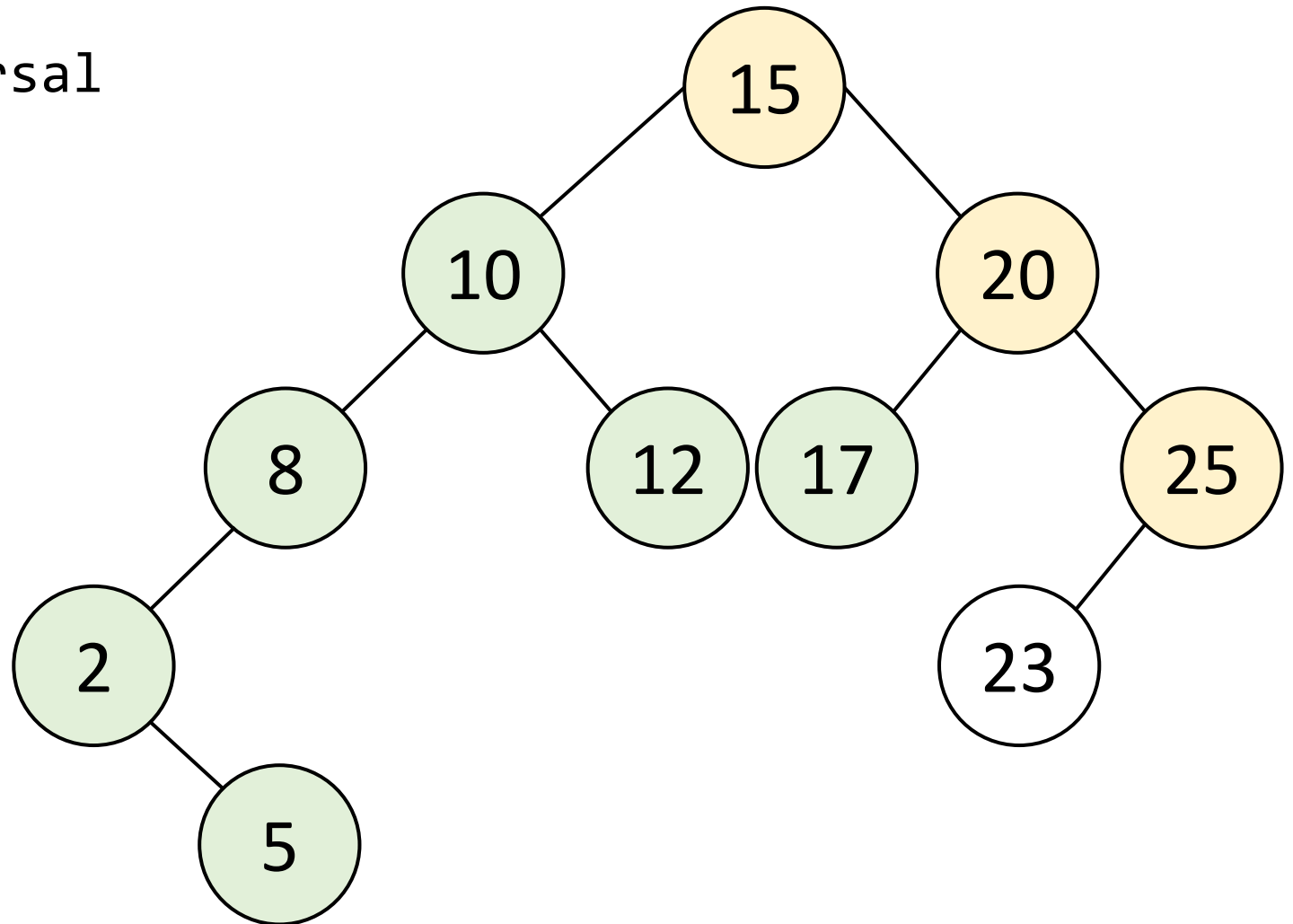
Let's use inorder traversal

Printing order will be:

2 5 8 10 12 15 17 20

Step:

F(25)



Print items of BST:

Idea:

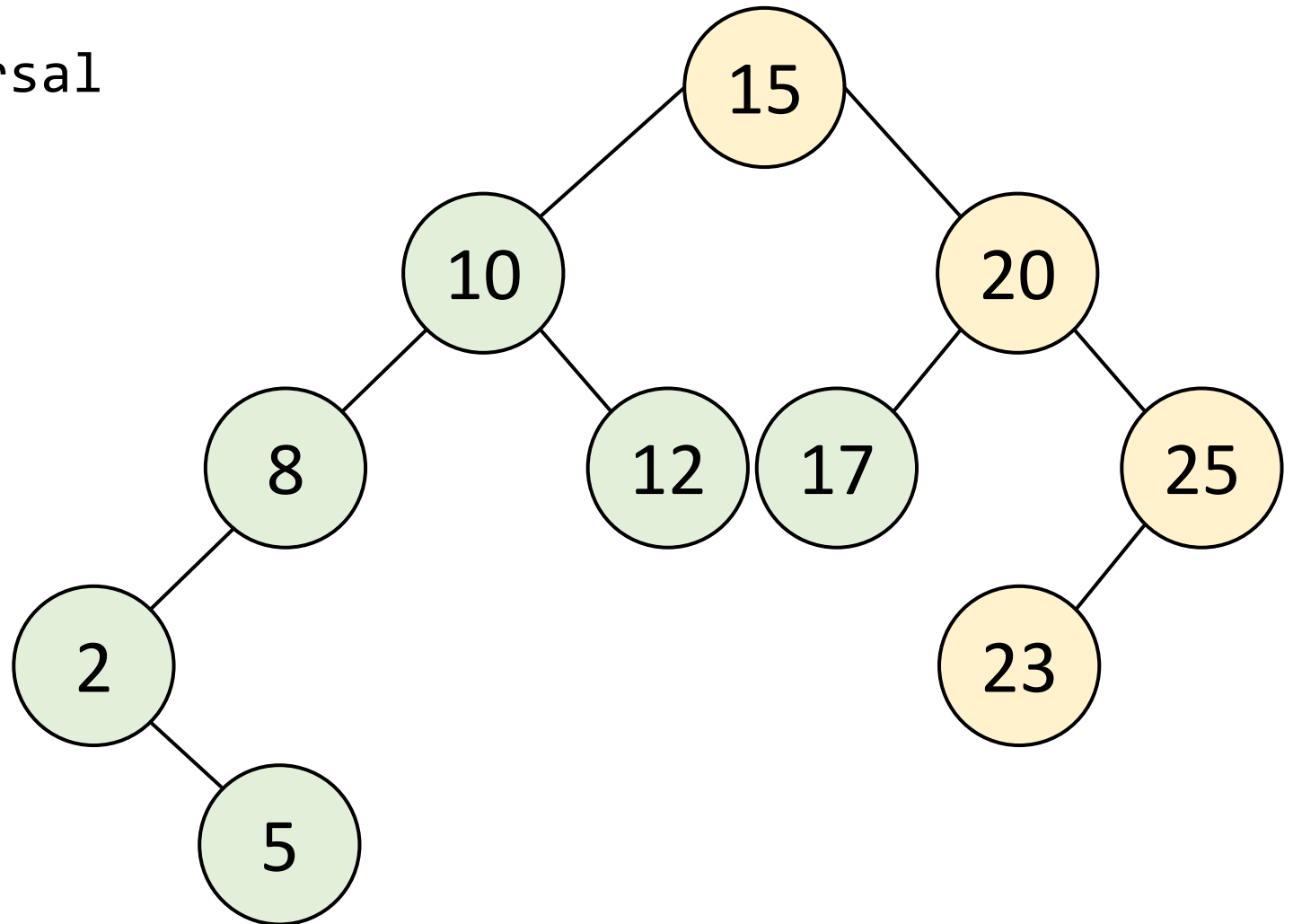
Let's use inorder traversal

Printing order will be:

2 5 8 10 12 15 17 20 23 25

Step:

F(23)



Print items of BST:

Idea:

Let's use inorder traversal

Printing order will be:

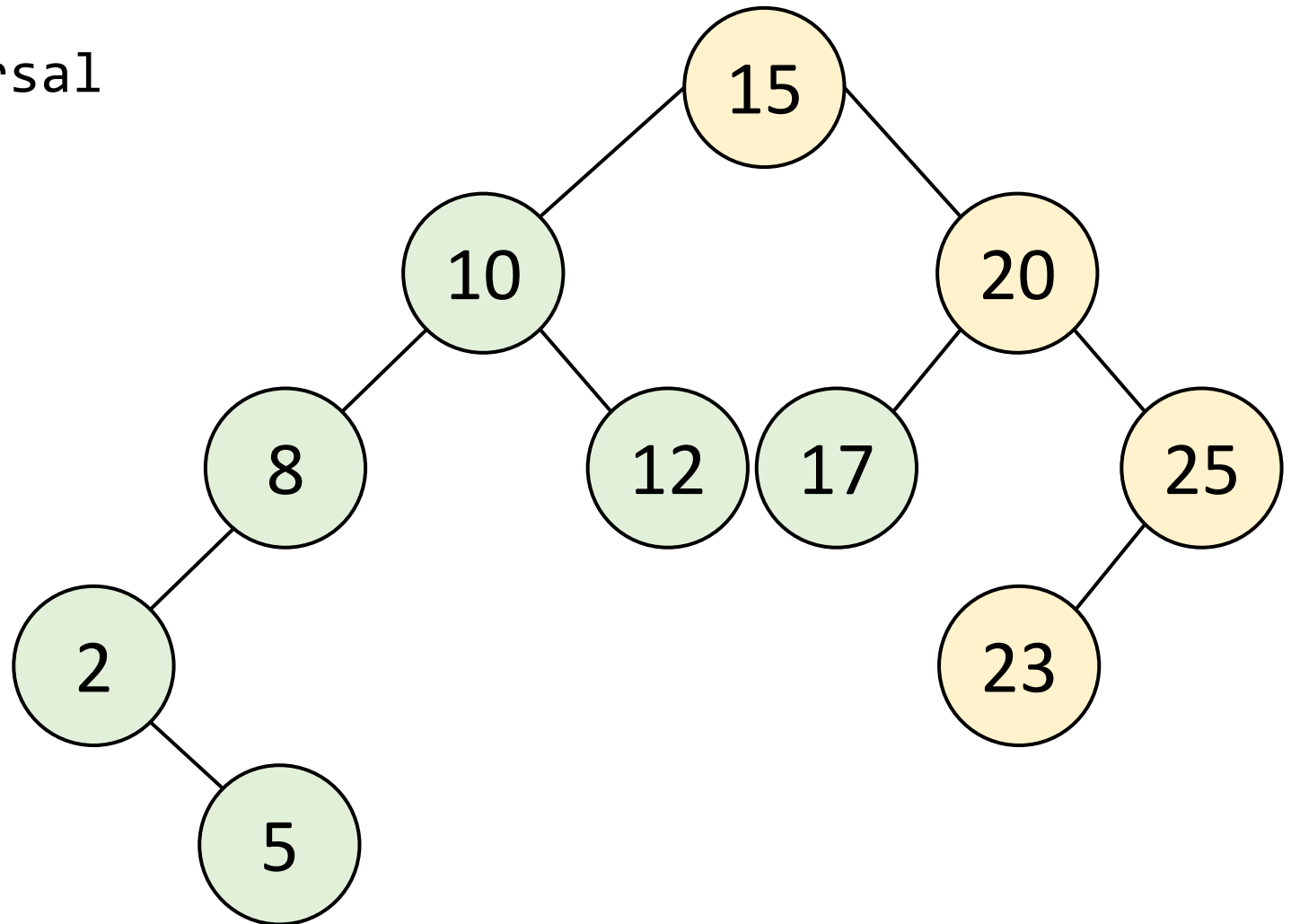
2 5 8 10 12 15 17 20 23

Step:

Check 23->left

Print 23

Check 23->right



Print items of BST:

Idea:

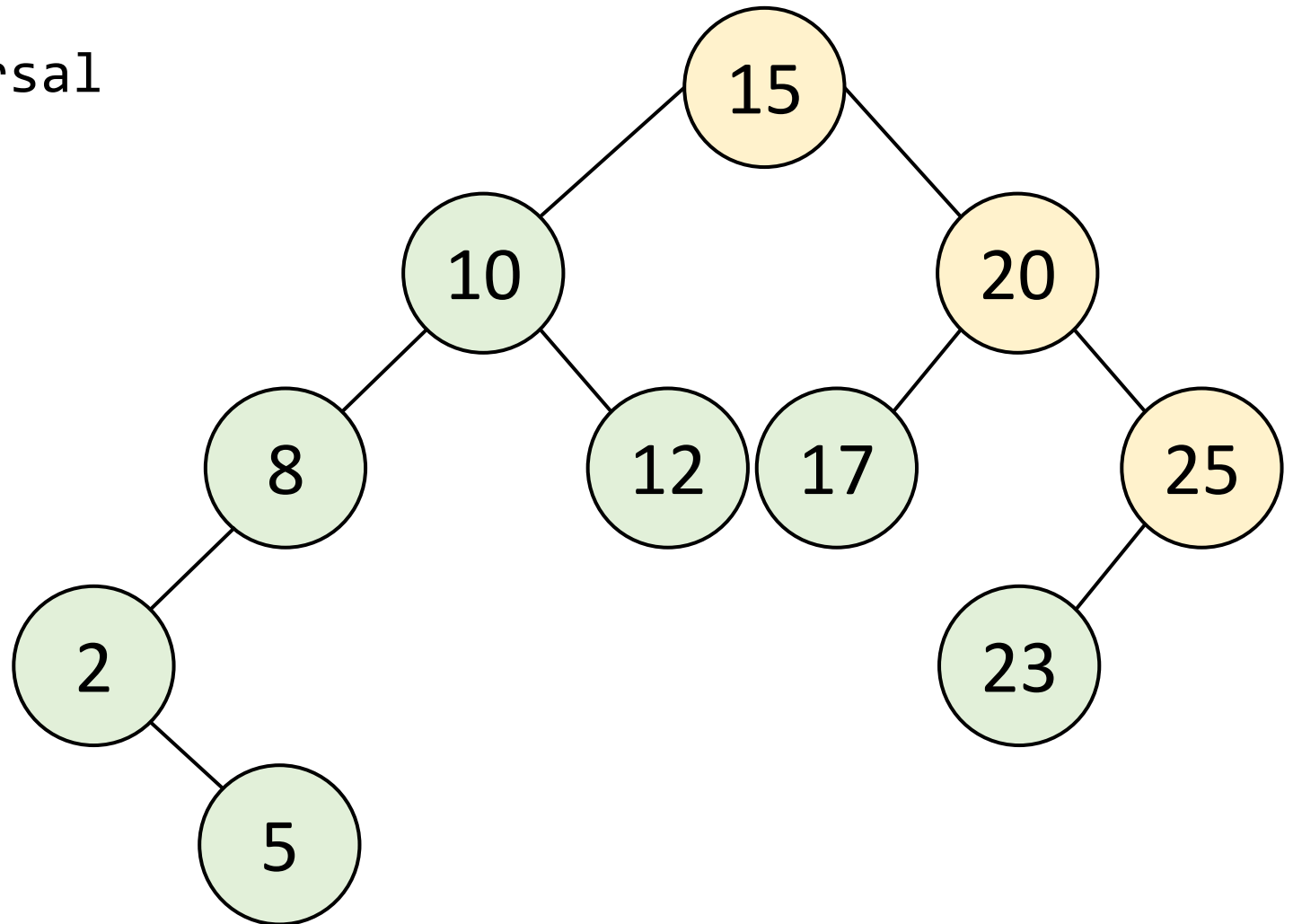
Let's use inorder traversal

Printing order will be:

2 5 8 10 12 15 17 20 23

Step:

Go back to 25



Print items of BST:

Idea:

Let's use inorder traversal

Printing order will be:

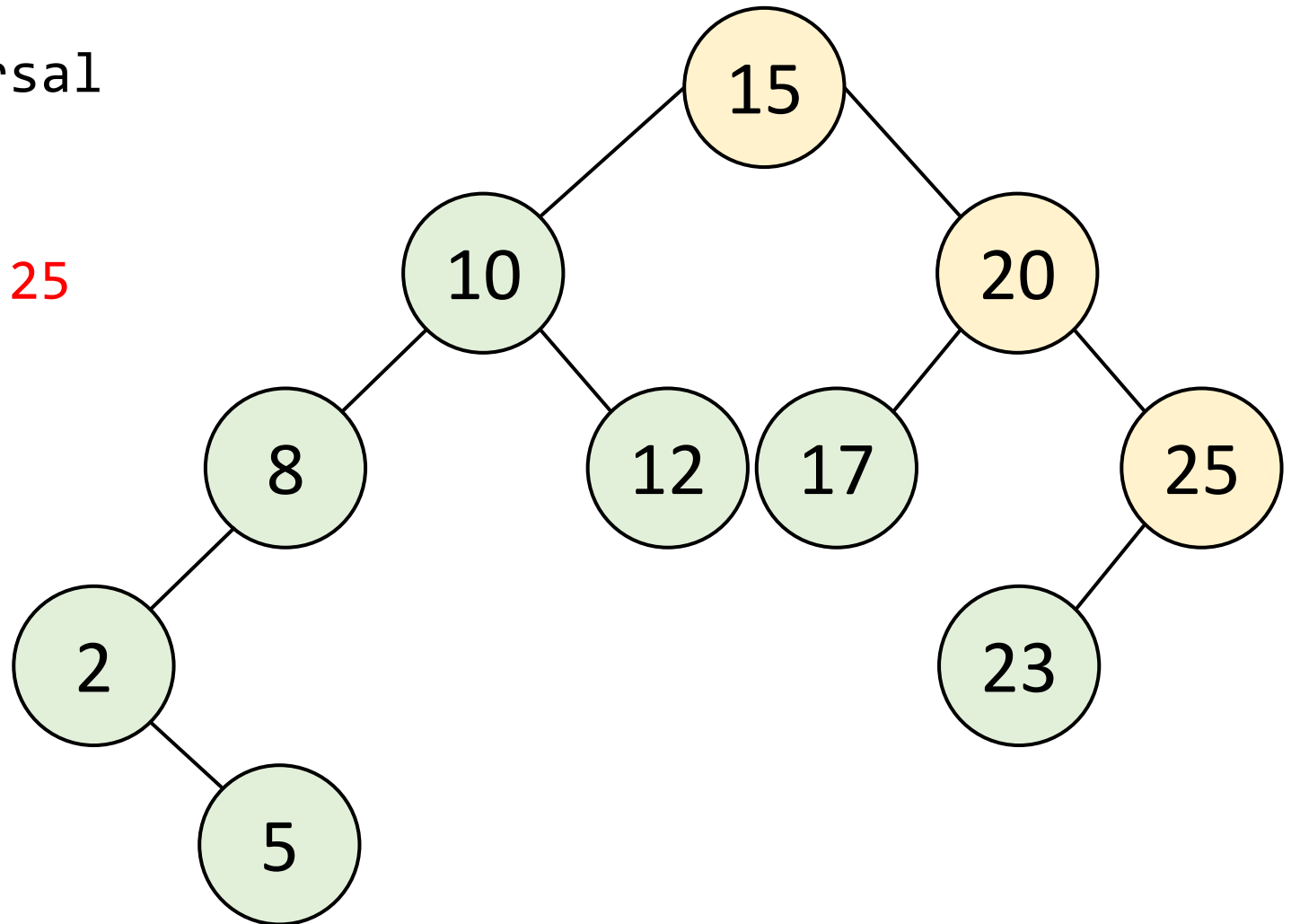
2 5 8 10 12 15 17 20 23 25

Step:

Print 25

Check 25->right

Go back to 20



Print items of BST:

Idea:

Let's use inorder traversal

Printing order will be:

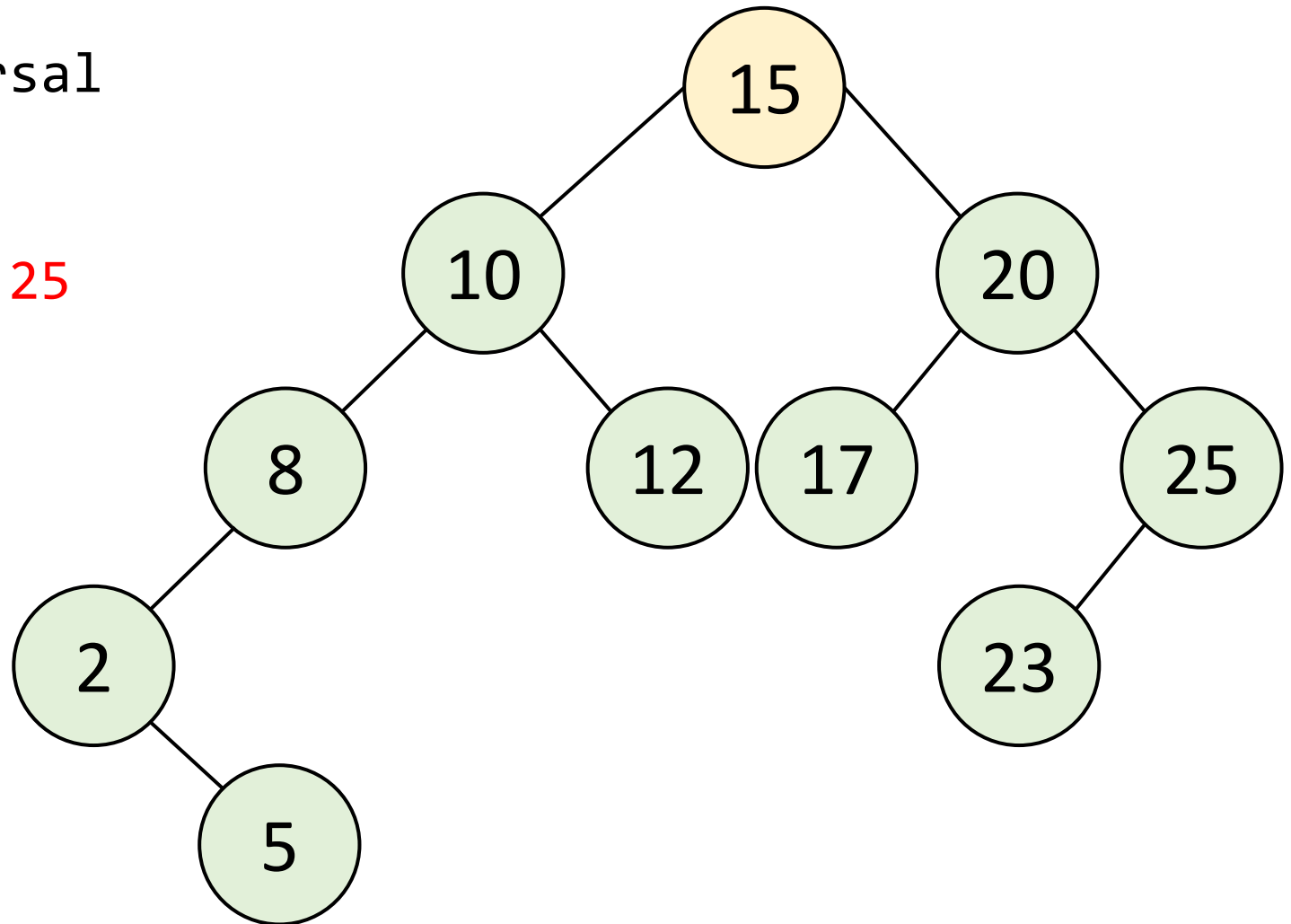
2 5 8 10 12 15 17 20 23 25

Step:

Go back to 15

F(15) all steps done

Procedure completed!



Print items of BST:

Idea:

Let's use inorder traversal

Printing order will be:

2 5 8 10 12 15 17 20 23 25

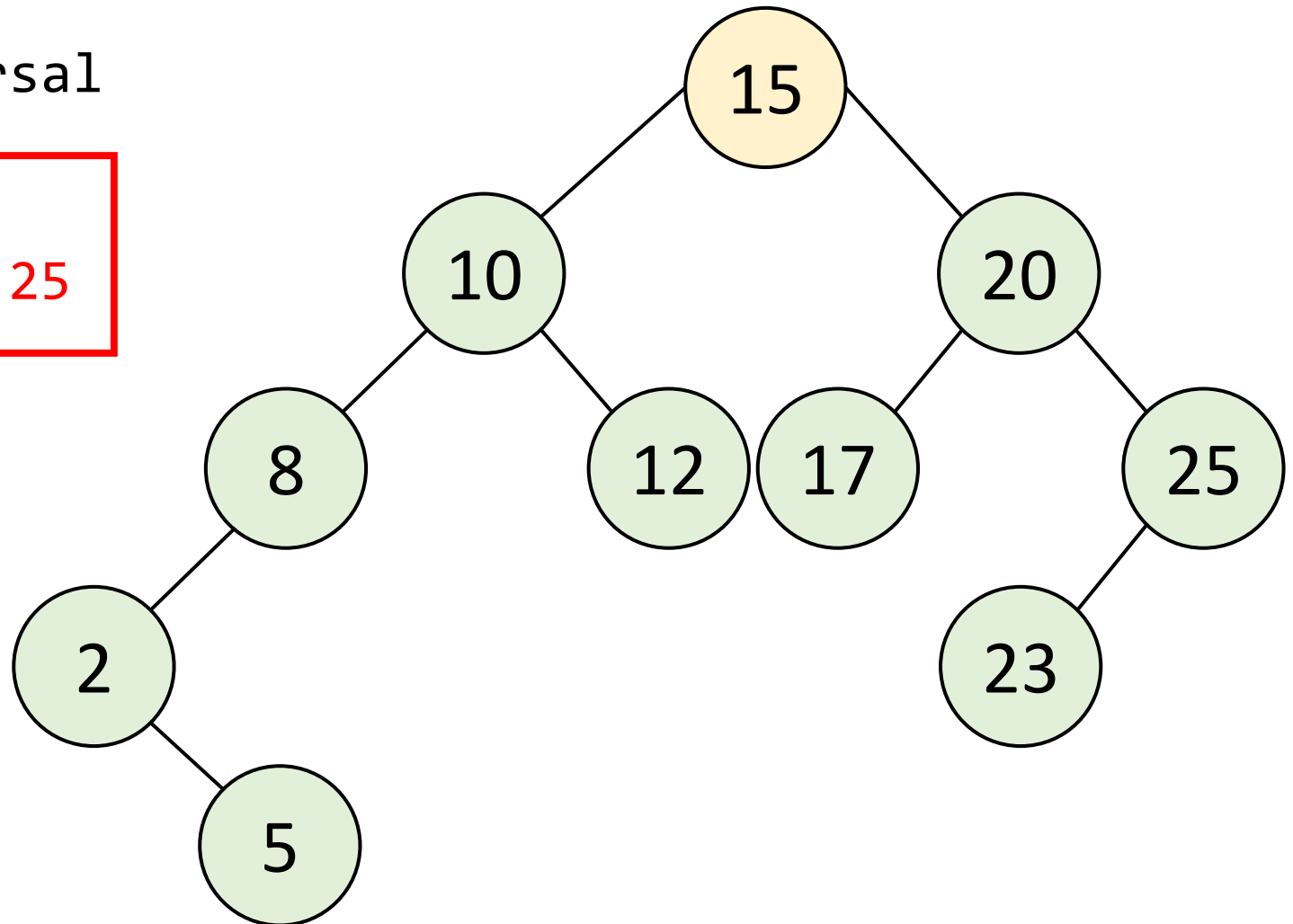
Step:

Go back to 15

F(15) all steps done.

Procedure completed!

Sorted
list!!!



```
void bst_sort(Node *x){
```

```
}
```

```
void bst_sort(Node *x){
```

?

```
    bst_sort(x->left)
```

```
    print x->data
```

```
    bst_sort(x-> right)
```

```
}
```

```
void bst_sort(Node *x){  
  
    if x == Null  
        return  
  
    bst_sort(x->left)  
    print x->data  
    bst_sort(x-> right)  
}
```

```
void bst_sort(Node *x){  
    if x == Null  
        return  
  
    bst_sort(x->left)  
    print x->data  
    bst_sort(x-> right)  
}
```

*Time
complexity?
 $O(N)$*

```
void bst_sort(Node *x){  
    if x == Null  
        return  
  
    bst_sort(x->left)  
    print x->data  
    bst_sort(x-> right)  
}
```

*Time
complexity?
 $O(N)$*

*Now we have a sorting algo
in $O(N)$!!!*

```

void bst_sort(Node *x){
    if x == Null
        return

    bst_sort(x->left)
    print x->data
    bst_sort(x-> right)
}

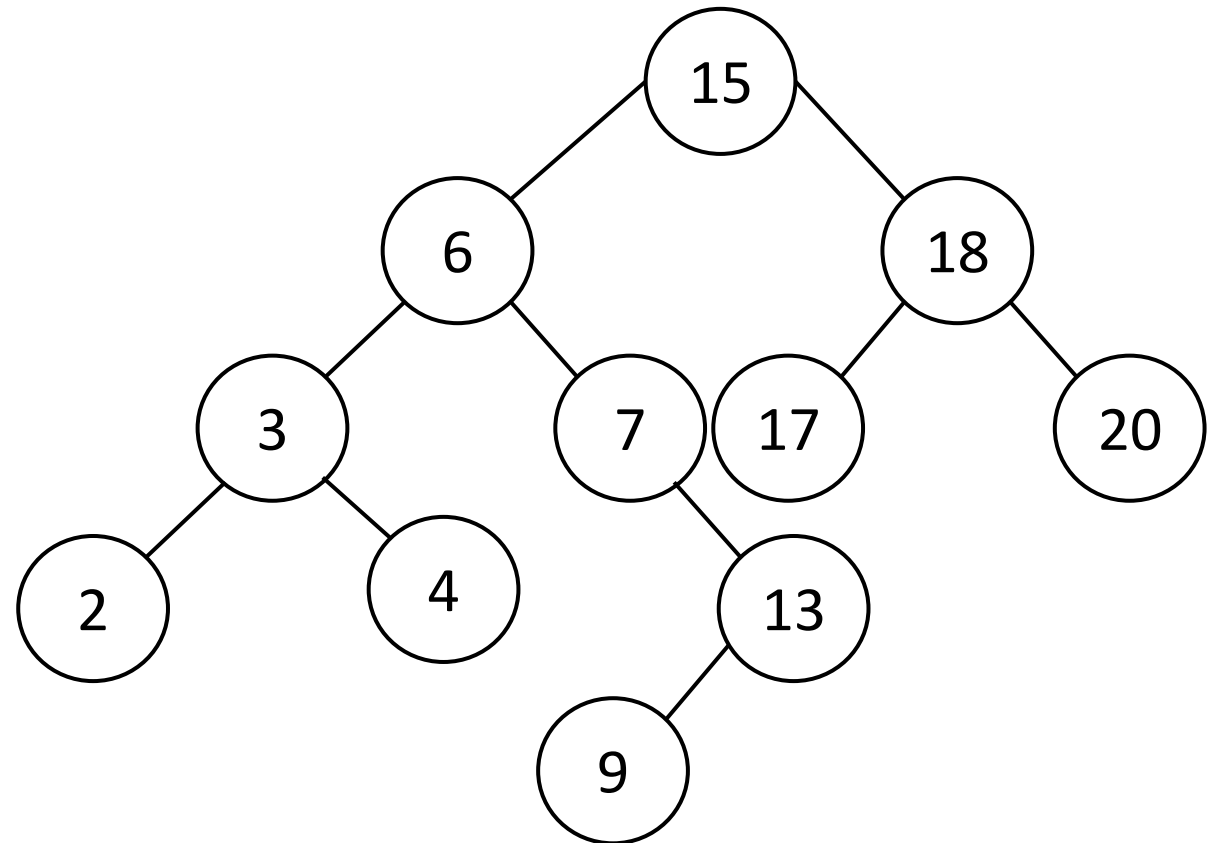
```

*Time
complexity?
 $O(N)$*

*~~Now we have a sorting algo
in $O(N)$!!!~~*

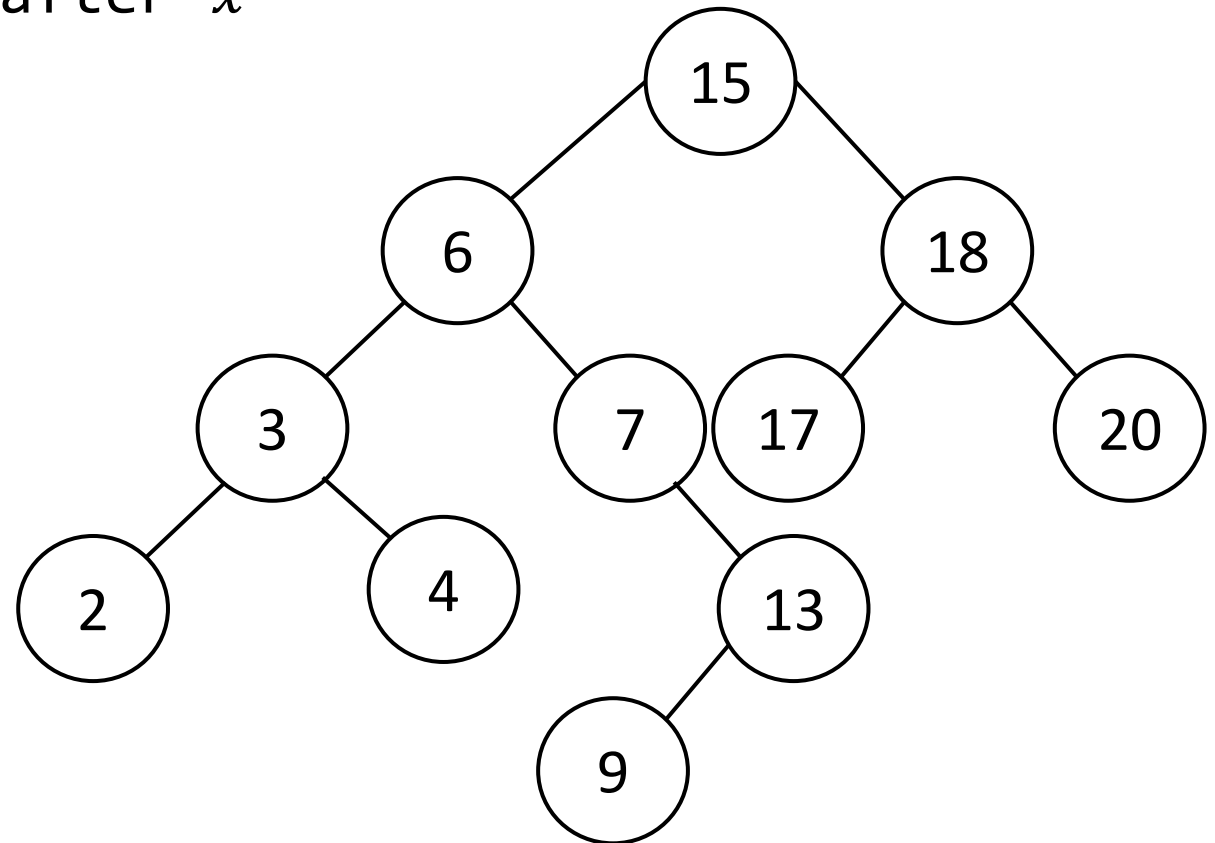
*Total time =
 $O(N \log N)$ for insertion +
 $O(N)$ for printing*

Find inorder successor of x in BST:



Find inorder successor of x in BST:

Inorder successor: item comes after x
during inorder traversal

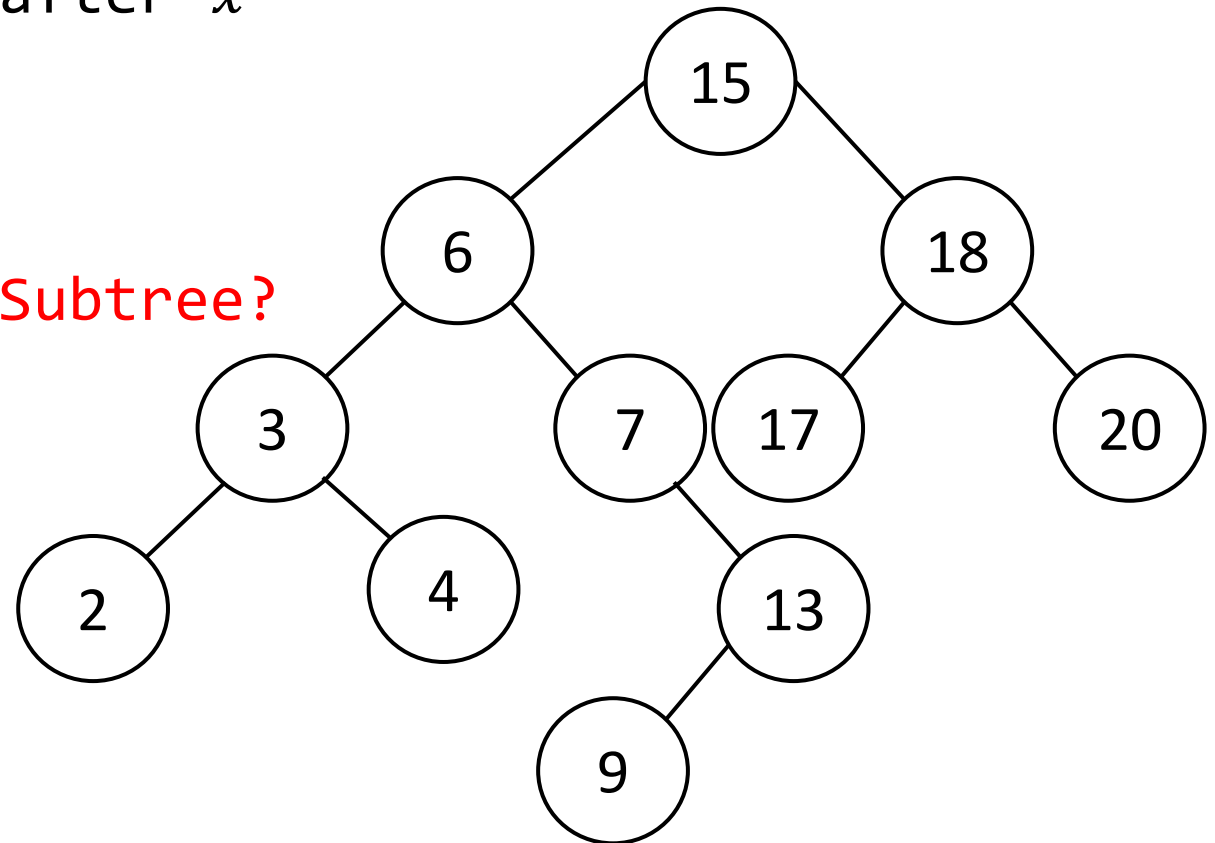


Find inorder successor of x in BST:

Inorder successor: item comes after x
during inorder traversal

Observations:

Can successor be found in leftSubtree?

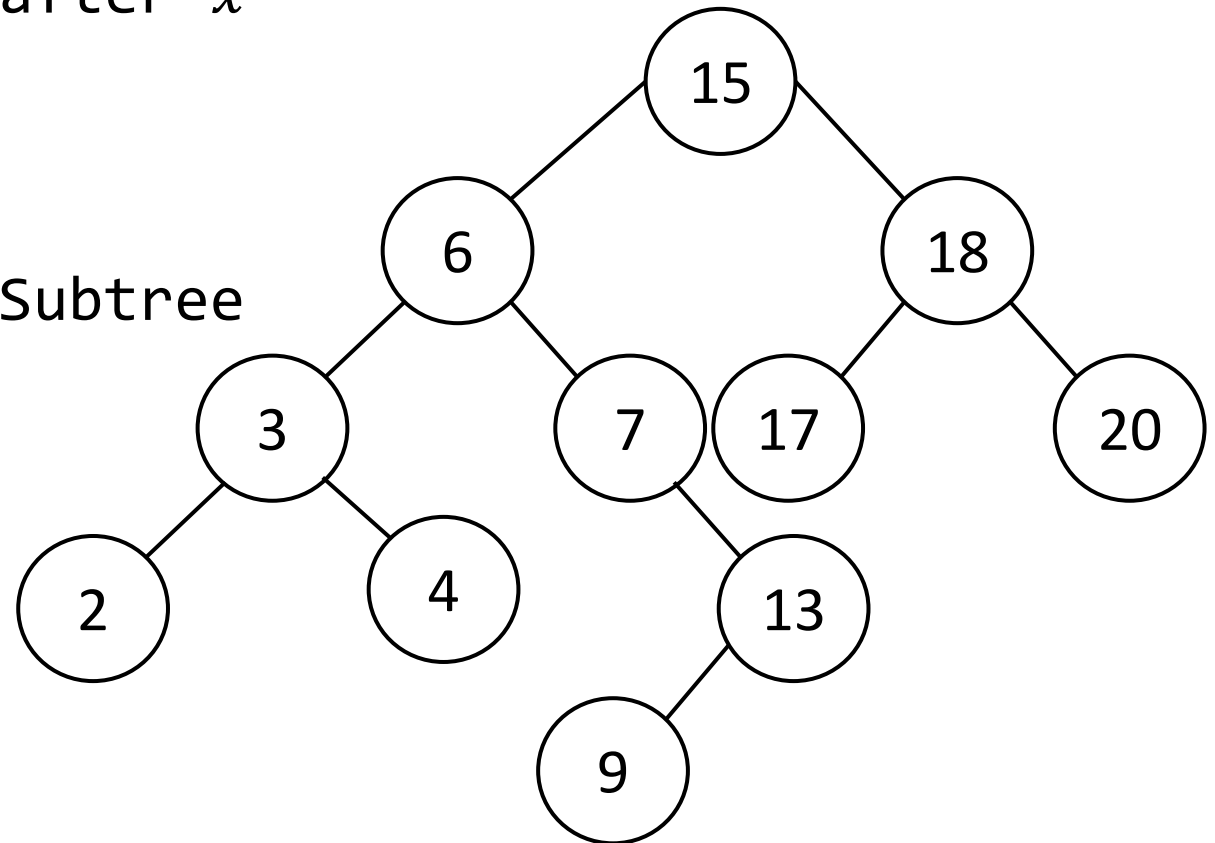


Find inorder successor of x in BST:

Inorder successor: item comes after x
during inorder traversal

Observations:

Successor can never be in leftSubtree

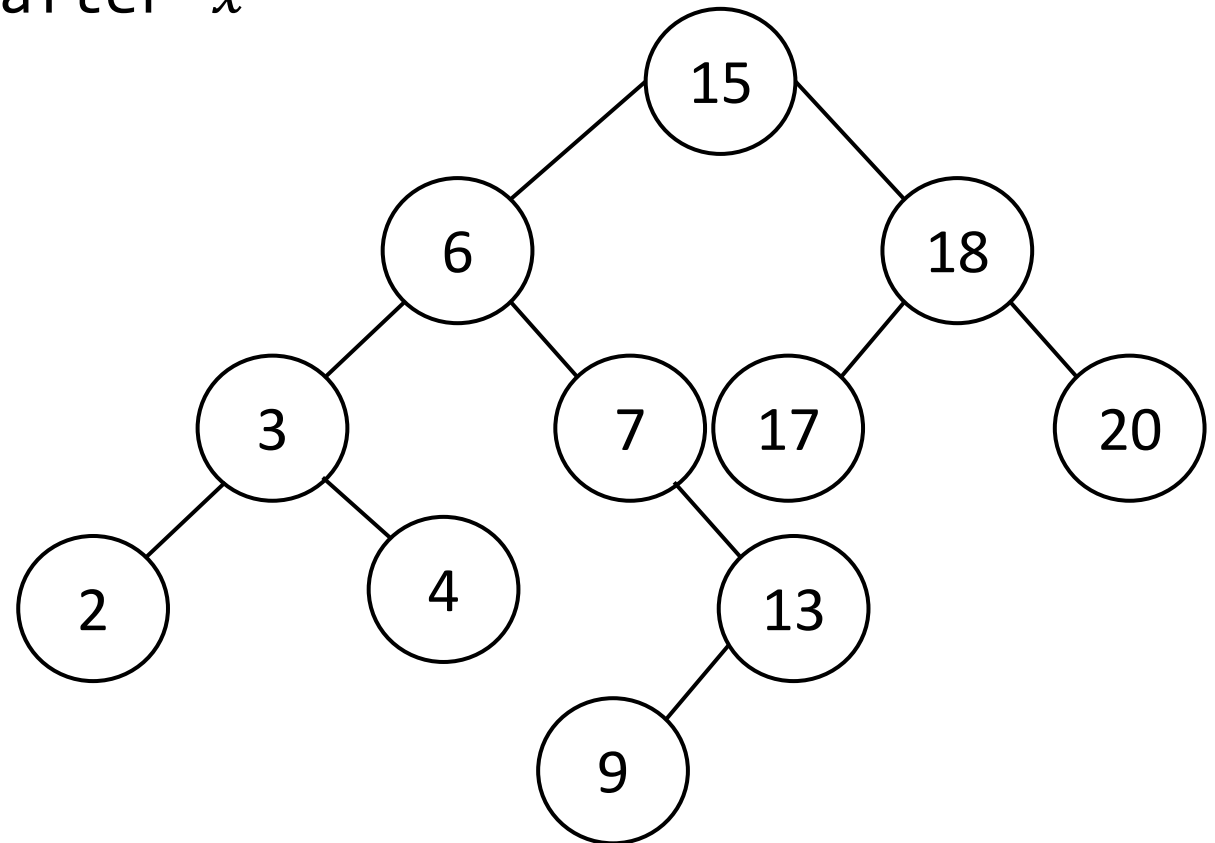


Find inorder successor of x in BST:

Inorder successor: item comes after x
during inorder traversal

Observations:

Successor(20): ?



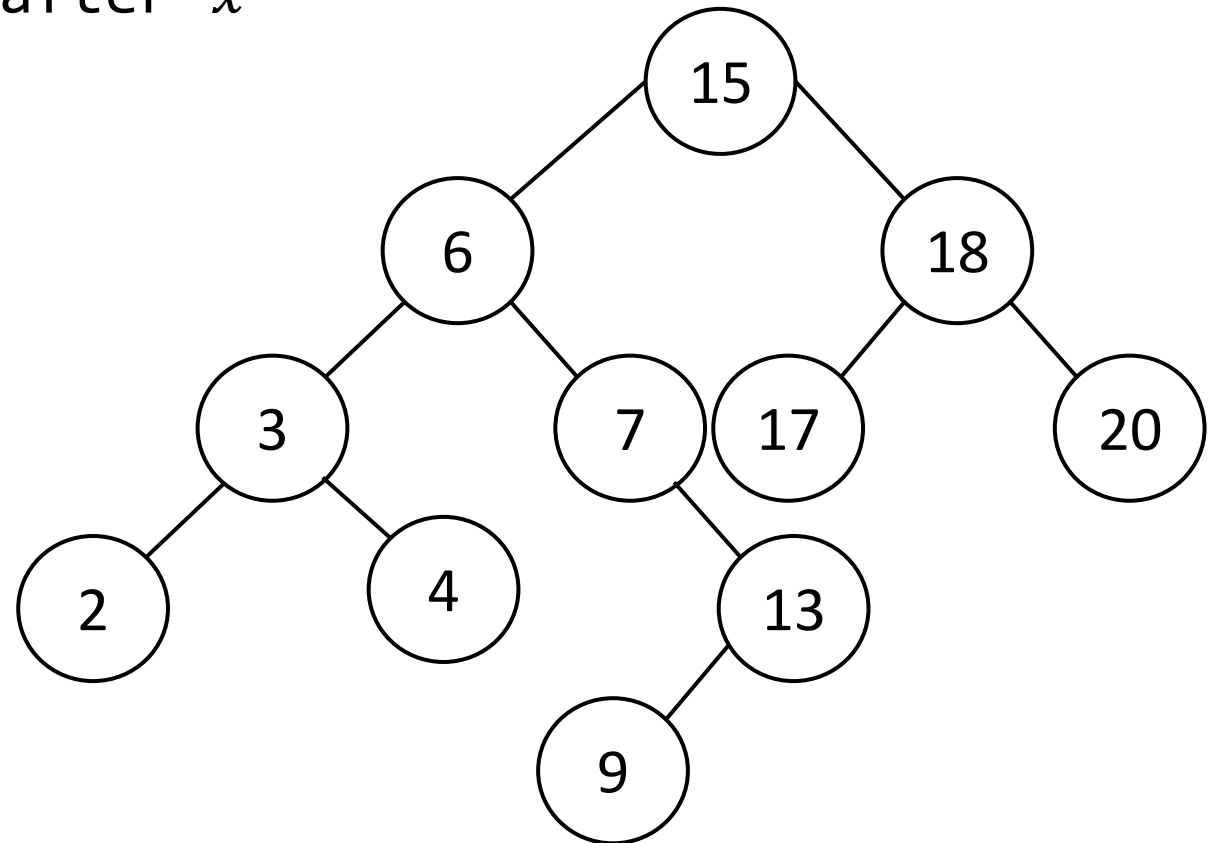
Find inorder successor of x in BST:

Inorder successor: item comes after x
during inorder traversal

Observations:

Successor(20): **Null**

Tree_max has no successor!!!



Find inorder successor of x in BST:

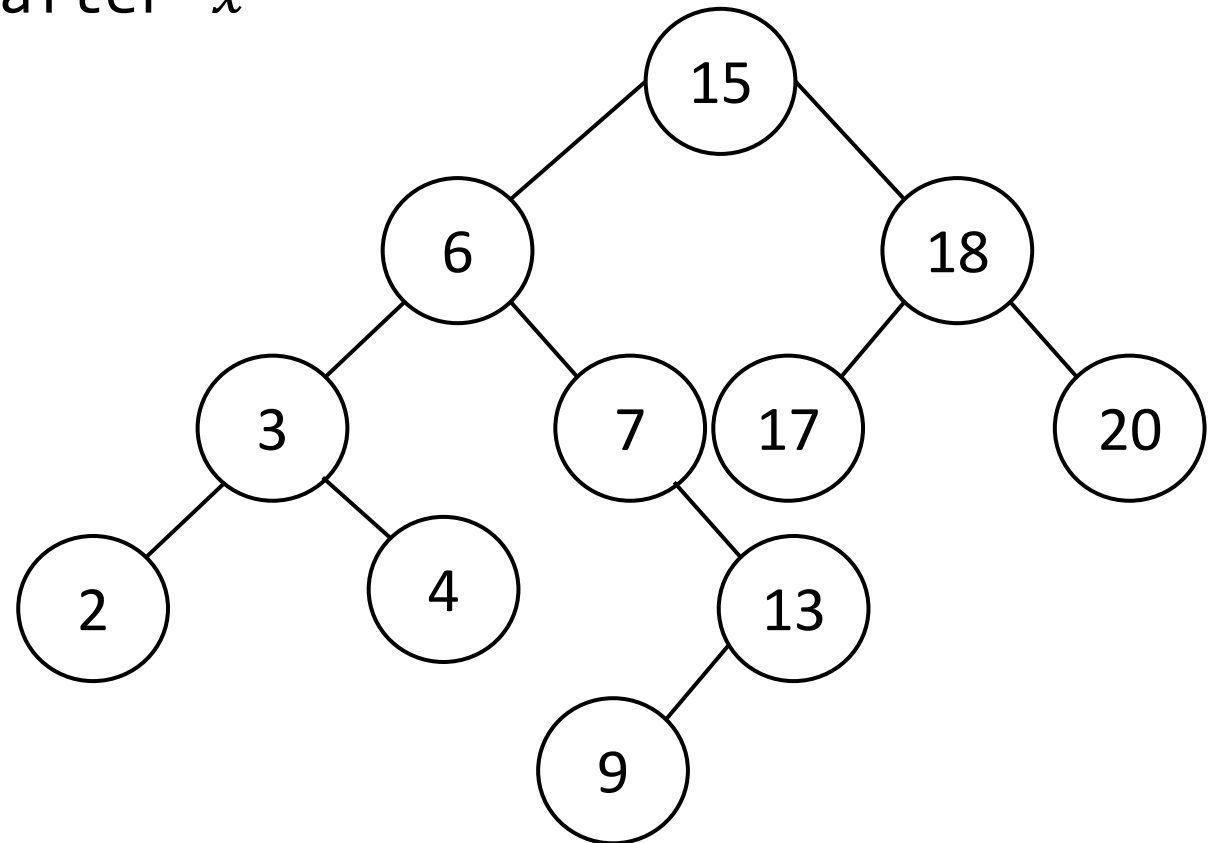
Inorder successor: item comes after x
during inorder traversal

Observations:

Successor(6): 7

Successor(15): 17

Successor(7): 9



Find inorder successor of x in BST:

Inorder successor: item comes after x during inorder traversal

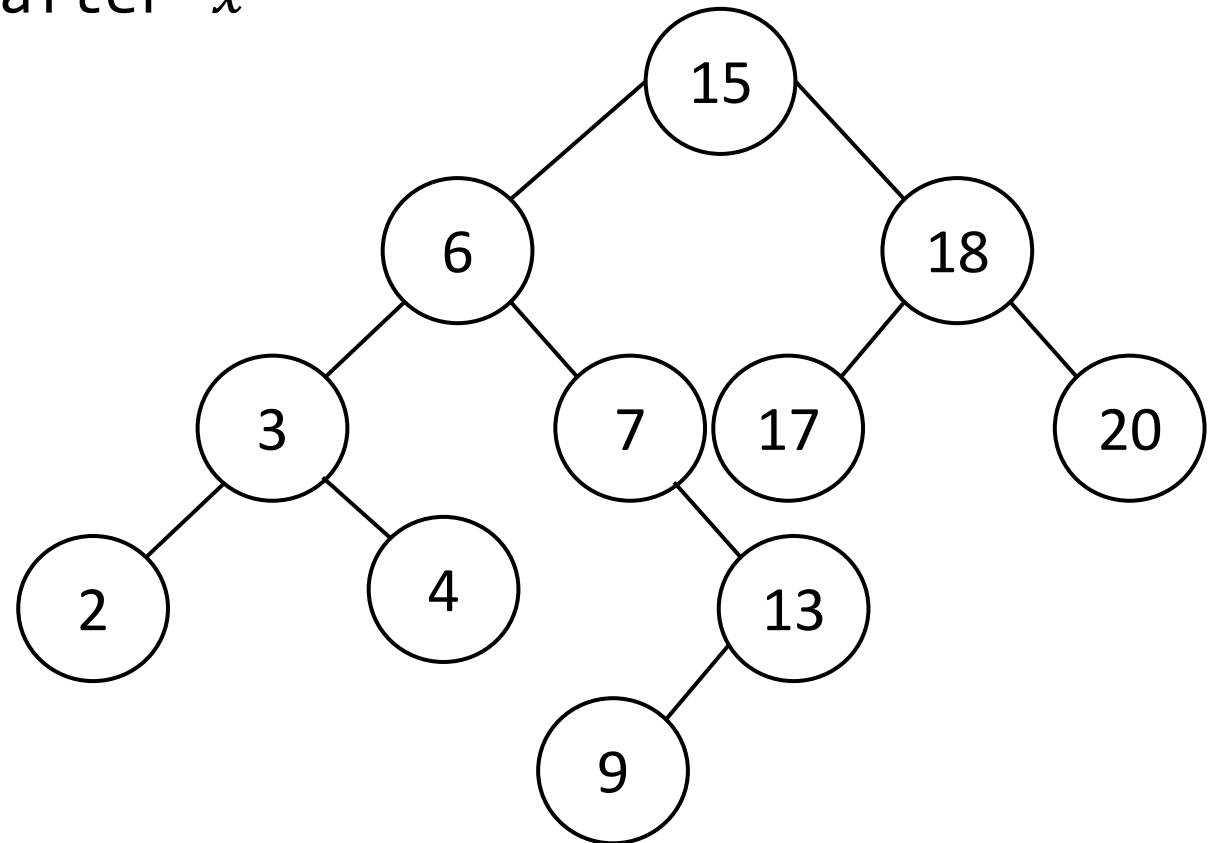
Observations:

Successor(6): 7

Successor(15): 17

Successor(7): 9

If there is a right subtree:
successor is the
minimum(right subtree)



Find inorder successor of x in BST:

Inorder successor: item comes after x during inorder traversal

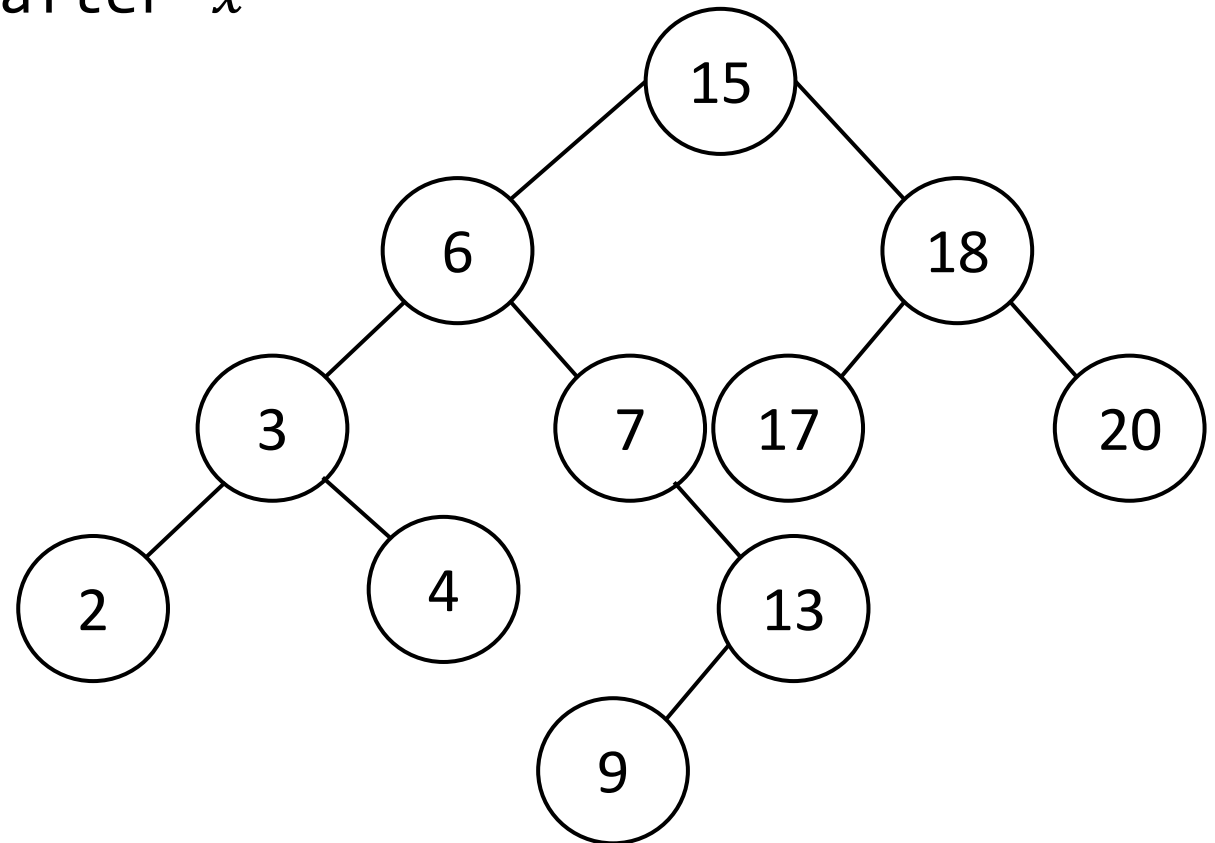
Observations:

Successor(9): 13

Successor(17): 18

Successor(4): 6

Successor(13): 15



Find inorder successor of x in BST:

Inorder successor: item comes after x during inorder traversal

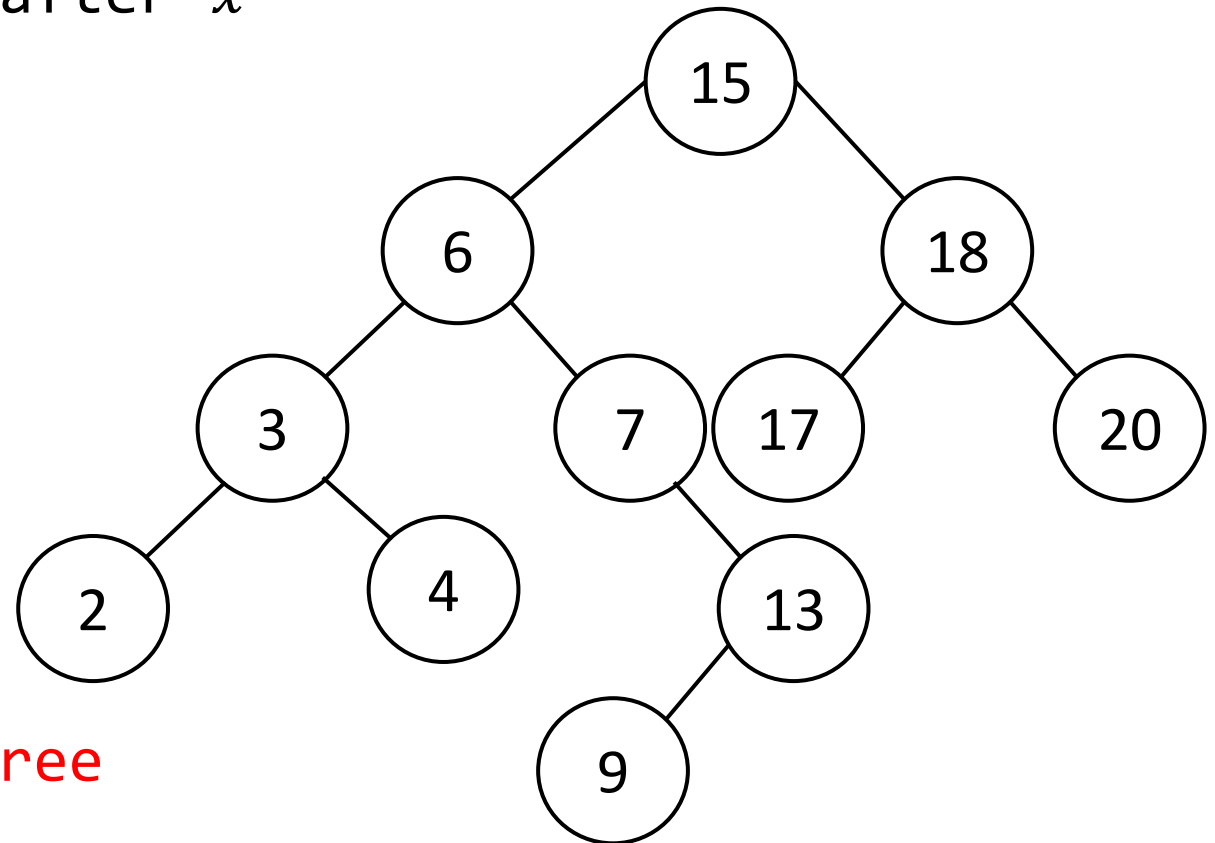
Observations:

Successor(9): 13

Successor(17): 18

Successor(4): 6

Successor(13): 15



If x doesn't have a right subtree
&& x is not the maximum value:
then successor(x) will be
one of its ancestors!

```
Node* successor(Node *x){
```

?

```
}
```

```
Node* successor(Node *x){  
    if    x == tree_maximum(root)  
        return Null
```

```
}
```

```
Node* successor(Node *x){  
    if    x == tree_maximum(root)  
        return Null  
  
    if    x->right != Null  
        return tree_minimum ( ? )  
  
}
```

```
Node* successor(Node *x){  
    if    x == tree_maximum(root)  
        return Null  
  
    if    x->right != Null  
        return tree_minimum (x->right)  
  
}
```

```
Node* successor(Node *x){  
    if    x == tree_maximum(root)  
        return Null  
  
    if    x->right != Null  
        return tree_minimum (x->right)  
  
    xp = x->parent  
    while ( xp->data < x->data)  
        xp = xp->parent  
  
    ?  
  
}
```

```
Node* successor(Node *x){  
    if    x == tree_maximum(root)  
        return Null  
  
    if    x->right != Null  
        return tree_minimum (x->right)  
  
    xp = x->parent  
    while ( xp->data < x->data)  
        xp = xp->parent  
  
    return xp  
}
```

Thank you